# DATABASE PROJECT REPORT

*Gym Membership & Personal Training Planner*

*(Project ID: #9)*

---

## A. PROJECT IDENTITY

| Team Name | Dong Vui |
|---|---|
| Project ID & Title | #9: Gym Membership & Personal Training Planner |
| Course | INT1313 - Database Basics |
| Instructor | Lê Hà Thanh |
| Academic Year | 2026-2027 (Semester 1) |

**Team Members:**


| Full Name | Student ID | Class | Institutional Email |
|---|---|---|---|
| Tran Duc Phuong | N24DCCN154 | D24CQCN02-N | n24dccn154@student.ptithcm.edu.vn |
| Nguyen Luu Trieu Quang | N24DCCN158 | D24CQCN02-N | n24dccn158@student.ptithcm.edu.vn |
| Nguyen Van Hoang Te | N24DCCN071 | D24CQCN01-N | n24dccn071@student.ptithcm.edu.vn |


### Table of Contents


**1. Introduction and Project Scope (adapted from ISO/IEC/IEEE 29148)..... 3**

1.1 System Objective and Requirements..... 3

1.2 Business Rules and Constraints..... 7

**2. Database Design (ISO/IEC 19505 / IE Standards)..... 19**
2.1 Conceptual Model (ER/EER Diagram)..... 19
2.2 Logical Schema Mapping..... 21
2.2.1 Mapping the EER Model to Tables (Elmasri & Navathe, Chapter 9: Steps 1-9)..... 21
2.2.2 Relational Schema (16 Normalized Tables)..... 29
2.2.3 Foreign Keys and Delete Rules..... 30
2.2.4 Physical Database Schema (Crow's Foot Notation)..... 32
2.3 Normalization Verification (1NF -> 2NF -> 3NF / BCNF -> 4NF)..... 34
2.3.1 Functional Dependencies and Normal Form Checks..... 35
2.3.2 Normal Forms Step by Step (1NF -> 2NF -> 3NF / BCNF)..... 37
2.3.3 Normalization Shown with FD Diagrams..... 37
2.3.4 BILLING_LOG: Exclusive Purchase Sources and BCNF..... 42
2.3.5 Fourth Normal Form (4NF)..... 42
2.3.6 Nullable Columns (NULL Policy)..... 43
**3. Data Dictionary (adapted from ISO/IEC 11179)..... 45**
3.1 Attribute Tables (16 Tables)..... 45
Table 1: MEMBER (Independent Strong Entity)..... 45
Table 2: DAILY_PASS_TICKET (Independent Strong Entity)..... 46
Table 3: MEMBERSHIP_PLAN (Plan Catalog)..... 47
Table 4: MEMBERSHIP (Member Contracts)..... 47
Table 5: BILLING_LOG (Exclusive Purchase Billing)..... 48
Table 6: STAFF (Superclass)..... 49
Table 7: TRAINER (Subclass of STAFF)..... 49
Table 8: RECEPTIONIST (Subclass of STAFF)..... 50
Table 9: ROOM (Training Rooms)..... 50
Table 10: SESSION (Superclass)..... 51
Table 11: CLASS (Subclass of SESSION)..... 51
Table 12: CLASS_REGISTRATION (Class Registrations)..... 52
Table 13: PT_BOOKING (Subclass of SESSION)..... 53
Table 14: EQUIPMENT (Machines)..... 53
Table 15: MAINTENANCE_LOG (Weak Entity: Service History)..... 54
Table 16: SESSION_EQUIPMENT_USAGE (Equipment Reservations)..... 55
3.2 Value Definitions for Status and Type Columns..... 56
**4. Database Implementation (SQL Style Guide Compliant)..... 60**
4.1 DDL Scripts (Tables, Views, Indexes, Triggers)..... 60
4.1.1 SQL Style Conventions..... 60
4.1.2 Tables..... 61
4.1.3 Views..... 61
4.1.4 Indexes..... 64

4.1.5 Triggers....64
4.1.6 Script List and Run Order.... 68
4.2 Advanced Queries and Performance Test Cases....68
4.2.1 Advanced Queries.... 69
4.2.2 Performance Test Cases: Larger Data Set and Index Effect.... 71
4.3 Java Application....72
**5. Verification and Security.... 77**
5.1 Test Cases: Constraints Prevent Bad Data....78
5.2 Role-Based Access Control (RBAC)....79
5.2.1 Roles and Principles.... 79
5.2.2 Permission Matrix.... 80
5.2.3 GRANT, DENY and REVOKE Statements....81
5.2.4 Access Tests....82
5.2.5 Views and the Java Application....83
**6. Conclusion.... 83**
6.1 Summary of the Design.... 83
6.2 Room for Extension.... 84
6.3 Known Limitations....84
**7. References.... 85**

# B. REPORT STRUCTURE

## 1. Introduction and Project Scope (adapted from ISO/IEC/IEEE 29148)

### 1.1 System Objective and Requirements

- **(Content)** In the modern fitness industry, managing memberships, tracking personal training schedules, and maintaining equipment manually often lead to data errors, scheduling conflicts, and operational inefficiency.
- **(Overall Objective)** To address these challenges, the Gym Membership and Personal Training Management System is designed to digitalize and streamline core gym operations.
- **(Specific Objectives**) The system aims to:
    - Manage member profiles, membership plans, and membership records in a centralized database.
    - Manage class schedules and personal trainer (PT) bookings while reducing scheduling conflicts among trainers, members, and rooms.

- Maintain fitness equipment and maintenance records to support facility management.
- Maintain accurate membership and billing records for reliable financial and membership tracking.
- Ensure data consistency and integrity across the database.

The requirements below are divided into Functional Requirements (FR) and Non-Functional Requirements (NFR), following the requirement structure of ISO/IEC/IEEE 29148:

- **Functional Requirements:**

  **FR-MEM (Member & Billing Domain): Record member registrations, independent daily-pass tickets, membership plans, membership contracts with start and end dates, and bills linked to exactly one membership or daily-pass ticket.**

  **FR-SCH (Trainer & Session Scheduling):** Manage group class schedules, private 1-on-1 PT bookings and room assignments, and use triggers to reject room or trainer schedule conflicts.

  **FR-FAC (Facility & Asset Maintenance):** Record equipment, keep a repair log for each machine (weak entity), and record which staff member signed each log.

- **Non-Functional Requirements:**

  **NFR-INT (Data Integrity): Keep the links between all 16 tables valid with primary and foreign keys, and use ON DELETE RESTRICT on the two foreign keys of BILLING_LOG so that no billing record is lost or left without its owner.**

  **NFR-PER (Query Performance):** Speed up frequent schedule lookups with indexes on the columns used for searching and for time ranges (e.g., Room_ID, Trainer_ID, Start_Time).

  **NFR-SEC (Data Protection):** Prevent duplicate accounts with UNIQUE constraints on phone and email, and keep member photos inside the database (VARBINARY), so they are covered by the same backups and permissions as the other data. Database roles also limit who can read or change each table (Section 5.2).

  **NFR-AVA (Recoverability):** The database can be restored from regular SQL Server backups; setting up the backup schedule is left to the gym's administrator (Section 6.3).

### 1.1.1 Main Entities and Their Meaning

- **Member:** A strong entity for registered gym members, identified by Member_ID. Full_Name, Phone_Number and Registration_Date are mandatory; Photo, Email and Birth_Date are optional.
- **Daily_Pass_Ticket**: An independent strong entity for one-day guests, identified by Ticket_ID. It neither inherits a key from MEMBER nor belongs to a specialization hierarchy.
- **Membership_Plan**: A strong entity defining the available plans, identified by Plan_ID; Plan_Name is unique and Duration_Months and Price must be positive.
- **Membership**: A strong entity identified by Membership_ID. Member_ID references MEMBER and Plan_ID references MEMBERSHIP_PLAN; Start_Date precedes End_Date.
- **Billing_Log:** A strong entity identified by Bill_ID. Membership_ID and Ticket_ID are nullable foreign keys, with an XOR CHECK enforcing exactly one non-NULL source and filtered unique indexes enforcing one billing record per purchase.

- **The Membership & Billing domain comprises five independent strong entities:** MEMBER, MEMBERSHIP_PLAN, MEMBERSHIP, DAILY_PASS_TICKET and BILLING_LOG.
- **Staff:** The superclass for all gym employees. Identified by Staff_ID; stores Full_Name, Phone, Role ('Trainer', 'Receptionist', 'Manager'), Hire_Date and monthly Salary (CHECK > 0).
- **Trainer / PT:** A subclass of Staff for personal trainers, with the attributes Specialty and Certification. Trainers lead group classes and coach 1-on-1 PT sessions.
- **Receptionist:** A subclass of Staff for front-desk staff, with the attribute Shift_Type (Morning, Evening, Night). Receptionists may sign equipment maintenance logs.
- **Manager:** A staff member with Role = 'Manager'. Managers have no attributes beyond those in Staff, so they have no subclass table; for this reason the specialization of Staff is partial. Managers may sign maintenance logs (BR-FAC-05).
- **Room:** A training room. Identified by Room_ID; stores Room_Name (UNIQUE), Capacity (CHECK > 0) and Floor_Location.
- **Session:** The superclass for every scheduled use of a room by a trainer. Identified by Session_ID; stores Session_Type ('Class' or 'PT'), Room_ID, Trainer_ID, Start_Time and End_Time, which both kinds of session share.
- **Class:** A subclass of Session for group classes (e.g., Zumba, Yoga, HIIT) scheduled by the gym. Inherits Session_ID (PK/FK) and adds Class_Name, Max_Capacity and Status ('Scheduled', 'Completed', 'Cancelled').
- **Class Registration:** The M:N relationship REGISTER between MEMBER and CLASS. The EER diagram draws it as one diamond; in the relational schema it becomes its own table, CLASS_REGISTRATION (Step 5), which records which member is registered for which class (linked to Member through Member_ID).
- **PT Booking:** A subclass of Session for private 1-on-1 coaching. Inherits Session_ID (PK/FK) and adds the member who made the booking (Member_ID) and Booking_Status; the trainer and the room are stored in Session.
- **Equipment:** Manages physical workout machines and assets. Identified by Equipment_ID (Primary Identifier), it tracks Equipment_Name, an optional Room_ID, an optional Purchase_Date, and Operational_Status ('Active', 'Under Maintenance', 'Broken'). If a machine is 'Broken' or 'Under Maintenance', the system rejects new reservations of it.
- **Maintenance Log:** A weak entity that records the service and repair history of one machine. Identified by the composite key {Equipment_ID, Log_no}, where Log_no is the partial key. Stores Maintenance_Date, Cost (CHECK >= 0), Description and Logged_By (the receptionist or manager who signed the log).
- **Session Equipment Usage:** The M:N relationship USES between SESSION and EQUIPMENT. It records which machine is reserved for which session (a group class or a PT booking). The EER diagram draws it as one diamond with the attributes Reserved_From and Reserved_To; in the relational schema it becomes its own table with the surrogate key Usage_ID (Step 5), and each row refers to exactly one session through Session_ID [FK, NOT NULL].

### 1.1.2 Technical Terms

- **Entity & Attribute:** An entity is a real-world object or concept that the database stores data about (e.g., MEMBER, EQUIPMENT). An attribute is one property of that entity (e.g., Phone, Email).
- **Enhanced Entity-Relationship (EER):** An extension of the ER model that adds superclasses and subclasses (specialization), attribute inheritance and weak entities.
- **Disjoint Specialization (d):** STAFF uses TRAINER and RECEPTIONIST subtypes; SESSION uses CLASS and PT_BOOKING subtypes. MEMBER and DAILY_PASS_TICKET are independent strong entities and are not a specialization.
- **Total & Partial Participation:** Total participation (double line) means that every entity must take part in the relationship (e.g., every MAINTENANCE_LOG must belong to an EQUIPMENT). Partial participation (single line) means that some entities may not take part (e.g., a new machine with no logs).
- **Strong Entity vs Weak Entity:** A strong entity has its own primary key. A weak entity (MAINTENANCE_LOG) cannot exist without its owner entity (EQUIPMENT); it is identified by its partial key (Log_no) together with the owner's primary key.
- **Composite Primary Identifier:** A key made of two or more attributes that together identify one row (e.g., {Session_ID, Member_ID} in CLASS_REGISTRATION).
- **Primary Key (PK) & Foreign Key (FK):** A primary key uniquely identifies each row of a table and cannot be NULL. A foreign key is a column whose value must match a primary key value in the referenced table (or be NULL when the column allows NULL).
- **CHECK & UNIQUE Constraints:** A CHECK constraint accepts a row only if a condition on that row's own columns is true (e.g., Price >= 0.00). A UNIQUE constraint rejects two rows with the same value in a column (e.g., Phone).
- **Trigger:** SQL code stored in the database that SQL Server runs automatically after an INSERT, UPDATE or DELETE on a table. This project uses AFTER triggers: the trigger checks a rule and, if the rule is broken, raises an error and rolls back the whole transaction that contains the statement. Triggers are used for rules that involve other rows or other tables, which a CHECK constraint cannot test.
- **Referential Actions (ON DELETE):** What happens to the child rows when a parent row is deleted. CASCADE deletes the child rows with the parent; NO ACTION (RESTRICT) rejects the delete while child rows exist; SET NULL keeps the child rows and sets their foreign key to NULL.
- **Filtered Unique Index:** A unique index that applies only to the rows that meet a condition. BR-MEM-10 uses two on BILLING_LOG (UX_BILL_Membership_NotNull and UX_BILL_Ticket_NotNull), so each membership and each daily-pass ticket has at most one bill.

### 1.1.3 Schedule Conflicts and Scope

A schedule assigns rooms, trainers and time slots. There are two kinds: the public class timetable published by the gym, and the private 1-on-1 PT slots of each trainer. A schedule conflict occurs when two sessions use the same room, or the same trainer, in overlapping time intervals [Start_Time, End_Time). Design note: group classes and PT sessions are generalized into one superclass SESSION, which holds the attributes both share (Room_ID, Trainer_ID, Start_Time, End_Time); CLASS and

PT_BOOKING are its disjoint subclasses. Room and trainer conflicts are therefore checked in one table (SESSION) by the triggers of script 02.

## 1.2 Business Rules and Constraints

Example identifiers such as M001 or MEM-501 are display labels only; stored Member_ID, Plan_ID, Membership_ID, Ticket_ID and Bill_ID keys are INT values.

### 1.2.1 Member and Membership Domain

**BR-MEM-01 (Unique Member Identification):**

- **Rule Statement:** Each registered Member must have a unique Member_ID. Full_Name, Phone_Number, and Registration_Date are mandatory, whereas Photo, Email, and Birth_Date are optional. A profile photo is not required before purchasing a membership.
- **Technical & EER Mapping:** The MEMBER entity uses Member_ID as its primary key. Mandatory attributes should be implemented using NOT NULL constraints, while optional attributes may accept NULL. If Phone_Number and Email are intended to be unique, their uniqueness must be enforced separately through UNIQUE constraints. The optional Photo attribute must not participate in membership-purchase eligibility validation.

**Concrete Example:** Member M001 may register with a full name, phone number, and registration date while leaving Photo, Email, and Birth_Date empty. The system must accept the registration and allow the member to purchase a membership.

**BR-MEM-02 (Membership Plan Definition):**

- **Rule Statement:** Each Membership Plan must have a unique Plan_ID and a unique Plan_Name. Duration_Months and Price must be greater than zero. A Membership Plan defines a standard package and is separate from an individual subscription record.
- **Technical & EER Mapping:** The MEMBERSHIP_PLAN entity uses Plan_ID as its primary key and enforces uniqueness on Plan_Name. Duration_Months and Price should be protected by NOT NULL and CHECK constraints requiring positive values. The plan stores reusable package information, while the MEMBERSHIP entity records an individual member's subscription. No additional Plan_Type attribute is required for this design.

**Concrete Example:** The plan “Basic Monthly” may have a duration of one month and a positive price. Multiple members can subscribe to this plan, with each subscription represented by a separate MEMBERSHIP record.

**BR-MEM-03 (Membership Association):**

- **Rule Statement:** Each Membership must reference exactly one registered Member and exactly one Membership Plan. Member_ID and Plan_ID must be non-null foreign keys.
- **Technical & EER Mapping:** The MEMBERSHIP entity contains Member_ID as a foreign key referencing MEMBER(Member_ID) and Plan_ID as a foreign key referencing MEMBERSHIP_PLAN(Plan_ID). Both foreign keys must be NOT NULL, ensuring that every membership belongs to an existing member and a valid plan. The relationship permits one member to have multiple membership records over time and one plan to be referenced by multiple membership records.

**Concrete Example:** Membership MS001 may reference Member M001 and Plan P001. The system must reject MS001 if either referenced record does not exist or either foreign key is missing.

**BR-MEM-04 (Membership Date Validity):**

- **Rule Statement:** Each Membership must have a defined Start_Date and End_Date, and Start_Date must be earlier than End_Date.
- **Technical & EER Mapping:** The MEMBERSHIP entity requires both date attributes to be non-null. A CHECK constraint should enforce Start_Date < End_Date. The application must calculate the end date according to the selected plan's duration and the system's defined date-calculation policy.

**Concrete Example:** A membership starting on January 1 and ending on February 1 is valid under an exclusive-end-date convention. A membership whose start date is equal to or later than its end date must be rejected.

**BR-MEM-05 (Membership Lifecycle and Historical Retention):**

- **Rule Statement:** A Membership status must be one of Active, Expired, or Cancelled. Expired and cancelled Membership records must be retained and must not be deleted solely because their validity period has ended.
- **Technical & EER Mapping:** The MEMBERSHIP entity stores its lifecycle status in a mandatory Status attribute. The allowed values should be enforced through a CHECK constraint or an equivalent database mechanism. The application must update the status when appropriate while preserving the original membership record. Status values must remain consistent with the membership's validity period and cancellation rules.

**Concrete Example:** When MS001 expires, its status may change from Active to Expired. Its member reference, plan reference, dates, and historical billing association must remain available for review.

**BR-MEM-06 (Single Active Membership):**

- **Rule Statement:** A Member may have at most one active Membership at any given time. An expired or cancelled Membership does not grant gym access.
- **Technical & EER Mapping:** The MEMBERSHIP entity references MEMBER through Member_ID. The system must prevent overlapping membership periods that grant access to the same member. This rule requires an appropriate database trigger, transaction-safe validation, or another mechanism supported by the selected DBMS. A simple UNIQUE(Member_ID) constraint on MEMBERSHIP is not appropriate because it would prevent members from retaining multiple historical memberships.

**Concrete Example:** If M001 has an active membership covering January 1 through January 31, the system must reject another membership for M001 that grants access during an overlapping period. A subsequent membership may be created for a later period without overwriting the earlier record.

**BR-MEM-07 (Daily Pass Ticket Validity and Uniqueness):**

- **Rule Statement:** Each Daily Pass Ticket must have a unique Ticket_ID, a guest name, a guest phone number, a visit date, and a positive daily rate. A guest may purchase at most one Daily Pass Ticket for the same date.
- **Technical & EER Mapping:** The DAILY_PASS_TICKET entity uses Ticket_ID as its primary key and stores Guest_Name, Guest_Phone, Visit_Date, and Daily_Rate. Mandatory attributes should use NOT NULL constraints, and Daily_Rate must be greater than zero. A composite UNIQUE(Guest_Phone, Visit_Date) constraint prevents multiple tickets with the same phone number and visit date. This assumes that the phone number is the chosen identifier for enforcing the rule; shared phone numbers may cause legitimate guests to be treated as the same person.

**Concrete Example:** If a ticket already exists for phone number 0901234567 on October 23, 2026, another ticket with the same phone number and date must be rejected. A ticket for a different date may be accepted if no other business rule is violated.

**BR-MEM-08 (Exclusive Ticket and Membership Access):**

- **Rule Statement:** A guest may not use a Daily Pass Ticket to obtain gym access during a period covered by an active Membership. If a Daily Pass Ticket is purchased before a Membership begins, the ticket remains valid until its own validity period ends, and the Membership must start afterward. Membership and Daily Pass access periods must not overlap for the same identifiable person.
- **Technical & EER Mapping:** The DAILY_PASS_TICKET entity does not contain Member_ID; therefore, foreign keys alone cannot enforce this cross-entity access rule. The application must validate the guest's identifying information, such as Guest_Phone, against member and membership records when issuing a ticket or creating a membership. Where necessary, a database trigger or transaction-safe validation mechanism may supplement the application logic. For a one-day ticket, the system must define its expiry boundary explicitly and calculate the subsequent membership start date accordingly. Phone-number matching is only a practical identification method and cannot guarantee that two records represent the same person.

**Concrete Example:** If a guest buys a one-day ticket valid for October 23, 2026, and then purchases a membership, the membership must begin no earlier than the end of that ticket's validity period. If the ticket expires at the end of October 23, the membership may begin on October 24. The original ticket must remain in the database as a historical record.

**BR-MEM-09 (Exclusive Billing Source):**

- **Rule Statement:** Each Billing Log record must reference exactly one billing source: either one Membership or one Daily Pass Ticket, but never both.
- **Technical & EER Mapping:** The BILLING_LOG entity contains nullable foreign keys Membership_ID and Ticket_ID, referencing MEMBERSHIP(Membership_ID) and DAILY_PASS_TICKET(Ticket_ID), respectively. A CHECK constraint must enforce that exactly one of these two foreign keys is non-null. The foreign keys ensure that the selected source exists, while the check constraint prevents both sources from being populated or both from being absent.

**Concrete Example:** A billing record for MS001 must contain Membership_ID = MS001 and Ticket_ID = NULL. A record containing both references, or neither reference, must be rejected.

**BR-MEM-10 (One Billing Record per Purchase):**

- **Rule Statement:** Each Membership and each Daily Pass Ticket must have exactly one associated Billing Log record. A payment attempt must update the existing billing record rather than create a duplicate record for the same purchase. Billing_Status may be Pending, Paid, or Failed.
- **Technical & EER Mapping:** BILLING_LOG uses Bill_ID as its primary key. Filtered unique indexes on non-NULL Membership_ID and Ticket_ID values prevent more than one billing record from referencing the same Membership or Daily Pass Ticket, respectively; BR-MEM-09's CHECK constraint separately ensures that exactly one source foreign key is populated. UNIQUE constraints alone do not guarantee that every Membership or Daily Pass Ticket has a billing record. The application must therefore create the purchase record and its Billing Log in the same database transaction, committing both together or rolling both back, or use an equivalent DBMS-supported enforcement mechanism. Payment retries update Billing_Status and the relevant payment information in the existing record. This design does not preserve a separate audit trail of individual payment attempts; that would require an additional entity.

**Concrete Example:** If billing record B001 is associated with Membership MS001 and has status Pending, a failed payment attempt may change its status to Failed. A later successful attempt may update the same record to Paid and set Payment_Date; the system must not create a second billing record for MS001. The system must also reject a purchase transaction if it cannot create the corresponding billing record atomically.

**BR-MEM-11 (Billing Amount and Payment Date Validity):**

- **Rule Statement:** Each Billing Log record must store a non-negative transaction amount and an invoice date. Payment_Date may be NULL while the transaction is Pending or Failed and must be recorded when the payment succeeds.
- **Technical & EER Mapping:** BILLING_LOG requires Amount_Paid and Invoice_Date to be NOT NULL. Amount_Paid must be non-negative and, under this simplified design, is set to the Membership Plan price or Daily Pass Ticket rate at the time of purchase; it is retained as the purchase-price snapshot even if the plan price changes later. Payment_Date may be NULL while Billing_Status is Pending or Failed and must be populated when Billing_Status is Paid. A CHECK constraint or equivalent validation should enforce this status/date relationship. The attribute name Amount_Paid is retained consistently across the schema and implementation.

**Concrete Example:** A billing record with status Pending, a non-negative amount, and no payment date is valid. Once payment succeeds, its status changes to Paid and its Payment_Date is recorded. A record marked Paid without a payment date must be rejected.

**BR-MEM-12 (Historical Record Retention and Deletion Restrictions):**

- **Rule Statement:** Member, Membership, Membership Plan, Daily Pass Ticket, and Billing Log records must not be hard-deleted when deletion would break referential integrity or destroy required financial history. Referenced records must be protected through ON DELETE RESTRICT or NO ACTION, as supported by the selected DBMS.
- **Technical & EER Mapping:** Foreign-key deletion policies must preserve membership and billing history. In particular, deleting a Membership or Daily Pass Ticket must not automatically delete its associated Billing Log or set the billing source foreign key to NULL. The BILLING_LOG foreign keys Membership_ID and Ticket_ID must use ON DELETE RESTRICT/NO ACTION, and Membership.Plan_ID and Membership.Member_ID must likewise protect referenced plan and member records. Membership Plans referenced by historical Membership records must not be hard-deleted. When a record should no longer be available for new transactions, use a lifecycle status (for example, Expired or Cancelled) or another non-destructive archival mechanism instead of deleting historical data. ON DELETE CASCADE is appropriate only for dependent non-financial rows where losing those rows is intended; it must not be used to erase purchase or billing history.

**Concrete Example:** When Membership MS001 expires, the system retains MS001 and its Billing Log B001. If MS001 is referenced by B001, a hard-delete request for MS001 must be rejected rather than deleting B001 or setting B001.Membership_ID to NULL. The same protection applies to a Member with historical memberships, a Daily Pass Ticket referenced by a billing record, and a plan referenced by a historical Membership.

**BR-MEM-13 (Membership Expiration and Renewal):**

- **Rule Statement:** A Membership must not grant access after its validity period ends or when its status is Expired or Cancelled. Members may renew their membership or purchase a new plan. Each new subscription period must be recorded as a separate MEMBERSHIP record with its corresponding BILLING_LOG record. A new membership may grant access only when payment is successful, its status permits access, and its start date has been reached.
- **Technical & EER Mapping:** The application validates Start_Date, End_Date, Status, and the associated billing status before granting access. When a member renews or purchases another plan, the system creates a new MEMBERSHIP record and a corresponding BILLING_LOG record rather than overwriting the previous subscription. The system must retain previous membership and billing records for historical tracking. Renewal is initiated by the member; automatic charging for subsequent subscription periods is outside the scope of this design.

**Concrete Example:** Membership MS001 ends on October 31, 2026, under the exclusive-end-date convention. The member purchases a new plan and receives Membership MS002 with its own billing record. MS002 must not grant access before its Start_Date or before its payment is marked Paid. MS001 and its billing history remain unchanged.

### 1.2.2 Training and Scheduling Domain

**Domain Requirements Analysis:**

**Guiding Questions**: To define the scheduling rules, the team analyzed the following questions:

- **Question for Personal Trainers**: *"Have you ever experienced schedule overlaps between group classes and 1-on-1 personal training sessions?"*
- **Question for Gym Managers**: *"How do room conflicts or workout equipment shortages affect customer satisfaction?"*

**Problem Analysis**: Based on common gym scheduling situations, two types of conflict were identified for peak hours (17:00 - 20:00): (1) the same room booked by a group class and a PT session at the same time, and (2) a trainer assigned to overlapping sessions. These situations led to rules BR-SCH-01 to BR-SCH-06 below.

**BR-SCH-01 (Trainer and Receptionist Subclasses)**

- **Rule Statement**: STAFF covers all employees. A staff member who coaches is modeled in the subclass TRAINER, and a front-desk staff member in the subclass RECEPTIONIST. A staff member's Role must match the subclass table that holds the row.
- **Technical & EER Mapping**: Disjoint, partial specialization STAFF (d) -> TRAINER / RECEPTIONIST, mapped with Option 8A. Shared attributes (name, salary) are in STAFF; trainer attributes (Specialty, Certification) are in TRAINER. The specialization is partial because a staff member with Role = 'Manager' has no extra attributes and therefore no subclass row. A trigger on TRAINER and RECEPTIONIST checks that STAFF.Role matches the subclass ('Trainer' for TRAINER, 'Receptionist' for RECEPTIONIST), and a trigger on STAFF rejects a change of Role that no longer matches an existing subclass row.

**Concrete Example**: Staff member S01 ('Le Van C') has Salary = $800 and Role = 'Trainer', and is stored in TRAINER as T01 with Specialty = 'Yoga' and Certification = 'NASM-CPT'. Inserting S01 into RECEPTIONIST is rejected, both because S01 is already a trainer (disjointness) and because S01's Role is 'Trainer'. A staff member with Role = 'Manager' cannot be inserted into TRAINER or RECEPTIONIST.

**BR-SCH-02 (Room and Class Capacity)**

- **Rule Statement**: Every group class (CLASS) and PT booking (PT_BOOKING) takes place in one ROOM during a set time interval. A class's Max_Capacity cannot exceed its room's Capacity, and the number of registrations holding a seat cannot exceed the class's Max_Capacity.
- **Technical & EER Mapping**: SESSION.Room_ID is a foreign key referencing ROOM(Room_ID), so classes and PT bookings use the same link to a room. CHECK (Capacity > 0) is declared on ROOM and CHECK (Max_Capacity > 0) on CLASS. Triggers on CLASS, SESSION and ROOM reject any insert or update that leaves a class's Max_Capacity above the Capacity of its session's room (SESSION.Room_ID). An AFTER INSERT/UPDATE trigger on CLASS_REGISTRATION counts the class's registrations whose Attendance_Status is not 'Cancelled' ('Confirmed' and 'Attended' both hold a seat) and rolls back the change if the count exceeds CLASS.Max_Capacity. If a session has no room yet (SESSION.Room_ID is NULL), the room-capacity check is skipped for that session.

**Concrete Example**: Studio A (Room_ID = R101) has Capacity = 20. Zumba class C101 is scheduled in R101 with Max_Capacity = 20. When member M021 submits the 21st registration for C101, the trigger rolls back the insert with the error 'BR-SCH-02: registration failed, class capacity reached.'

**BR-SCH-03 (No Room Double-Booking)**

- **Rule Statement**: A room cannot hold more than one session (class or PT booking) at overlapping times.
- **Technical & EER Mapping**: A trigger on SESSION compares the proposed interval [Start_Time, End_Time) with all other sessions in the same Room_ID, using the overlap condition (S1 < E2 AND S2 < E1). Because classes and PT bookings are both stored in SESSION, only one table is checked. If an overlap is found, the insert or update is rolled back. Sessions with no room (Room_ID IS NULL) are not compared. Sessions whose PT_BOOKING row has Booking_Status = 'Cancelled', or whose CLASS row has Status = 'Cancelled', are ignored; the check runs again when the status changes back from 'Cancelled'.

**Concrete Example**: Studio A is booked for Zumba class C101 from 09:00 to 10:30. An attempt to schedule PT booking B201 in Studio A from 10:00 to 11:00 is rejected.

**BR-SCH-04 (No Trainer Double-Booking)**

- **Rule Statement**: A trainer cannot lead a class and coach a PT session at overlapping times, and cannot take two PT bookings at the same time.
- **Technical & EER Mapping**: A trigger on SESSION compares the proposed interval [Start_Time, End_Time) with all other sessions of the same Trainer_ID, using (S1 < E2 AND S2 < E1). If an overlap is found, the change is rolled back with the error 'Trainer Schedule Conflict'. Sessions whose PT_BOOKING row has Booking_Status = 'Cancelled' are ignored; the check runs again when a booking's status changes back from 'Cancelled'.

**Concrete Example**: Trainer T01 leads Zumba class C102 from 17:00 to 18:30. Member M002 asks for a PT session with T01 at 18:00. The booking is rejected because the trainer is already busy.

**BR-SCH-05 (Who Can Register for a Class or Book PT)**

- **Rule Statement:** Only members with an active membership (Status = 'Active' and Start_Date <= today AND today < End_Date) can register for a class or book a PT session.
- **Technical & EER Mapping:** The foreign key CLASS_REGISTRATION.Member_ID references MEMBER(Member_ID). An AFTER INSERT trigger checks that the member has a row in MEMBERSHIP with Status = 'Active' and Start_Date <= today AND today < End_Date; otherwise the insert is rolled back. An AFTER INSERT trigger on PT_BOOKING applies the same check to the booking member.

**Concrete Example:** Member M001 has the active membership MEM-501 and registers for Zumba class C101 successfully. Daily pass holder T002 tries to register for C101; the foreign key rejects it because T002 does not exist in MEMBER. Member M099, whose membership has expired, tries to register; the trigger rejects the insert because M099 has no membership with Status = 'Active' that covers today's date.

**BR-SCH-06 (Session Types and Equipment Reservations)**

- **Rule Statement**: Every session (SESSION) is either a group class (CLASS) or a PT booking (PT_BOOKING), never both (total and disjoint: SESSION = CLASS ∪ PT_BOOKING and CLASS ∩ PT_BOOKING = ∅). When a session reserves equipment, the reservation is stored in SESSION_EQUIPMENT_USAGE and refers to that one session.
- **Technical & EER Mapping**: A total, disjoint specialization (d) of the superclass SESSION(Session_ID) into the subclasses CLASS(Session_ID) and PT_BOOKING(Session_ID), mapped with Option 8A. Session_Type ('Class' or 'PT') records the subclass of each session. A trigger checks that a CLASS row refers to a session with Session_Type = 'Class' and a PT_BOOKING row to a session with Session_Type = 'PT', so a session cannot appear in both subclasses; an UPDATE trigger on SESSION blocks changes to Session_Type once the subclass row exists. Each SESSION row and its subclass row are inserted in the same transaction (total participation). SESSION_EQUIPMENT_USAGE links Equipment_ID to one Session_ID [FK -> SESSION, NOT NULL], so it needs no nullable foreign keys.

**Concrete Example**: Pilates class C105 is stored as SESSION C105 (Session_Type = 'Class') and CLASS C105. It reserves the Pilates Reformer machine E501, so the reservation row stores Session_ID = C105. An attempt to also insert C105 into PT_BOOKING is rejected by the trigger.

### 1.2.3 Detecting Schedule Conflicts: Boundary Cases

**Interval Overlap Condition**: Two time intervals [S1, E1) and [S2, E2) overlap if and only if (S1 < E2 AND S2 < E1).

**Boundary Case A (Adjacent Non-Overlapping Sessions)**: Session 1 runs from 09:00:00 to 10:30:00 (E1 = 10:30). Session 2 is requested from 10:30:00 to 11:30:00 (S2 = 10:30). Checking the condition: (09:00 < 11:30) is TRUE, but (10:30 < 10:30) is FALSE. Both parts must be true, so Overlap = FALSE. Session 2 is ACCEPTED.

**Boundary Case B (1-Minute Overlap Conflict)**: Session 1 runs from 09:00:00 to 10:30:00. Session 2 is requested from 10:29:00 to 11:30:00. Checking: (09:00 < 11:30) is TRUE, and (10:29 < 10:30) is TRUE. Overlap = TRUE. Session 2 is REJECTED.

### 1.2.4 Facility and Maintenance Domain

**BR-FAC-01 (Equipment Records)**

- **Rule Statement**: Fitness equipment (Equipment) is a strong entity uniquely identified by Equipment_ID, tracking Equipment_Name, an optional Room_ID, an optional Purchase_Date, and Operational_Status ('Active', 'Under Maintenance', 'Broken').
- **Technical & EER Mapping**: Governs physical assets. Attributes: Equipment_ID (PK), Equipment_Name, Room_ID (FK, nullable), Purchase_Date (nullable), Operational_Status (CHECK Operational_Status IN ('Active', 'Under Maintenance', 'Broken')).

**Concrete Example**: A NordicTrack treadmill is stored with Equipment_ID = E101 and Operational_Status = 'Active' in room R102.

**BR-FAC-02 (Maintenance Log Is a Weak Entity)**

- **Rule Statement**: A maintenance log (Maintenance_Log) cannot exist without its machine; it is a weak entity with the partial key Log_no.
- **Technical & EER Mapping**: The logs of one machine are numbered 1, 2, 3, ... and depend on that machine. In the EER diagram the weak entity is drawn as a double rectangle.

**Concrete Example**: Log #1 of treadmill E101 is identified together with E101 and cannot exist without it. (Log_no has no DEFAULT value, so the application must give each log its number; the primary key {Equipment_ID, Log_no} rejects a second log with the same number for the same machine.)

**BR-FAC-03 (Composite Key of the Log)**

- **Rule Statement**: The key of Maintenance_Log combines the owner's key and the partial key: {Equipment_ID, Log_no}.
- **Technical & EER Mapping**: The primary key {Equipment_ID, Log_no} lets the same machine have many repair records without duplicate keys.

**Concrete Example**: Treadmill E101 is repaired in January, May and August. The records are {E101, 1}, {E101, 2} and {E101, 3}. (Log_no has no DEFAULT; the application sets it to MAX(Log_no) + 1 for that machine when a new log is added.)

**BR-FAC-04 (Participation)**

- **Rule Statement**: The relationship LOGGED_FOR between Equipment and Maintenance_Log has total participation (double line) on the Maintenance_Log side and partial participation (single line) on the Equipment side.
- **Technical & EER Mapping**: Total participation of Maintenance_Log means that every log must reference an existing Equipment_ID. Partial participation of Equipment allows a new machine to have no logs.

**Concrete Example**: A newly bought Cable Crossover machine E205 has no maintenance logs, which is valid. An attempt to insert a maintenance log for Equipment_ID = E999, which does not exist, is rejected.

**BR-FAC-05 (Who Signs a Maintenance Log)**

- **Rule Statement**: Every maintenance log is signed by exactly one staff member, who must be a Receptionist or a Manager, through the foreign key Logged_By.
- **Technical & EER Mapping**: Logged_By references STAFF(Staff_ID). A trigger checks that the signing staff member has Role IN ('Receptionist', 'Manager').

**Concrete Example**: Receptionist R01 (Staff_ID = S02) signs maintenance log {E101, 1} after the technician replaces the belt, so Logged_By = S02.

**BR-FAC-06 (Out-of-Service Equipment)**

- **Rule Statement**: If a machine's status is 'Broken' or 'Under Maintenance', the database rejects any new reservation of that machine for a session. Existing reservations are not changed automatically.
- **Technical & EER Mapping**: Enforced by a trigger on SESSION_EQUIPMENT_USAGE (trg_SEU_EquipmentActive). Future reservations that already use the machine are not changed automatically; staff can list them with a query and move them to another machine.

**Concrete Example**: Leg press E301 is set to 'Broken'. New reservations of E301 are rejected, and a query on SESSION_EQUIPMENT_USAGE lists the future sessions that still use E301.

**BR-FAC-07 (Maintenance Cost)**

- **Rule Statement**: The Cost in each maintenance log cannot be negative (CHECK (Cost >= 0.00)).
- **Technical & EER Mapping**: Cost has the default value 0.00, which is used for repairs covered by the manufacturer's warranty.

**Concrete Example**: Routine lubrication of treadmill E101 under warranty is recorded with Cost = 0.00. Entering Cost = -50.00 violates the CHECK constraint.

**BR-FAC-08 (Equipment Reservation)**

- **Rule Statement**: A reservation links exactly one machine to exactly one session. The same machine can be reserved for many sessions, and a session can reserve many machines.
- **Technical & EER Mapping**: M:N relationship between EQUIPMENT and SESSION (USES in the EER diagram), mapped to the table SESSION_EQUIPMENT_USAGE with its own key Usage_ID. Equipment_ID and Session_ID are NOT NULL foreign keys, both ON DELETE CASCADE. The pair (Equipment_ID, Session_ID) is not unique, because the same machine can be reserved twice in one session at different times.

**Concrete Example**: Treadmill E101 is reserved for Yoga session S10 from 09:00 to 09:30 and again from 09:45 to 10:00. These are two rows with two different Usage_ID values.

**BR-FAC-09 (Reservation Time Window)**

- **Rule Statement**: Each reservation has a start time and an end time. The end time must be later than the start time, and the whole window must lie inside the time of its session.
- **Technical & EER Mapping**: Reserved_From and Reserved_To are NOT NULL, with CHECK (Reserved_To > Reserved_From). A trigger on SESSION_EQUIPMENT_USAGE and a trigger on SESSION reject a window that falls outside the session.

**Concrete Example**: Session S10 runs from 09:00 to 10:00. Reserving E101 from 09:30 to 10:30 is rejected, because it ends after the session.

**BR-FAC-10 (No Double Booking of a Machine)**

- **Rule Statement**: The same machine cannot be reserved for two overlapping time windows. Reservations that belong to a cancelled session (a cancelled PT booking or a cancelled class) are ignored.
- **Technical & EER Mapping**: A trigger on SESSION_EQUIPMENT_USAGE compares each new or changed reservation with the other reservations of the same Equipment_ID. Two windows overlap when each starts before the other ends.

**Concrete Example**: E101 is reserved from 09:00 to 09:30 for session S10. Reserving E101 from 09:15 to 09:45 for session S11 is rejected. Reserving it from 09:30 to 10:00 is accepted.

# 2. Database Design (ISO/IEC 19505 / IE Standards)

This section describes the database design in three steps: the conceptual EER model (Section 2.1), its mapping to a relational schema (Section 2.2), and the normalization proofs (Section 2.3). The course template names ISO/IEC 19505 (UML) and Information Engineering (IE) notation for the design diagrams. This report draws the conceptual model in EER notation (Elmasri and Navathe, 7th ed.) and the physical schema in IE crow's foot notation.

## 2.1 Conceptual Model (ER/EER Diagram)

The conceptual model uses Enhanced Entity-Relationship (EER) notation. Its main features are:

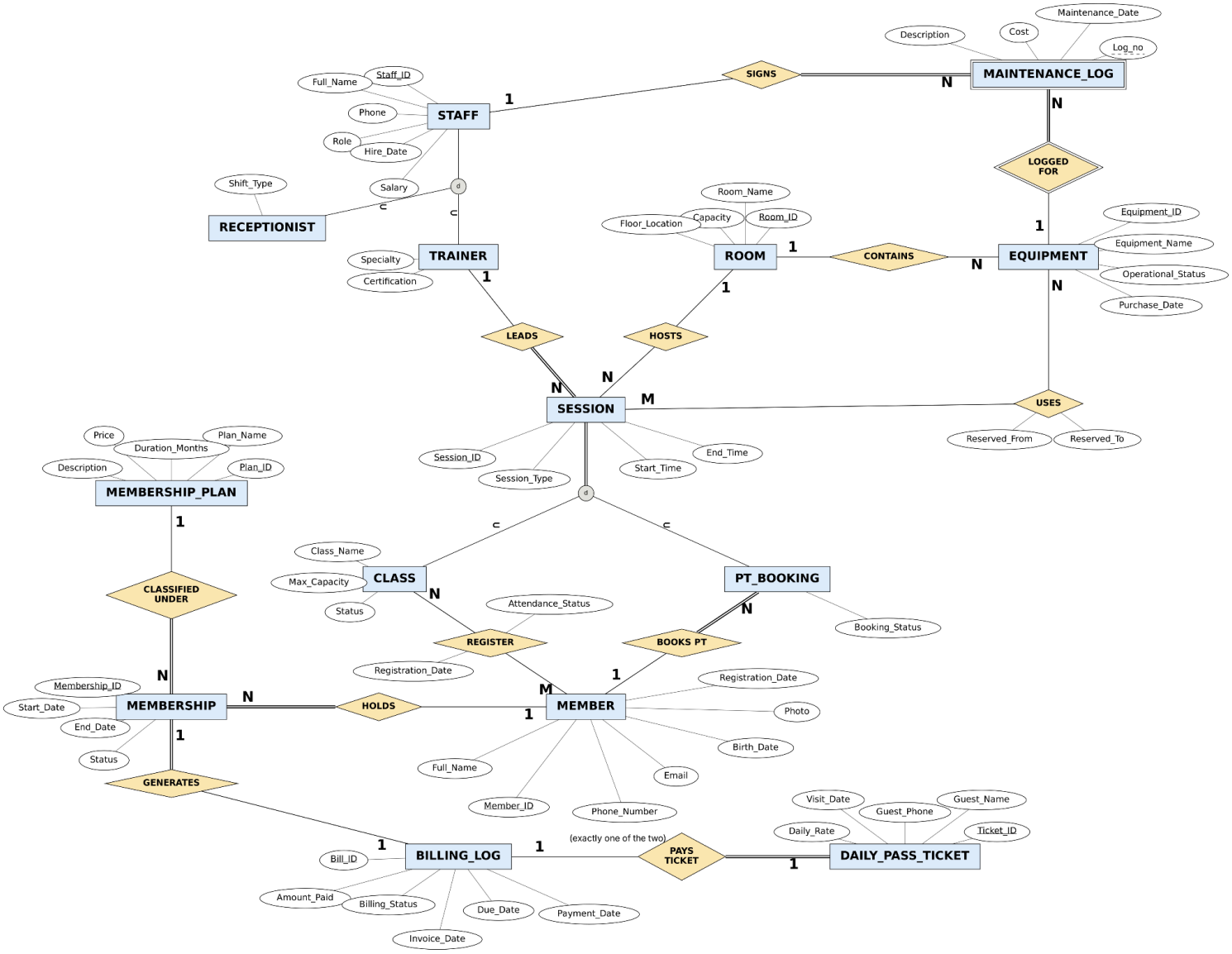
- **Independent strong entities: MEMBER (Member_ID) and DAILY_PASS_TICKET (Ticket_ID) are separate strong entities with no shared superclass and no specialization. Membership purchases are linked through MEMBERSHIP; day passes through DAILY_PASS_TICKET.**
- **STAFF Superclass with Disjoint Specialization (d):** Divided into the **TRAINER** and **RECEPTIONIST** subclasses, which hold the attributes of each role.
- **BILLING_LOG records either a membership purchase (Membership_ID) or a day-pass purchase (Ticket_ID). Both source foreign keys are nullable individually; a CHECK constraint requires exactly one to be populated (XOR), and filtered UNIQUE indexes prevent duplicate billing for either source.**

- **Weak Entity Relationship (EQUIPMENT & MAINTENANCE_LOG):** EQUIPMENT (strong entity) is linked to MAINTENANCE_LOG (weak entity) through the identifying relationship LOGGED_FOR (double diamond), with total participation of MAINTENANCE_LOG.
- **SESSION Superclass with Total Disjoint Specialization (d):** CLASS and PT_BOOKING are subclasses of SESSION and share one Session_ID, so the table made from USES (SESSION_EQUIPMENT_USAGE) refers to a class or a PT booking through a single mandatory foreign key (Session_ID).

- **Notation:** Entity types are drawn as rectangles and attributes as ovals. Key attributes are underlined, and the partial key of a weak entity has a dashed underline. A double line means total participation and a single line means partial participation. In a specialization circle, d means disjoint, and the subset symbol (⊂) on each line points to the subclass.

- **M:N Relationships (REGISTER and USES):** Each M:N relationship is drawn as a single diamond labeled M and N, and the attributes of the relationship are attached to the diamond (Registration_Date and Attendance_Status for REGISTER; Reserved_From and Reserved_To for USES). The EER model has no separate box for them; they become tables only in the mapping (Step 5).

**EER Figure: conceptual EER model of the gym database.**




# 2.2 Logical Schema Mapping

## 2.2.1 Mapping the EER Model to Tables (Elmasri & Navathe, Chapter 9: Steps 1-9)

- **Step 1: Map strong entity types:** Create a table R for each strong entity type E with all of its simple attributes, and choose one candidate key as the primary key (PK). Other candidate

keys become UNIQUE columns [UQ]. Foreign keys are not added yet; they are added in Steps 4 and 5:

  - MEMBER(Member_ID [PK], Full_Name [NOT NULL], Phone_Number [NOT NULL], Email [NULLABLE], Birth_Date [NULLABLE], Photo [NULLABLE], Registration_Date [NOT NULL])
  - MEMBERSHIP_PLAN(Plan_ID [PK], Plan_Name [NOT NULL, UNIQUE], Duration_Months [NOT NULL, CHECK > 0], Price [NOT NULL, CHECK > 0], Description [NULLABLE])
  - DAILY_PASS_TICKET(Ticket_ID [PK], Guest_Name [NOT NULL], Guest_Phone [NOT NULL], Visit_Date [NOT NULL], Daily_Rate [NOT NULL, CHECK > 0]; UNIQUE(Guest_Phone, Visit_Date))
  - STAFF(Staff_ID [PK], Full_Name, Phone [UQ], Role, Hire_Date, Salary)
  - ROOM(Room_ID [PK], Room_Name [UQ], Capacity, Floor_Location)
  - SESSION(Session_ID [PK], Session_Type, Start_Time, End_Time)
  - EQUIPMENT(Equipment_ID [PK], Equipment_Name, Operational_Status, Purchase_Date)
  - MEMBERSHIP(Membership_ID [PK], Start_Date, End_Date, Status)
  - BILLING_LOG(Bill_ID [PK], Invoice_Date [NOT NULL], Due_Date [NULLABLE], Payment_Date [NULLABLE], Amount_Paid [NOT NULL, CHECK >= 0], Billing_Status [NOT NULL]; CHECK Payment_Date matches Billing_Status)

- **Step 2: Map weak entity types:** Create a table R for each weak entity type W with owner entity type E. Include all simple attributes of W and the PK of E as a foreign key (FK). The PK of R combines the PK of E and the partial key of W. Elmasri and Navathe recommend the CASCADE option on this foreign key, because a weak entity cannot exist without its owner (FK_MLOG_EQUIPMENT in Section 2.2.3; see Mapping Figure 4):
  - MAINTENANCE_LOG({Equipment_ID [FK -> EQUIPMENT], Log_no} [PK], Maintenance_Date, Cost, Description)
- **Step 3: Map binary 1:1 relationship types:** The book gives three options: a foreign key in one of the two tables (preferred), merging the two tables, or a separate relationship table. This design has no binary 1:1 relationship type: the link between a superclass and its subclass is handled in Step 8, and every other relationship type is 1:N or M:N.
- **Step 4: Map binary 1:N relationship types:** For each 1:N relationship type, add the PK of the table on the 1-side to the table on the N-side as a foreign key (the foreign-key approach, which the book prefers because it needs no extra table). Each foreign key is added to a table created in Step 1 or Step 2:

- MEMBERSHIP: add Member_ID [FK -> MEMBER, NOT NULL] (1:N HOLDS) and Plan_ID [FK -> MEMBERSHIP_PLAN, NOT NULL] (1:N CLASSIFIED_UNDER).
- BILLING_LOG: add Membership_ID [FK -> MEMBERSHIP, NULL] and Ticket_ID [FK -> DAILY_PASS_TICKET, NULL]; CHECK ((Membership_ID IS NOT NULL AND Ticket_ID IS NULL) OR (Membership_ID IS NULL AND Ticket_ID IS NOT NULL)); filtered UNIQUE indexes on each non-NULL source key.
- SESSION: add Room_ID [FK -> ROOM, NULLABLE] (1:N HOSTS) and Trainer_ID [FK -> TRAINER, NOT NULL] (1:N LEADS)
- PT_BOOKING (SESSION subclass): add Member_ID [FK -> MEMBER] (1:N BOOKS_PT).
- EQUIPMENT: add Room_ID [FK -> ROOM, NULLABLE] (1:N CONTAINS)
- MAINTENANCE_LOG: add Logged_By [FK -> STAFF] (1:N SIGNS)

- **Step 5: Map binary M:N relationship types:** Create a new table R for each M:N relationship type. Include the PKs of the participating tables as FKs in R; together they normally form the composite PK of R (one exception is explained below). The attributes of the relationship become columns of R, and the foreign keys to SESSION, CLASS and EQUIPMENT use ON DELETE CASCADE, while CLASS_REGISTRATION.Member_ID uses NO ACTION to keep the registration history (Section 2.2.3; see Mapping Figure 5):
  - CLASS_REGISTRATION({Session_ID [FK -> CLASS], Member_ID [FK -> MEMBER]} [PK], Registration_Date, Attendance_Status)
  - SESSION_EQUIPMENT_USAGE(Usage_ID [PK], Session_ID [FK -> SESSION, NOT NULL], Equipment_ID [FK -> EQUIPMENT, NOT NULL], Reserved_From, Reserved_To)
- **Surrogate key for USES:** The M:N relationship USES also gets its own table, SESSION_EQUIPMENT_USAGE. It uses a surrogate primary key Usage_ID instead of the pair (Session_ID, Equipment_ID), because the same machine can be reserved more than once in one session, at different times, so the pair is not unique. Both foreign keys are NOT NULL and use ON DELETE CASCADE.
- **Step 6: Map multivalued attributes: None are modeled. Each MEMBER phone/email and each DAILY_PASS_TICKET guest phone is stored as a single-valued attribute.**
- **Step 7: Map N-ary relationship types:** Create a table R for each relationship type that joins three or more entity types. This design has none: every relationship type is binary (Steps 3 to 5), so no table is created in this step.
- **Step 8: Map specialization/generalization hierarchies using Option 8A for STAFF -> TRAINER / RECEPTIONIST and SESSION -> CLASS / PT_BOOKING only. MEMBER and DAILY_PASS_TICKET are independent strong entities, not subclasses.**
  - 1) STAFF (d) -> TRAINER / RECEPTIONIST:

- TRAINER(Staff_ID [PK, FK -> STAFF], Specialty, Certification)
- RECEPTIONIST(Staff_ID [PK, FK -> STAFF], Shift_Type)

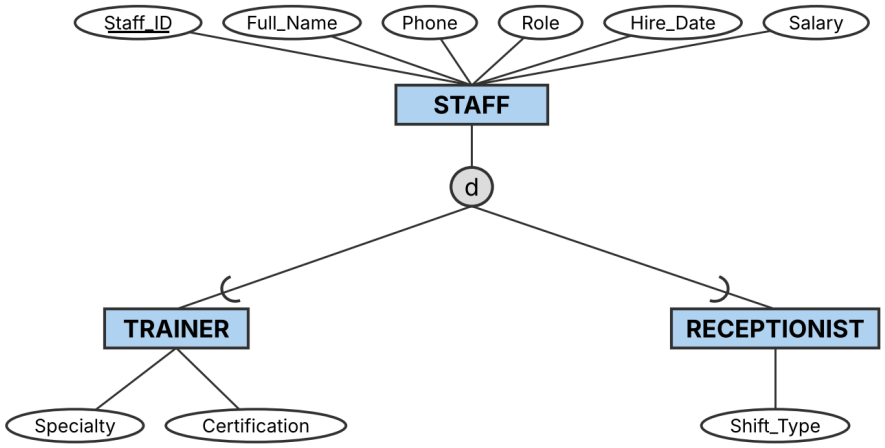
- 2) SESSION (d) -> CLASS / PT_BOOKING:
  - CLASS(Session_ID [PK, FK -> SESSION], Class_Name, Max_Capacity, Status)
  - PT_BOOKING(Session_ID [PK, FK -> SESSION], Member_ID [FK -> MEMBER], Booking_Status)

**Mapping Figure 1: STAFF specialization mapped with Option 8A (partial, disjoint). STAFF, TRAINER, and RECEPTIONIST each get one relation; the subclass key Staff_ID is also a foreign key to STAFF.**

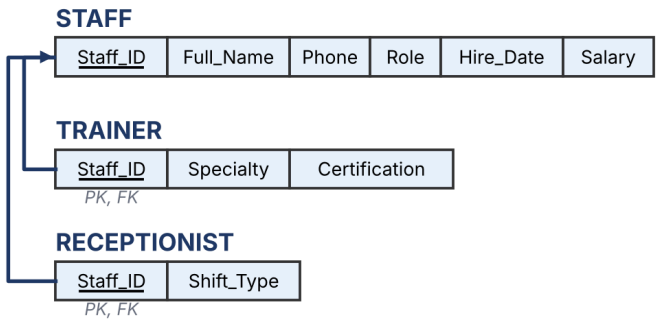
## Mapping 1: STAFF specialization

*Partial, disjoint specialization (d): a Manager has no extra attributes, so it has no subclass row*

**EER diagram (fragment)**




**Relational schema**



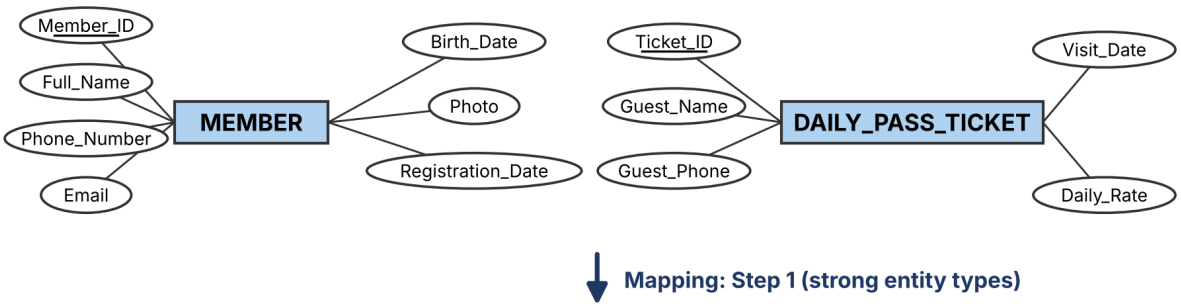

*Underline = primary key. Arrow = foreign key to STAFF(Staff_ID). Single line to the circle = partial participation.*

**Mapping Figure 2: Independent MEMBER (Member_ID) and DAILY_PASS_TICKET (Ticket_ID) relations. Neither inherits a key from the other; both are mapped as strong entities in Step 1.**

## Mapping 2: MEMBER and DAILY_PASS_TICKET (Step 1)

*Two independent strong entities: each keeps its own key, there is no VISITOR superclass*

**EER diagram (fragment)**




**Mapping: Step 1 (strong entity types)**

**Relational schema**

**MEMBER**

| Member_ID | Full_Name | Phone_Number | Email | Birth_Date | Photo | Registration_Date |
|---|---|---|---|---|---|---|

*PK*

*UNIQUE(Phone_Number); Email is unique when it is not NULL*

**DAILY_PASS_TICKET**

| Ticket_ID | Guest_Name | Guest_Phone | Visit_Date | Daily_Rate |
|---|---|---|---|---|

*PK*

*UNIQUE(Guest_Phone, Visit_Date); CHECK(Daily_Rate > 0)*

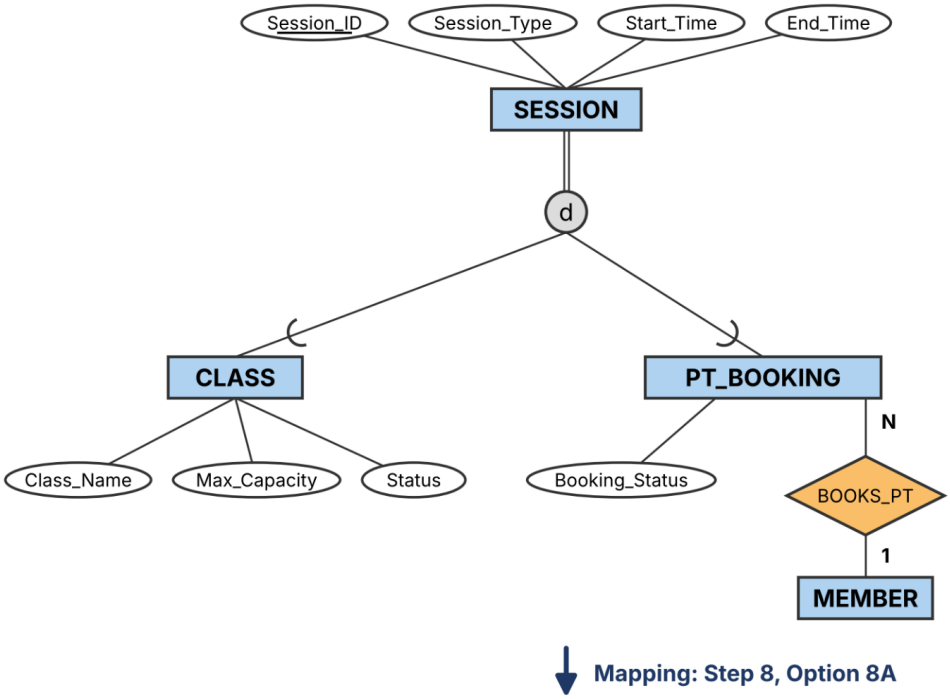
*Underline = primary key. Neither relation has a foreign key to the other. A guest who later becomes a member is matched by phone number (Guest_Phone = Phone_Number): a business rule enforced by triggers, not by a shared key.*

**Mapping Figure 3: SESSION specialization mapped with Option 8A (total, disjoint). CLASS and PT_BOOKING share Session_ID with SESSION; Session_Type records which subclass table holds each session; BOOKS_PT becomes the foreign key Member_ID.**

## Mapping 3: SESSION specialization

*Total, disjoint specialization (d): every session is a CLASS or a PT_BOOKING, never both*

**EER diagram (fragment)**




**Mapping: Step 8, Option 8A**

**Relational schema**

**SESSION**

| Session_ID | Session_Type | Room_ID | Trainer_ID | Start_Time | End_Time |
|---|---|---|---|---|---|
| | | *FK, NULL* | *FK* | | |

**CLASS**

| Session_ID | Class_Name | Max_Capacity | Status |
|---|---|---|---|
| *PK, FK* | | | |

**PT_BOOKING**

| Session_ID | Member_ID | Booking_Status |
|---|---|---|
| *PK, FK* | *FK to MEMBER* | |

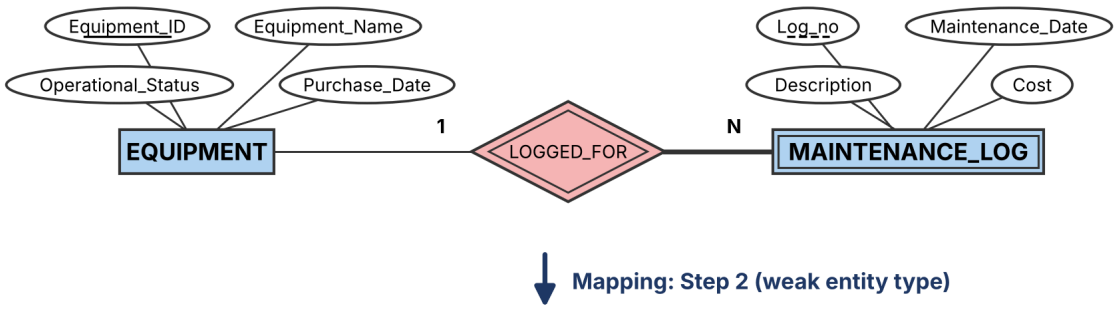
*Underline = primary key. Arrow = foreign key to SESSION(Session_ID). Session_Type tells which subclass table holds the session. Room_ID references ROOM(Room_ID) and may be NULL (room not assigned yet). Trainer_ID references TRAINER(Staff_ID).*

**Mapping Figure 4: Weak entity MAINTENANCE_LOG mapped in Step 2. The primary key combines the owner key Equipment_ID (also a foreign key to EQUIPMENT) with the partial key Log_no.**

## Mapping 4: weak entity MAINTENANCE_LOG (Step 2)

*MAINTENANCE_LOG depends on EQUIPMENT: its key is the owner key plus the partial key Log_no*

**EER diagram (fragment)**




**Relational schema**

**EQUIPMENT**

| Equipment_ID | Equipment_Name | Operational_Status | Purchase_Date |
|---|---|---|---|

**MAINTENANCE_LOG**

| Equipment_ID | Log_no | Maintenance_Date | Cost | Description |
|---|---|---|---|---|
| *PK, FK* | *PK* | | | |

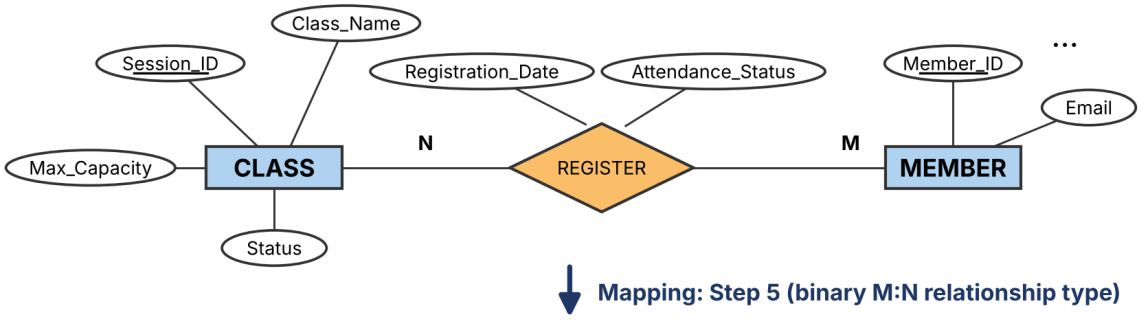
*Underline = primary key. Dashed underline = partial key. Arrow = foreign key to EQUIPMENT(Equipment_ID), ON DELETE CASCADE. Room_ID (relationship HOSTS) and Logged_By (relationship SIGNS) are added in Step 4.*

**Mapping Figure 5: M:N relationship REGISTER mapped in Step 5. CLASS_REGISTRATION has one foreign key to CLASS and one to MEMBER; together they form the primary key.**

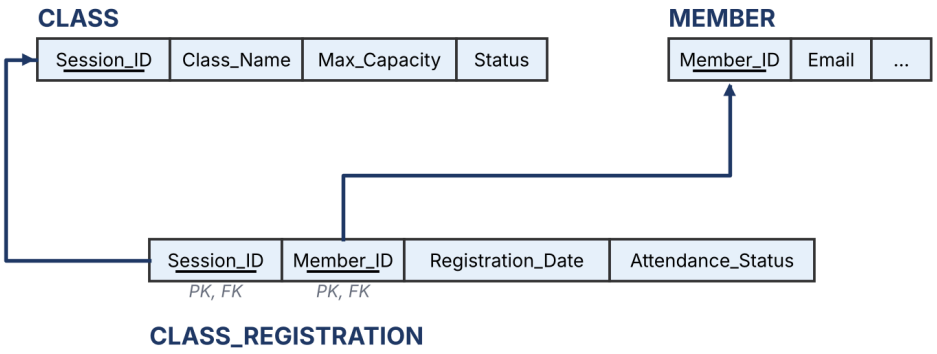
# Mapping 5: M:N relationship REGISTER (Step 5)

*A member can join many classes and a class has many members: the relationship becomes its own table*

**EER diagram (fragment)**




**Relational schema**




*Underline = primary key (the two foreign keys together). Arrows = foreign keys: Session_ID → CLASS is ON DELETE CASCADE; Member_ID → MEMBER is ON DELETE NO ACTION (a member with registrations cannot be deleted, so the history is kept). The attributes of the relationship (Registration_Date, Attendance_Status) become columns of the new table.*

- **Why Option 8A: STAFF and SESSION retain their own specialization hierarchies.**
- **Type attributes: STAFF.Role and SESSION.Session_Type identify their respective subtype categories and can be checked by their domain-specific constraints.**
- **Step 9: Map categories (union types):** Not needed. A category is a subclass of the union of several superclasses that can have different keys. Every subclass here has exactly one superclass and shares its key, so no surrogate key is added.
- **Referential actions:** The book suggests CASCADE for both ON UPDATE and ON DELETE on weak entities, M:N tables and multivalued attributes. This design uses ON DELETE CASCADE on these tables (MAINTENANCE_LOG, CLASS_REGISTRATION, SESSION_EQUIPMENT_USAGE) but ON UPDATE NO ACTION, because key values are never changed once they are created (Section 2.2.3).

### 2.2.2 Relational Schema (16 Normalized Tables)

- **MEMBER:** (Member_ID [PK], Full_Name [NOT NULL], Phone_Number [NOT NULL], Email [NULLABLE], Birth_Date [NULLABLE], Photo [NULLABLE], Registration_Date [NOT NULL])
    - **DAILY_PASS_TICKET:** (Ticket_ID [PK], Guest_Name [NOT NULL], Guest_Phone [NOT NULL], Visit_Date [NOT NULL], Daily_Rate [NOT NULL, CHECK > 0]; UNIQUE(Guest_Phone, Visit_Date))
    - **MEMBERSHIP_PLAN:** (Plan_ID [PK], Plan_Name [NOT NULL, UNIQUE], Duration_Months [NOT NULL, CHECK > 0], Price [NOT NULL, CHECK > 0], Description [NULLABLE])
- **MEMBERSHIP:** (Membership_ID [PK], Member_ID [FK -> MEMBER, NOT NULL], Plan_ID [FK -> MEMBERSHIP_PLAN, NOT NULL], Start_Date, End_Date, Status)
- **BILLING_LOG:** (Bill_ID [PK], Membership_ID [FK -> MEMBERSHIP, NULLABLE; filtered UNIQUE index], Ticket_ID [FK -> DAILY_PASS_TICKET, NULLABLE; filtered UNIQUE index], Invoice_Date [NOT NULL], Due_Date [NULLABLE], Payment_Date [NULLABLE], Amount_Paid [NOT NULL, CHECK >= 0], Billing_Status [NOT NULL]; CHECK exactly one source FK is non-NULL; CHECK Payment_Date/Billing_Status consistency)
- **Cross-domain references: PT_BOOKING.Member_ID and CLASS_REGISTRATION.Member_ID both reference MEMBER(Member_ID). BILLING_LOG has no direct Member_ID; it references either MEMBERSHIP(Membership_ID) or DAILY_PASS_TICKET(Ticket_ID).**
- **STAFF**: (Staff_ID [PK], Full_Name, Phone [UQ], Role, Hire_Date, Salary)
    - **TRAINER**: (Staff_ID [PK/FK->STAFF], Specialty, Certification)
    - **RECEPTIONIST**: (Staff_ID [PK/FK->STAFF], Shift_Type)
- **ROOM**: (Room_ID [PK], Room_Name [UQ], Capacity, Floor_Location)
- **SESSION**: (Session_ID [PK], Session_Type, Room_ID [FK->ROOM], Trainer_ID [FK->TRAINER], Start_Time, End_Time)
    - **CLASS**: (Session_ID [PK/FK->SESSION], Class_Name, Max_Capacity, Status)
    - **PT_BOOKING:** (Session_ID [PK/FK->SESSION], Member_ID [FK->MEMBER], Booking_Status)
- **CLASS_REGISTRATION:** (Session_ID [PK/FK->CLASS], Member_ID [PK/FK->MEMBER], Registration_Date, Attendance_Status)
- **EQUIPMENT**: (Equipment_ID [PK], Equipment_Name, Room_ID [FK->ROOM], Operational_Status, Purchase_Date)
    - **MAINTENANCE_LOG**: (Equipment_ID [PK/FK->EQUIPMENT], Log_no [PK], Maintenance_Date, Cost, Description, Logged_By [FK->STAFF])

- **SESSION_EQUIPMENT_USAGE**: (Usage_ID [PK], Equipment_ID [FK->EQUIPMENT], Session_ID [FK->SESSION], Reserved_From, Reserved_To)

### 2.2.3 Foreign Keys and Delete Rules

| Referencing Table (FK) | Referenced Table (PK) | ON DELETE | ON UPDATE | Reason |
|---|---|---|---|---|
| MEMBERSHIP (Member_ID) | MEMBER (Member_ID) | NO ACTION | NO ACTION | A member referenced by a membership cannot be deleted; preserve member and subscription history. |
| MEMBERSHIP (Plan_ID) | MEMBERSHIP_PLAN (Plan_ID) | NO ACTION | NO ACTION | A plan referenced by a membership cannot be deleted; preserve the plan terms associated with historical subscriptions. |
| BILLING_LOG (Membership_ID) | MEMBERSHIP (Membership_ID) | NO ACTION | NO ACTION | A membership referenced by a billing log cannot be deleted; preserve the financial audit trail. |
| BILLING_LOG (Ticket_ID) | DAILY_PASS_TICKET (Ticket_ID) | NO ACTION | NO ACTION | A daily-pass ticket referenced by a billing log cannot be deleted; preserve the financial audit trail. |
| TRAINER (Staff_ID) | STAFF (Staff_ID) | CASCADE | NO ACTION | A trainer row is a subtype of STAFF and must not exist without its parent staff record; staff identifiers are immutable. |

| | | | | |
|---|---|---|---|---|
| RECEPTIONIST (Staff_ID) | STAFF (Staff_ID) | CASCADE | NO ACTION | A receptionist row is a subtype of STAFF and must not exist without its parent staff record; staff identifiers are immutable. |
| SESSION (Room_ID) | ROOM (Room_ID) | NO ACTION | NO ACTION | A room referenced by a scheduled session cannot be deleted; preserve the session location history. |
| SESSION (Trainer_ID) | TRAINER (Staff_ID) | NO ACTION | NO ACTION | A trainer referenced by a session cannot be deleted; preserve schedule and trainer assignment history. |
| CLASS (Session_ID) | SESSION (Session_ID) | CASCADE | NO ACTION | CLASS is a subtype of SESSION and is deleted with its parent session; session identifiers are immutable. |
| PT_BOOKING (Session_ID) | SESSION (Session_ID) | CASCADE | NO ACTION | PT_BOOKING is a subtype of SESSION and is deleted with its parent session; session identifiers are immutable. |
| CLASS_REGISTRATION (Session_ID) | CLASS (Session_ID) | CASCADE | NO ACTION | A registration is dependent on its class and is removed when that class is deleted. |
| CLASS_REGISTRATION (Member_ID) | MEMBER (Member_ID) | NO ACTION | NO ACTION | Only registered members can register for classes; retain registration history as required. |
| PT_BOOKING (Member_ID) | MEMBER (Member_ID) | NO ACTION | NO ACTION | PT bookings belong to registered members; |

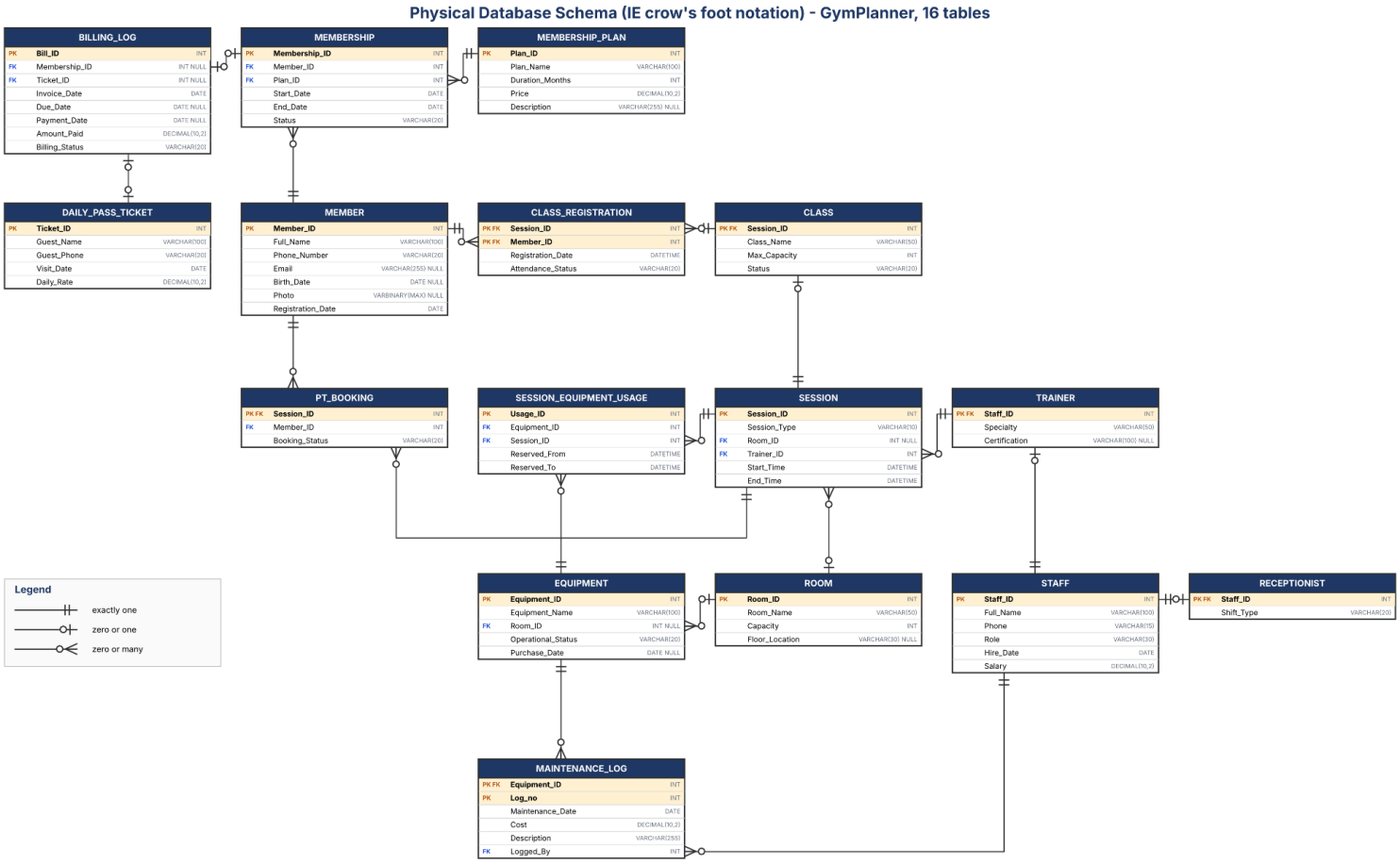
| | | | | |
|---|---|---|---|---|
| | | | | retain booking history as required. |
| EQUIPMENT (Room_ID) | ROOM (Room_ID) | SET NULL | NO ACTION | Machine kept, but its room is cleared when the room is deleted |
| MAINTENANCE_LOG (Equipment_ID) | EQUIPMENT (Equipment_ID) | CASCADE | NO ACTION | Logs of the weak entity deleted with the machine |
| MAINTENANCE_LOG (Logged_By) | STAFF (Staff_ID) | NO ACTION | NO ACTION | Keeps a record of who signed each log |
| SESSION_EQUIPMENT_USAGE (Equipment_ID) | EQUIPMENT (Equipment_ID) | CASCADE | NO ACTION | Reservations deleted when the machine is removed |
| SESSION_EQUIPMENT_USAGE (Session_ID) | SESSION (Session_ID) | CASCADE | NO ACTION | Reservations deleted when the session is deleted |

Implementation note (SQL Server): SQL Server has no RESTRICT keyword; RESTRICT is implemented as NO ACTION, which also blocks the delete. ON UPDATE is NO ACTION for all foreign keys because primary key values are never updated; this also avoids SQL Server error 1785 (multiple cascade paths), e.g., ROOM -> SESSION -> SESSION_EQUIPMENT_USAGE and ROOM -> EQUIPMENT -> SESSION_EQUIPMENT_USAGE.

**How deletes work: A NO ACTION foreign key blocks deletion of a parent while a referencing child exists. A CASCADE foreign key removes dependent rows unless another NO ACTION relationship blocks the transaction. For example, a MEMBER with existing MEMBERSHIP or PT_BOOKING references cannot be deleted while those protected references remain. A DAILY_PASS_TICKET referenced by BILLING_LOG also cannot be deleted without resolving the dependent invoice.**

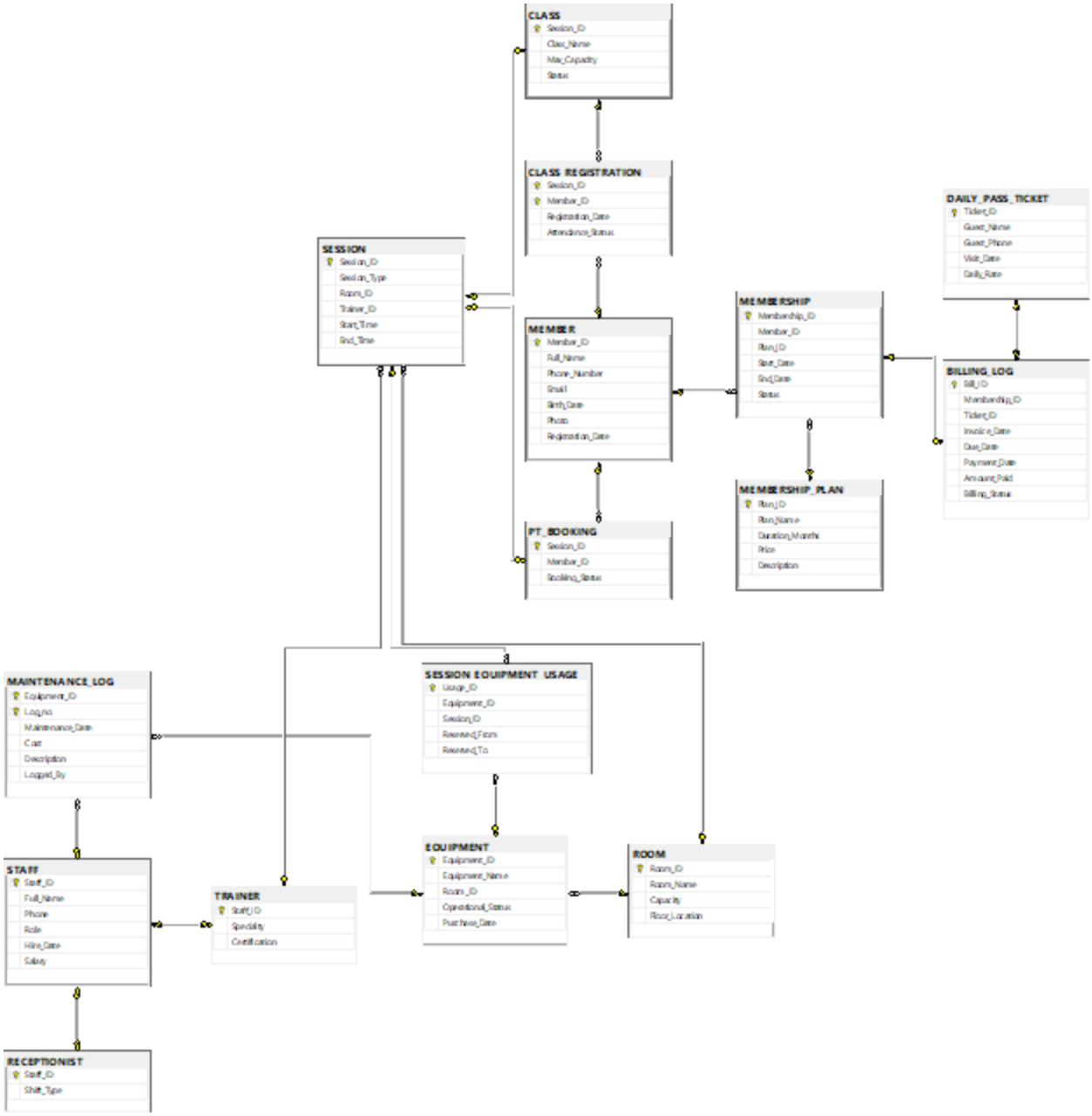
### 2.2.4 Physical Database Schema (Crow's Foot Notation)

**Schema Figure 1: physical schema of the 16 tables in crow's foot notation.**




Schema Figure 2 below is the database diagram that SQL Server Management Studio generated from the GymPlanner_dev database after scripts 01 to 03 were run. It shows the 16 tables and the foreign keys that actually exist in the database, and it matches Schema Figure 1: a key symbol marks the "one" side of a relationship and an infinity symbol marks the "many" side.

**Schema Figure 2: database diagram of GymPlanner_dev, generated by SQL Server Management Studio (SSMS).**




## 2.3 Normalization Verification (1NF -> 2NF -> 3NF / BCNF -> 4NF)

### Formal Definitions:

- **Functional Dependency (FD):** X ⟶ Y holds in R if, for any two tuples t1 and t2 in R, t1[X] = t2[X] implies t1[Y] = t2[Y].
- **Attribute Closure (X+):** The set of all attributes of R that X determines under the FD set F.

- **Superkey Test for BCNF:** A relation R is in BCNF if, for every non-trivial FD X ⟶ Y, X is a superkey of R (i.e., X+ = R).
- **Superkey Test for 3NF:** A relation R is in 3NF if, for every non-trivial FD X ⟶ Y, EITHER X is a superkey of R, OR every attribute A in (Y - X) is a prime attribute (part of a candidate key).

### 2.3.1 Functional Dependencies and Normal Form Checks

**• Relation 1: MEMBER**

- **Candidate Key(s):** {Member_ID}
  **FD Set:** Member_ID -> {Full_Name, Phone_Number, Email, Birth_Date, Photo, Registration_Date}
  **Proof & Verdict:** {Member_ID}+ = MEMBER (all attributes). The left side Member_ID is a superkey. In BCNF (and 3NF).

**• Relation 2: DAILY_PASS_TICKET**

- **Candidate Key(s):** {Ticket_ID}, {Guest_Phone, Visit_Date}
  **FD Set:** Ticket_ID -> {Guest_Name, Guest_Phone, Visit_Date, Daily_Rate}; {Guest_Phone, Visit_Date} -> {Ticket_ID, Guest_Name, Daily_Rate}
  **Proof & Verdict:** {Ticket_ID}+ = DAILY_PASS_TICKET and {Guest_Phone, Visit_Date}+ = DAILY_PASS_TICKET (all attributes). Both left sides are superkeys. In BCNF (and 3NF).

**• Relation 3: MEMBERSHIP_PLAN**

- **Candidate Key(s):** {Plan_ID}, {Plan_Name}
  **FD Set:** Plan_ID -> {Plan_Name, Duration_Months, Price, Description}; Plan_Name -> {Plan_ID, Duration_Months, Price, Description}
  **Proof & Verdict:** {Plan_ID}+ = MEMBERSHIP_PLAN and {Plan_Name}+ = MEMBERSHIP_PLAN (all attributes). Both left sides are superkeys. In BCNF (and 3NF).

**• Relation 4: MEMBERSHIP**

- **Candidate Key(s):** {Membership_ID}
  **FD Set:** Membership_ID -> {Member_ID, Plan_ID, Start_Date, End_Date, Status}
  **Proof & Verdict:** {Membership_ID}+ = MEMBERSHIP (all attributes). The left side Membership_ID is a superkey. In BCNF (and 3NF).

**• Relation 5: BILLING_LOG**

- **Candidate Key(s):** {Bill_ID}
  **FD Set:** Bill_ID -> {Membership_ID, Ticket_ID, Invoice_Date, Due_Date, Payment_Date, Amount_Paid, Billing_Status}
  **Proof & Verdict:** {Bill_ID}+ = BILLING_LOG (all attributes). The left side Bill_ID is a superkey. In BCNF (and 3NF).

**• Relation 6: STAFF**

- **Candidate Key(s):** {Staff_ID}, {Phone}
  **FD Set:** Staff_ID -> {Full_Name, Phone, Role, Hire_Date, Salary}; Phone -> Staff_ID

**Proof & Verdict:** {Staff_ID}+ = STAFF and {Phone}+ = STAFF (all attributes). Both left sides are superkeys. In BCNF (and 3NF).

- **Relation 7: TRAINER**
  - **Candidate Key(s):** {Staff_ID}
    **FD Set:** Staff_ID -> {Specialty, Certification}
    **Proof & Verdict:** {Staff_ID}+ = TRAINER (all attributes). The left side Staff_ID is a superkey. In BCNF (and 3NF).
- **Relation 8: RECEPTIONIST**
  - **Candidate Key(s):** {Staff_ID}
    **FD Set:** Staff_ID -> {Shift_Type}
    **Proof & Verdict:** {Staff_ID}+ = RECEPTIONIST (all attributes). The left side Staff_ID is a superkey. In BCNF (and 3NF).
- **Relation 9: ROOM**
  - **Candidate Key(s):** {Room_ID}, {Room_Name}
    **FD Set:** Room_ID -> {Room_Name, Capacity, Floor_Location}; Room_Name -> Room_ID
    **Proof & Verdict:** {Room_ID}+ = ROOM and {Room_Name}+ = ROOM (all attributes). Both left sides are superkeys. In BCNF (and 3NF).
- **Relation 10: SESSION**
  - **Candidate Key(s):** {Session_ID}
    **FD Set:** Session_ID -> {Session_Type, Room_ID, Trainer_ID, Start_Time, End_Time}
    **Proof & Verdict:** {Session_ID}+ = SESSION (all attributes). The left side Session_ID is a superkey. In BCNF (and 3NF). ({Room_ID, Start_Time} is not a key: a cancelled PT booking may keep the same room and time.)
- **Relation 11: CLASS**
  - **Candidate Key(s):** {Session_ID}
    **FD Set:** Session_ID -> {Class_Name, Max_Capacity, Status}
    **Proof & Verdict:** {Session_ID}+ = CLASS (all attributes). The left side Session_ID is a superkey. In BCNF (and 3NF).
- **Relation 12: CLASS_REGISTRATION**
  - **Candidate Key(s): {Session_ID, Member_ID}**
    **FD Set: {Session_ID, Member_ID} -> {Registration_Date, Attendance_Status}**
    **Proof & Verdict: {Session_ID, Member_ID}+ = CLASS_REGISTRATION (all attributes). The composite key is a superkey. In BCNF (and 3NF).**
- **Relation 13: PT_BOOKING**
  - **Candidate Key(s): {Session_ID}**
    **FD Set: Session_ID -> {Member_ID, Booking_Status}**
    **Proof & Verdict: {Session_ID}+ = PT_BOOKING (all attributes). The left side Session_ID is a superkey. In BCNF (and 3NF).**

**• Relation 14: EQUIPMENT**

- **Candidate Key(s):** {Equipment_ID}
  **FD Set:** Equipment_ID -> {Equipment_Name, Room_ID, Operational_Status, Purchase_Date}
  **Proof & Verdict:** {Equipment_ID}+ = EQUIPMENT (all attributes). The left side Equipment_ID is a superkey. In BCNF (and 3NF).

**• Relation 15: MAINTENANCE_LOG**

- **Candidate Key(s):** {Equipment_ID, Log_no}
  **FD Set:** {Equipment_ID, Log_no} -> {Maintenance_Date, Cost, Description, Logged_By}
  **Proof & Verdict:** {Equipment_ID, Log_no}+ = MAINTENANCE_LOG (all attributes). The composite key on the left side is a superkey. In BCNF (and 3NF).

**• Relation 16: SESSION_EQUIPMENT_USAGE**

- **Candidate Key(s):** {Usage_ID}
- **FD Set:** Usage_ID -> {Equipment_ID, Session_ID, Reserved_From, Reserved_To}
- **Proof & Verdict:** {Usage_ID}+ = SESSION_EQUIPMENT_USAGE (all attributes). The left side Usage_ID is a superkey. In BCNF (and 3NF).

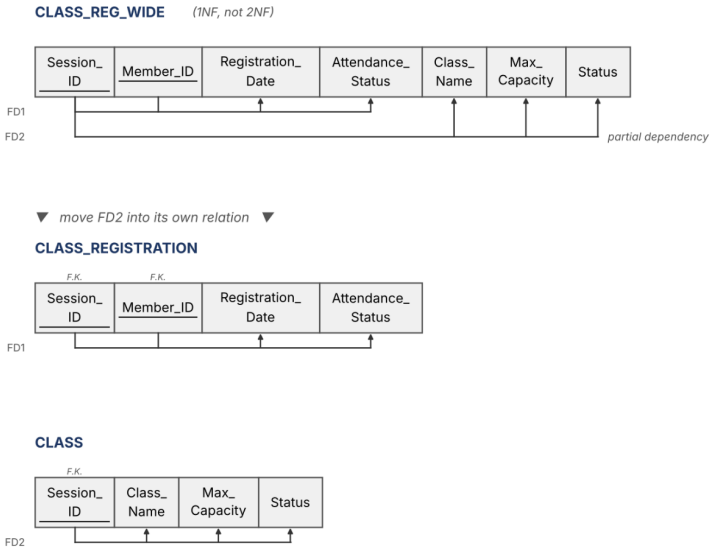
### 2.3.2 Normal Forms Step by Step (1NF -> 2NF -> 3NF / BCNF)

- **1NF:** All attributes in the 16 relations hold single, atomic values of simple types (INT, VARCHAR, VARBINARY, DATETIME, DECIMAL, DATE). There are no multivalued attributes or repeating groups.
- **2NF:** Every non-prime attribute in every table depends on the whole primary key. In MAINTENANCE_LOG, with the composite key {Equipment_ID, Log_no}, Cost and Maintenance_Date depend on the whole key, not on Equipment_ID alone.
- **3NF** and **BCNF**: Under the stated functional dependencies, all relations are designed to satisfy BCNF (and therefore 3NF). BILLING_LOG uses Bill_ID as its primary key, with filtered unique indexes for non-NULL Membership_ID and Ticket_ID values and an exclusive-source CHECK.

### 2.3.3 Normalization Shown with FD Diagrams

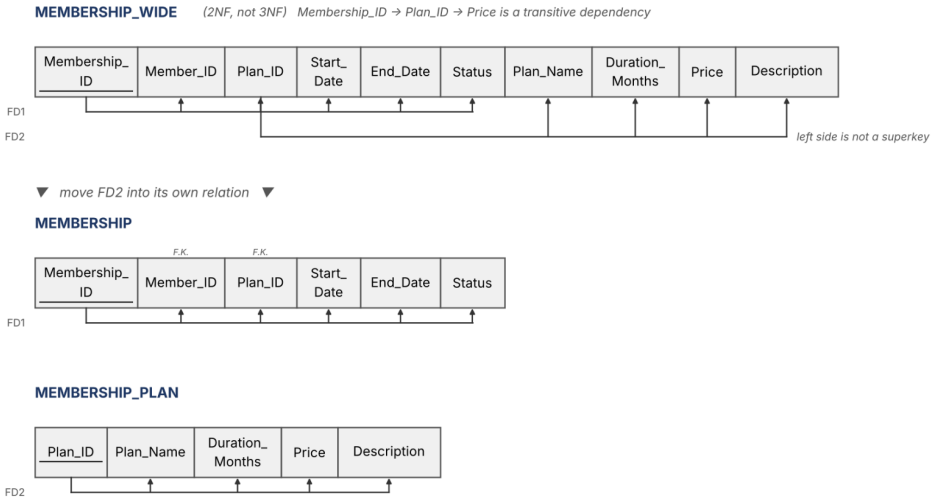
Sections 2.3.1 and 2.3.2 list the functional dependencies as text. This section draws them in the notation used in the course: each relation is a row of cells with the primary key underlined, and each FD is a line that starts under its left side and ends in arrowheads under its right side. Normalization Figures 1 and 2 use combined tables made up for this illustration (CLASS_REG_WIDE and MEMBERSHIP_WIDE). They are not an earlier version of the team's design; they show which dependency a combined table would break and how the final tables avoid it. Normalization Figures 3 to 6 draw 15 of the 16 final relations in four groups, and Normalization Figure 7 draws BILLING_LOG, so together the figures cover all 16 relations.

**Normalization Figure 1: a combined registration table that is in 1NF but not in 2NF, decomposed into CLASS_REGISTRATION and CLASS.**



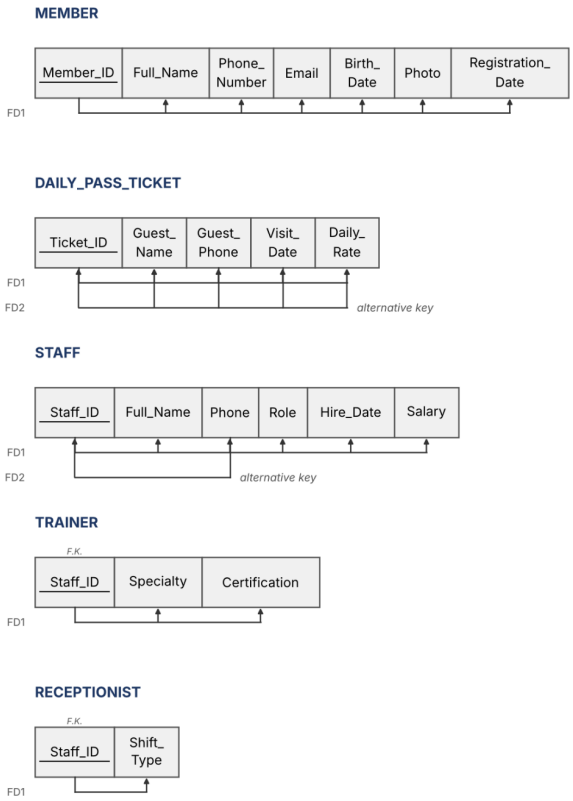

**2NF: In CLASS_REG_WIDE the key is {Session_ID, Member_ID}. FD1 needs the whole key: a registration date and an attendance status belong to one member in one class. FD2 needs only Session_ID: the class name and capacity are the same for every member of the class. A non-prime attribute that depends on part of the key is a partial dependency, so the table is in 1NF but not in 2NF. If 30 members join one class, its name and capacity are stored 30 times; changing the capacity needs 30 updates, and a new class with no members cannot be stored at all, because Member_ID is part of the key and cannot be NULL. Moving FD2 into its own relation gives CLASS(Session_ID, Class_Name, Max_Capacity) and CLASS_REGISTRATION(Session_ID, Member_ID, Registration_Date, Attendance_Status), the two tables used in the design.**

**Normalization Figure 2: a combined membership table that is in 2NF but not in 3NF, decomposed into MEMBERSHIP and MEMBERSHIP_PLAN.**



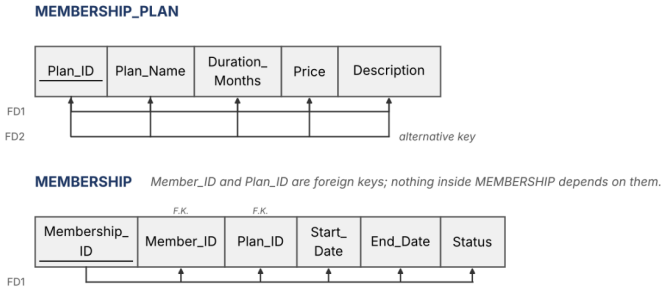

**3NF:** MEMBERSHIP_WIDE has the single-attribute key Membership_ID, so it cannot have a partial dependency and is in 2NF. FD2 (Plan_ID -> Plan_Name, Duration_Months, Price, Description) has a left side that is not a superkey, and Membership_ID -> Plan_ID -> Price is a transitive dependency, so the table is not in 3NF. If 100 memberships use the same plan, its price is stored 100 times; a price change must update all of them, and deleting the last membership of a plan also deletes the only record of the plan. Moving FD2 into MEMBERSHIP_PLAN leaves MEMBERSHIP with Plan_ID as a foreign key, which is the design in Section 2.2.2.

**Normalization Figure 3: FD diagrams of MEMBER, DAILY_PASS_TICKET, STAFF, TRAINER and RECEPTIONIST.**



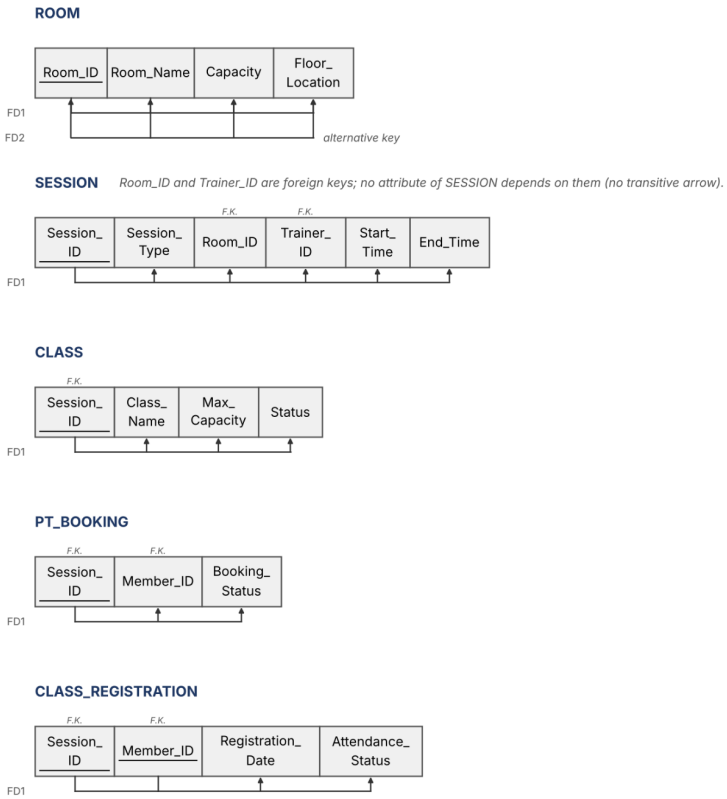

**Person-related relations: MEMBER has Member_ID as its declared candidate key; DAILY_PASS_TICKET has Ticket_ID and the alternative candidate key (Guest_Phone, Visit_Date).**

**Normalization Figure 4: FD diagrams of MEMBERSHIP_PLAN and MEMBERSHIP.**



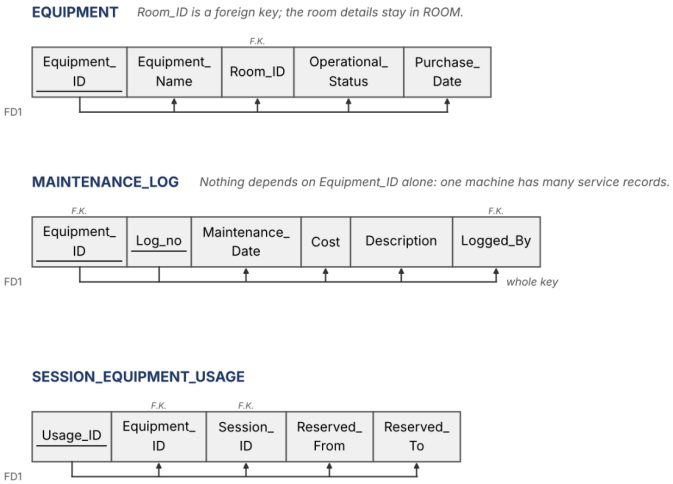

**Membership relations: MEMBERSHIP_PLAN has Plan_ID and Plan_Name (NOT NULL, UNIQUE) as candidate keys. MEMBERSHIP has Membership_ID as its primary key and references MEMBER(Member_ID) and MEMBERSHIP_PLAN(Plan_ID).**

**Normalization Figure 5: FD diagrams of ROOM, SESSION, CLASS, PT_BOOKING and CLASS_REGISTRATION.**




**Room and session relations: ROOM has two candidate keys, Room_ID and Room_Name (UNIQUE). SESSION has two foreign keys (Room_ID, Trainer_ID), but no attribute inside SESSION depends on them: room capacity and trainer details are stored in ROOM and TRAINER, so there is no transitive arrow. CLASS and PT_BOOKING are the two subclasses of SESSION, and their primary key Session_ID is also a foreign key to SESSION. CLASS_REGISTRATION has the composite key {Session_ID, Member_ID}: the registration date and the attendance status belong to one member in one class, so they depend on the whole key. This is the result of the 2NF step in Normalization Figure 1. In all five relations every FD starts from a key, so they are in BCNF.**

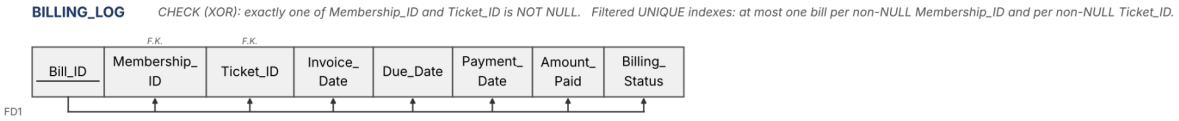
**Normalization Figure 6: FD diagrams of EQUIPMENT, MAINTENANCE_LOG and SESSION_EQUIPMENT_USAGE.**




**Equipment relations:** EQUIPMENT holds the foreign key Room_ID, but the room details stay in ROOM. MAINTENANCE_LOG has the composite key {Equipment_ID, Log_no}; the date, cost, description and signer of a service record depend on the pair, not on Equipment_ID alone, because one machine has many service records. SESSION_EQUIPMENT_USAGE has its own single-attribute key Usage_ID, so it cannot have a partial dependency. In all three relations every FD starts from a key, which is the BCNF condition.

### 2.3.4 BILLING_LOG: Exclusive Purchase Sources and BCNF

**Normalization Figure 7: FD summary of BILLING_LOG with one invoice key and mutually exclusive purchase-source references.**




BILLING_LOG stores both membership and day-pass invoices using exclusive nullable source keys. Bill_ID determines all billing attributes. Filtered UNIQUE indexes prevent multiple billing rows for the same non-NULL Membership_ID or Ticket_ID; the XOR CHECK enforces exactly one purchase source.

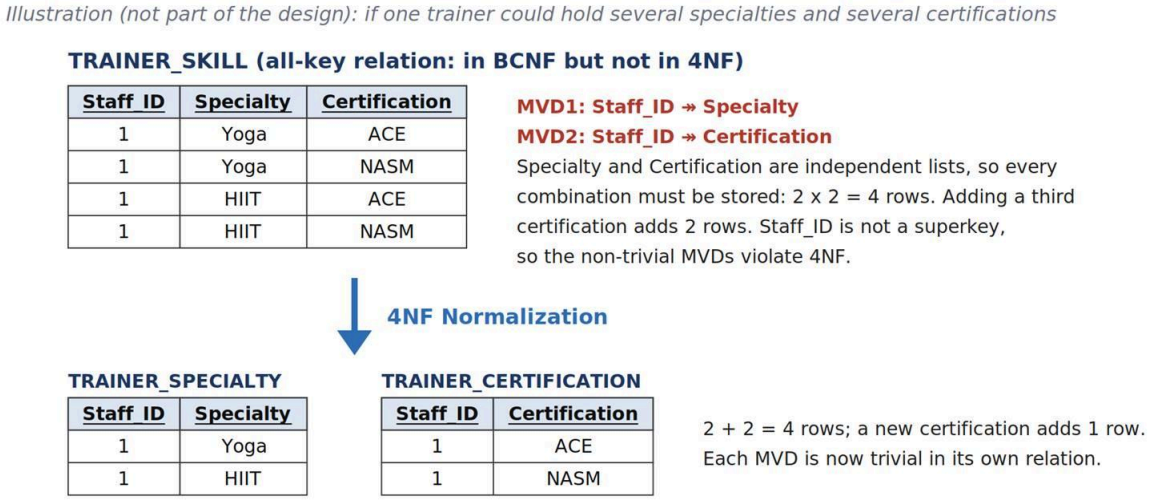
**Alternative considered: splitting membership invoices and daily-pass invoices into separate tables. The selected unified BILLING_LOG instead enforces source exclusivity through a CHECK constraint and one-to-one source billing through filtered UNIQUE indexes.**

### 2.3.5 Fourth Normal Form (4NF)

**Definition:** a relation R is in 4NF if, for every non-trivial multivalued dependency (MVD) X ->> Y, X is a superkey of R. A non-trivial MVD appears when one table stores two independent lists about the same thing, so every combination of the two lists has to be stored.

**Normal-form conclusion: all 16 relations satisfy BCNF under the documented functional dependencies. No relation stores two independent lists about the same key, so there is no non-trivial MVD and all 16 relations are also in 4NF. Cross-row and cross-table business restrictions are enforced by additional constraints or application logic rather than by normalization alone.**

**Normalization Figure 8: illustration of a 4NF violation (not part of the design) and its decomposition.**




Normalization Figure 8 shows what a 4NF violation would look like in this domain. In the design, TRAINER stores one Specialty and one Certification per trainer, so it has no MVD. If the gym later records several specialties and several certifications per trainer, the 4NF form is two separate relations, TRAINER_SPECIALTY and TRAINER_CERTIFICATION, not one combined table.

### 2.3.6 Nullable Columns (NULL Policy)

Nullable columns are documented in the NULL policy table. MEMBER.Photo, MEMBER.Email, and MEMBER.Birth_Date are optional. BILLING_LOG.Membership_ID and BILLING_LOG.Ticket_ID are individually nullable but exactly one must be present.

| Table | Column | Why NULL is allowed | Meaning of NULL / rule applied |
|---|---|---|---|
| MEMBER | Birth_Date | Date of birth is optional. | NULL means the date of birth has not been provided. |

| | | | |
|---|---|---|---|
| MEMBER | Photo | A profile photo is not required when registering or buying a service. | NULL means no photo has been provided; registration, buying a membership, registering for a class and booking PT must not be blocked because of a missing photo (BR-MEM-01). |
| MEMBER | Email | Email is optional information, so the column is not NOT NULL. | NULL means the member has not provided an email; it does not block buying a membership. |
| MEMBERSHIP_PLAN | Description | A free-text description; it is not needed to sell a plan. | NULL means the plan has no extra description; it does not affect the price, the duration or registration. |
| BILLING_LOG | Membership_ID | A bill belongs to either a membership or a daily-pass ticket, so one of the two source keys is always NULL. | NULL when the bill belongs to a Daily Pass Ticket; BR-MEM-09 requires exactly one of Membership_ID and Ticket_ID to be non-NULL. |
| BILLING_LOG | Ticket_ID | A bill belongs to either a membership or a daily-pass ticket, so one of the two source keys is always NULL. | NULL when the bill belongs to a Membership; BR-MEM-09 requires exactly one of Membership_ID and Ticket_ID to be non-NULL. |
| BILLING_LOG | Payment_Date | An invoice can be created before the payment succeeds. | NULL when Billing_Status is 'Pending' or 'Failed'; when it is 'Paid', Payment_Date must have a value (BR-MEM-11). |
| TRAINER | Certification | A certification may not be recorded yet when a trainer is hired. | NULL means the certification has not been recorded; it follows the hiring policy and does not change the staff identity. |
| ROOM | Floor_Location | Floor_Location may not be recorded yet; if a default is used, the default value is 'Floor 1'. | NULL means the floor is not known yet; if DEFAULT is used, it must be stated in the DDL and the Data Dictionary. |

| | | | |
|---|---|---|---|
| SESSION | Room_ID | A session can be created before a room is assigned; the HOSTS relationship has partial participation (single line). | NULL means no room has been assigned to the session yet; the room-overlap rule (BR-SCH-03) only compares sessions that already have a room. |
| EQUIPMENT | Room_ID | The equipment may be kept in storage; ON DELETE SET NULL clears this value when the room of the equipment is deleted. | NULL means the equipment is not assigned to a room yet; if the room is deleted, ON DELETE SET NULL keeps the equipment. |
| EQUIPMENT | Purchase_Date | The purchase date may be unknown for old equipment. | NULL means the purchase date is not available or could not be verified. |
| BILLING_LOG | Due_Date | The due date may not be set when the invoice is created. | NULL means the invoice has no separate due date. |

## 3. Data Dictionary (adapted from ISO/IEC 11179)

ISO/IEC 11179 describes each data element by a name, definition, data type and allowed values. The tables below list attribute names, data types, key types, NULL policy, defaults and constraints. Permitted status and type values appear in Section 3.2. Naming rules: table names use singular uppercase words joined with underscores (for example, MEMBER and MEMBERSHIP_PLAN); attribute names join capitalized words using underscores (for example, Full_Name); artificial keys end with _ID (for example, Member_ID and Bill_ID).

## 3.1 Attribute Tables (16 Tables)

### Table 1: MEMBER (Independent Strong Entity)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Member_ID** | INT | PK | NO | IDENTITY | Member identifier |

| | | | | | |
|---|---|---|---|---|---|
| **Full_Name** | VARCHAR(100) | None | NO | None | Member full name |
| **Phone_Number** | VARCHAR(20) | None | NO | None | Required phone number |
| **Email** | VARCHAR(255) | None | YES | NULL | Optional email |
| **Birth_Date** | DATE | None | YES | NULL | Optional date of birth |
| **Photo** | VARBINARY(MAX) | None | YES | NULL | Optional profile photo |
| **Registration_Date** | DATE | None | NO | None | Member registration date |

**Table 2: DAILY_PASS_TICKET (Independent Strong Entity)**

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Ticket_ID** | INT | PK | NO | IDENTITY | Daily-pass ticket identifier |
| **Guest_Name** | VARCHAR(100) | None | NO | None | Walk-in guest name |
| **Guest_Phone** | VARCHAR(20) | None | NO | None | Phone number used for guest/date uniqueness |
| **Visit_Date** | DATE | None | NO | None | Date the daily pass is valid |
| **Daily_Rate** | DECIMAL(10,2) | None | NO | None | CHECK (Daily_Rate > 0) |

| | | | | | |
|---|---|---|---|---|---|
| **(Guest_Phone, Visit_Date)** | Composite UNIQUE | Alternate key | NO | None | At most one ticket per phone number per date |

### Table 3: MEMBERSHIP_PLAN (Plan Catalog)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Plan_ID** | INT | PK | NO | IDENTITY | Membership-plan identifier |
| **Plan_Name** | VARCHAR(100) | UNIQUE | NO | None | Unique plan name |
| **Duration_Months** | INT | None | NO | None | CHECK (Duration_Months > 0) |
| **Price** | DECIMAL(10,2) | None | NO | None | CHECK (Price > 0) |
| **Description** | VARCHAR(255) | None | YES | NULL | Optional plan description |

### Table 4: MEMBERSHIP (Member Contracts)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Membership_ID** | INT | PK | NO | IDENTITY | Membership record identifier |
| **Member_ID** | INT | FK -> MEMBER | NO | None | References MEMBER(Member_ID) |
| **Plan_ID** | INT | FK -> MEMBERSHIP_PLAN | NO | None | References MEMBERSHIP_PLAN(Plan_ID) |
| **Start_Date** | DATE | None | NO | None | Membership start date |

| | | | | | |
|---|---|---|---|---|---|
| **End_Date** | DATE | None | NO | None | CHECK (Start_Date < End_Date) |
| **Status** | VARCHAR(20) | None | NO | None | CHECK (Status IN ('Active','Expired','Cancelled') ) |

### Table 5: BILLING_LOG (Exclusive Purchase Billing)

| Attribute Name | Data Type | Key Type | Nullable ? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Bill_ID** | INT | PK | NO | IDENTITY | Billing identifier |
| **Membership_ID** | INT | FK -> MEMBERSHIP; filtered UNIQUE | YES | NULL | Used for membership billing only |
| **Ticket_ID** | INT | FK -> DAILY_PASS_TICKET; filtered UNIQUE | YES | NULL | Used for daily-pass billing only |
| **Invoice_Date** | DATE | None | NO | None | Invoice creation date |
| **Due_Date** | DATE | None | YES | NULL | Optional due date |
| **Payment_Date** | DATE | None | YES | NULL | Required when Billing_Status = Paid; must be NULL when Pending or Failed |
| **Amount_Paid** | DECIMAL(10,2) | None | NO | None | CHECK (Amount_Paid >= 0) |
| **Billing_Status** | VARCHAR(20) | None | NO | None | CHECK (Billing_Status IN ('Pending','Paid','Failed')) |

| | | | | | |
|---|---|---|---|---|---|
| **Source XOR CHECK** | CHECK constraint | None | NO | None | Exactly one of Membership_ID and Ticket_ID must be non-NULL |

### Table 6: STAFF (Superclass)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Staff_ID** | INT | PK | NO | IDENTITY | Staff identifier |
| **Full_Name** | VARCHAR(100) | None | NO | None | Full name of the staff member |
| **Phone** | VARCHAR(15) | UNIQUE | NO | None | Phone number of the staff member |
| **Role** | VARCHAR(30) | None | NO | 'Receptionist' | Role: CHECK (Role IN ('Trainer','Receptionist','Manager')); must match the subclass table (BR-SCH-01) |
| **Hire_Date** | DATE | None | NO | CAST(GETDATE() AS DATE) | Date the staff member was hired |
| **Salary** | DECIMAL(10,2) | None | NO | 500.00 | Monthly salary (CHECK Salary > 0.00) |

### Table 7: TRAINER (Subclass of STAFF)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Staff_ID** | INT | PK, FK -> STAFF | NO | None | Subclass PK; references STAFF(Staff_ID); STAFF.Role must |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | be 'Trainer' (BR-SCH-01) |
| **Specialty** | VARCHAR(50) | None | NO | 'General' | Coaching field (Yoga, Strength, HIIT) |
| **Certification** | VARCHAR(100) | None | YES | NULL | Name of the trainer's qualification |

### Table 8: RECEPTIONIST (Subclass of STAFF)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Staff_ID** | INT | PK, FK -> STAFF | NO | None | Subclass PK; references STAFF(Staff_ID); STAFF.Role must be 'Receptionist' (BR-SCH-01) |
| **Shift_Type** | VARCHAR(20) | None | NO | 'Morning' | Shift: CHECK (Shift_Type IN ('Morning','Evening','Night')) |

### Table 9: ROOM (Training Rooms)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Room_ID** | INT | PK | NO | IDENTITY | Room identifier |
| **Room_Name** | VARCHAR(50) | UNIQUE | NO | None | Room name (Studio A, Pilates Room) |
| **Capacity** | INT | None | NO | 20 | Maximum number of |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | people (CHECK Capacity > 0) |
| **Floor_Location** | VARCHAR(30) | None | YES | 'Floor 1' | Floor where the room is |

### Table 10: SESSION (Superclass)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Session_ID** | INT | PK | NO | IDENTITY | Session identifier (class or PT booking) |
| **Session_Type** | VARCHAR(10) | None | NO | None | Subclass of the session: CHECK (Session_Type IN ('Class','PT')) |
| **Room_ID** | INT | FK -> ROOM | YES | NULL | Refers to the room (may be NULL if no room is assigned yet) |
| **Trainer_ID** | INT | FK -> TRAINER | NO | None | Refers to the assigned trainer |
| **Start_Time** | DATETIME | None | NO | None | Planned start date and time |
| **End_Time** | DATETIME | None | NO | None | Planned end date and time (CHECK End_Time > Start_Time) |

### Table 11: CLASS (Subclass of SESSION)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Session_ID** | INT | PK, FK -> SESSION | NO | None | Subclass PK; references SESSION(Session_ID) ; Session_Type = 'Class' |
| **Class_Name** | VARCHAR(50) | None | NO | None | Name of the class (Zumba, Yoga, HIIT) |
| **Max_Capacity** | INT | None | NO | 20 | Maximum number of attendees (CHECK Max_Capacity > 0; not more than ROOM.Capacity, checked by trigger) |
| **Status** | VARCHAR(20) | None | NO | 'Scheduled' | Status: CHECK (Status IN ('Scheduled','Completed','Cancelled')); a 'Cancelled' class does not block rooms or equipment (BR-SCH-03, BR-FAC-10) |

### Table 12: CLASS_REGISTRATION (Class Registrations)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Session_ID** | INT | PK, FK -> CLASS | NO | None | Part 1 of the composite PK (the class) |

| | | | | | |
|---|---|---|---|---|---|
| **Member_ID** | INT | PK, FK -> MEMBER | NO | None | Part 2 of the composite PK (a member registers once per class) |
| **Registration_Date** | DATETIME | None | NO | CURRENT_TIMESTAMP | Date and time of registration |
| **Attendance_Status** | VARCHAR(20) | None | NO | 'Confirmed' | Status: CHECK (Attendance_Status IN ('Confirmed','Attended','Cancelled')) |

### Table 13: PT_BOOKING (Subclass of SESSION)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Session_ID** | INT | PK, FK -> SESSION | NO | None | Subclass PK; references SESSION(Session_ID); Session_Type = 'PT' |
| **Member_ID** | INT | FK -> MEMBER | NO | None | Refers to the member, MEMBER(Member_ID) |
| **Booking_Status** | VARCHAR(20) | None | NO | 'Scheduled' | Status: CHECK (Booking_Status IN ('Scheduled','Completed','Cancelled')) |

### Table 14: EQUIPMENT (Machines)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Equipment_ID** | INT | PK | NO | IDENTITY | Machine identifier |

| | | | | | |
|---|---|---|---|---|---|
| **Equipment_Name** | VARCHAR(100) | None | NO | None | Machine name (brand/model) |
| **Room_ID** | INT | FK -> ROOM | YES | NULL | Refers to the room |
| **Operational_Status** | VARCHAR(20) | None | NO | 'Active' | Status: CHECK (Operational_Status IN ('Active','Under Maintenance','Broken')) |
| **Purchase_Date** | DATE | None | YES | NULL | Date the machine was bought |

### Table 15: MAINTENANCE_LOG (Weak Entity: Service History)

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Equipment_ID** | INT | PK, FK -> EQUIPMENT | NO | None | Part 1 of the composite PK; refers to the machine |
| **Log_no** | INT | PK (partial key) | NO | None | Part 2 of the composite PK (log number for each machine) |
| **Maintenance_Date** | DATE | None | NO | CAST(GETDATE() AS DATE) | Date of the service |
| **Cost** | DECIMAL(10,2) | None | NO | 0.00 | Cost of the repair (CHECK Cost >= 0.00) |

| | | | | | |
|---|---|---|---|---|---|
| **Description** | VARCHAR(255) | None | NO | None | Short description of the work done |
| **Logged_By** | INT | FK -> STAFF | NO | None | Refers to the staff member who signed the log; Role must be 'Receptionist' or 'Manager' (BR-FAC-05, checked by trigger) |

**Table 16: SESSION_EQUIPMENT_USAGE (Equipment Reservations)**

| Attribute Name | Data Type | Key Type | Nullable? | Default Value | Business Rules / Constraints |
|---|---|---|---|---|---|
| **Usage_ID** | INT | PK | NO | IDENTITY | Reservation identifier |
| **Equipment_ID** | INT | FK -> EQUIPMENT | NO | None | Refers to the reserved machine; the machine must not be 'Broken' or 'Under Maintenance' (BR-FAC-06); no other reservation of the same machine may overlap in time (BR-FAC-10) |

| | | | | | |
|---|---|---|---|---|---|
| **Session_ID** | INT | FK -> SESSION | NO | None | Refers to the class or PT session that uses the machine |
| **Reserved_From** | DATETIME | None | NO | None | Start of the reservation; must lie inside the session (BR-FAC-09) |
| **Reserved_To** | DATETIME | None | NO | None | End of the reservation (CHECK Reserved_To > Reserved_From); must lie inside the session (BR-FAC-09) |

## 3.2 Value Definitions for Status and Type Columns

Nine columns accept only a fixed list of values (enforced by the CHECK constraints in Section 4.1.2). The table below defines each value, so that the business rules, triggers and queries all use it with the same meaning.

| Table | Column | Value | Default? | Used by | Meaning |
|---|---|---|---|---|---|
| MEMBERSHIP | Status | Active | Yes | BR-MEM-05 | Membership is within its valid access period and may grant gym access only when payment/access conditions are satisfied. |

| | | | | | |
|---|---|---|---|---|---|
| MEMBERSHIP | Status | Expired | No | BR-MEM-05 | Membership validity period has ended; the historical record is retained and does not grant gym access. |
| MEMBERSHIP | Status | Cancelled | No | BR-MEM-05 | Membership was cancelled; the historical record is retained and does not grant gym access. |
| BILLING_LOG | Billing_Status | Pending | No | BR-MEM-10, BR-MEM-11 | Invoice/payment is awaiting completion. Payment_Date must be NULL. |
| BILLING_LOG | Billing_Status | Paid | No | BR-MEM-10, BR-MEM-11 | Payment is successful. Payment_Date must contain the payment date. |
| BILLING_LOG | Billing_Status | Failed | No | BR-MEM-10, BR-MEM-11 | Payment attempt failed. Payment_Date must be NULL; a retry updates this existing Billing Log. |
| STAFF | Role | Trainer | No | BR-SCH-01 | Leads group classes or coaches PT sessions; has a row in TRAINER (a TRAINER row for any other role is rejected). |
| STAFF | Role | Receptionist | Yes | BR-SCH-01, | Front-desk staff; has a row in RECEPTIONIST (a |

| | | | | | |
|---|---|---|---|---|---|
| | | | | BR-FAC-05 | RECEPTIONIST row for any other role is rejected); may sign maintenance logs. |
| STAFF | Role | Manager | No | BR-FAC-05 | Runs the gym; has no subclass row (a TRAINER or RECEPTIONIST row for a Manager is rejected); may sign maintenance logs. |
| RECEPTIONIST | Shift_Type | Morning / Evening / Night | Yes (Morning) | 1.1.1 | Name of the receptionist's regular shift; the default is 'Morning'. The hours of each shift are set by the gym's staff roster and are not stored in the database. |
| SESSION | Session_Type | Class | No | BR-SCH-06 | The session is a group class; its details are in CLASS. |
| SESSION | Session_Type | PT | No | BR-SCH-06 | The session is a private 1-on-1 PT booking; its details are in PT_BOOKING. |
| CLASS | Status | Scheduled | Yes | BR-SCH-03, BR-FAC-10 | The class is planned and holds its room and equipment. |
| CLASS | Status | Completed | No | BR-SCH-03 | The class has taken place; the record is kept. |
| CLASS | Status | Cancelled | No | BR-SCH-03, | The class was cancelled; it no |

| | | | | | |
|---|---|---|---|---|---|
| | | | | BR-FAC-10 | longer holds its room or equipment, so others can use that time. |
| CLASS_REGISTRATION | Attendance_Status | Confirmed | Yes | BR-SCH-02 | The member is registered and holds a seat; has not checked in yet. |
| CLASS_REGISTRATION | Attendance_Status | Attended | No | BR-SCH-02 | The member checked in when the class took place. Still counts as a seat. |
| CLASS_REGISTRATION | Attendance_Status | Cancelled | No | BR-SCH-02 | The registration was cancelled; the seat is released. The row is kept as history. |
| PT_BOOKING | Booking_Status | Scheduled | Yes | BR-SCH-03, BR-SCH-04, BR-FAC-10 | Upcoming booking; holds the room, the trainer and any reserved equipment for its time slot. |
| PT_BOOKING | Booking_Status | Completed | No | BR-SCH-03, BR-SCH-04, BR-FAC-10 | The session took place. Kept as history; still counted in overlap checks. |
| PT_BOOKING | Booking_Status | Cancelled | No | BR-SCH-03, BR-SCH-04, BR-FAC-10 | The booking was cancelled; the room, the trainer's time slot and the equipment reservations of the session are released (not |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | counted in overlap checks). |
| EQUIPMENT | Operational_Status | Active | Yes | BR-FAC-06 | The machine works and can be reserved. |
| EQUIPMENT | Operational_Status | Under Maintenance | No | BR-FAC-06 | The machine is being serviced; new reservations are rejected. |
| EQUIPMENT | Operational_Status | Broken | No | BR-FAC-06 | The machine is out of order; new reservations are rejected until it is repaired. |

# 4. Database Implementation (SQL Style Guide Compliant)

The design of Section 2 is implemented in SQL Server (database GymPlanner_dev) by twelve scripts. They need SQL Server 2016 SP1 or newer, because they use CREATE OR ALTER. Section 4.1 describes the scripts that create the database objects (tables, views, indexes, triggers). Section 4.2 describes the advanced queries and the performance tests.

# 4.1 DDL Scripts (Tables, Views, Indexes, Triggers)

### 4.1.1 SQL Style Conventions

**Layout:** Every script starts with a header comment (purpose, run order). SQL keywords are written in capital letters, indentation uses four spaces (no tabs), each statement ends with a semicolon, and batches are separated by GO. Scripts 08 to 11 also put one column on each line, write every table alias with AS, and indent the JOIN conditions under the JOIN. The triggers of script 02 use the shorter form (alias without AS).

**Names: Constraints and objects have a fixed prefix: PK_ (primary key), FK_ (foreign key), UQ_ (UNIQUE), CK_ (CHECK), DF_ (DEFAULT), IX_ and UX_ (index and unique index), trg_ (trigger), vw_ (view) and role_ (database role). Tables are always written with their schema (dbo.MEMBER).**

**Running again:** Scripts 02, 08 and 10 can be run again without error (CREATE OR ALTER, IF NOT EXISTS). Script 01 creates the database only if it does not exist yet; to rebuild the tables, the DROP lines at its top are used.

**Dates and errors:** Dates are written as yyyy-mm-dd, and the scripts that load or test data start with SET DATEFORMAT ymd, so they give the same result on any language setting of the server. Business-rule errors use the THROW numbers 50001 to 50029.

**Style guide:** These choices follow the SQL Style Guide of Simon Holywell (sqlstyle.guide) where it fits T-SQL. The main difference is that table and column names keep capital letters, to match the data dictionary and this report.

### 4.1.2 Tables

Script 01 defines the 16 normalized relations and their primary/foreign keys.

```
CREATE TABLE dbo.MEMBERSHIP (
        Membership_ID INT IDENTITY(1,1) NOT NULL,
  Member_ID     INT         NOT NULL,
        Plan_ID     INT         NOT NULL,
        Start_Date DATE       NOT NULL,
        End_Date  DATE        NOT NULL,
        Status      VARCHAR(20)          NOT NULL,
        CONSTRAINT PK_MEMBERSHIP PRIMARY KEY (Membership_ID),
        CONSTRAINT FK_MEMBERSHIP_MEMBER FOREIGN KEY (Member_ID)
        REFERENCES dbo.MEMBER (Member_ID)
        ON DELETE NO ACTION ON UPDATE NO ACTION,
        CONSTRAINT FK_MEMBERSHIP_PLAN FOREIGN KEY (Plan_ID)
        REFERENCES dbo.MEMBERSHIP_PLAN (Plan_ID)
        ON DELETE NO ACTION ON UPDATE NO ACTION,
        CONSTRAINT CK_MEMBERSHIP_Dates  CHECK (End_Date > Start_Date),
        CONSTRAINT CK_MEMBERSHIP_Status CHECK (Status IN ('Active','Expired','Cancelled'))
);
```

### 4.1.3 Views

Script 08 creates five views. A view is a saved SELECT. A view lets a query be reused without repeating it, and it can hide columns: script 10 lets some roles read vw_Trainer_Public but not the table STAFF, which holds the salaries.

| View | What it shows | Tables read |
|---|---|---|
| vw_Active_Membership | Members with a membership whose Status is 'Active', and the days left (counted from the day the view is read) | MEMBERSHIP, MEMBER, MEMBERSHIP_PLAN |
| vw_Class_Schedule | Every class with its room, trainer, seats taken and seats left | CLASS, SESSION, ROOM, STAFF, CLASS_REGISTRATION |
| vw_Member_Billing_Summary | Per member: number of bills, total paid, and bills still 'Pending' or 'Failed' | MEMBER, MEMBERSHIP, BILLING_LOG |
| vw_Trainer_Public | Trainers without Salary and Phone, safe to show to members | TRAINER, STAFF |
| vw_Equipment_Status | Equipment with its room, last maintenance date, number of logs and total cost | EQUIPMENT, ROOM, MAINTENANCE_LOG |

The definitions of the five views:

```
CREATE OR ALTER VIEW dbo.vw_Active_Membership
AS
SELECT
        ms.Membership_ID,
        m.Member_ID,
        m.Full_Name                     AS Member_Name,
        p.Plan_Name,
        ms.Start_Date,
        ms.End_Date,
        DATEDIFF(DAY, CAST(GETDATE() AS DATE), ms.End_Date)     AS Days_Left
FROM dbo.MEMBERSHIP AS ms
INNER JOIN dbo.MEMBER AS m
        ON m.Member_ID = ms.Member_ID
-- MEMBER is the person relation; use MEMBER.Member_ID and MEMBER.Full_Name directly (no superclass join).

INNER JOIN dbo.MEMBERSHIP_PLAN AS p
        ON p.Plan_ID = ms.Plan_ID
WHERE ms.Status = 'Active';

CREATE OR ALTER VIEW dbo.vw_Class_Schedule
AS
SELECT
        c.Session_ID,
        c.Class_Name,
        s.Start_Time,
        s.End_Time,
        r.Room_Name,
        st.Full_Name                    AS Trainer_Name,
        c.Max_Capacity,
        COUNT(cr.Member_ID)             AS Seats_Taken,
        c.Max_Capacity - COUNT(cr.Member_ID)    AS Seats_Left
FROM dbo.CLASS AS c
INNER JOIN dbo.[SESSION] AS s
        ON s.Session_ID = c.Session_ID
LEFT JOIN dbo.ROOM AS r
        ON r.Room_ID = s.Room_ID
INNER JOIN dbo.STAFF AS st
        ON st.Staff_ID = s.Trainer_ID
LEFT JOIN dbo.CLASS_REGISTRATION AS cr
        ON cr.Session_ID = c.Session_ID
  AND cr.Attendance_Status <> 'Cancelled'
WHERE c.Status <> 'Cancelled'
GROUP BY
        c.Session_ID,
        c.Class_Name,
        s.Start_Time,
        s.End_Time,
        r.Room_Name,
        st.Full_Name,
        c.Max_Capacity;

CREATE OR ALTER VIEW dbo.vw_Member_Billing_Summary
AS
SELECT
        m.Member_ID,
        m.Full_Name AS Member_Name,
        COUNT(b.Bill_ID)                AS Bills,
```

```
        COALESCE(SUM(CASE WHEN b.Billing_Status = 'Paid'
                    THEN b.Amount_Paid END), 0.00) AS Total_Paid,
        COALESCE(SUM(CASE WHEN b.Billing_Status IN ('Pending', 'Failed')
                    THEN 1 END), 0)                AS Open_Bills
FROM dbo.MEMBER AS m



LEFT JOIN dbo.MEMBERSHIP AS ms ON ms.Member_ID = m.Member_ID
LEFT JOIN dbo.BILLING_LOG AS b ON b.Membership_ID = ms.Membership_ID
GROUP BY
        m.Member_ID,
        m.Full_Name



CREATE OR ALTER VIEW dbo.vw_Trainer_Public
AS
SELECT
        st.Staff_ID,
        st.Full_Name,
        t.Specialty,
        t.Certification
FROM dbo.TRAINER AS t
INNER JOIN dbo.STAFF AS st
        ON st.Staff_ID = t.Staff_ID;

CREATE OR ALTER VIEW dbo.vw_Equipment_Status
AS
SELECT
        e.Equipment_ID,
        e.Equipment_Name,
        r.Room_Name,
        e.Operational_Status,
        MAX(ml.Maintenance_Date)       AS Last_Maintenance,
        COUNT(ml.Log_no)               AS Maintenance_Count,
        COALESCE(SUM(ml.Cost), 0.00) AS Total_Cost
FROM dbo.EQUIPMENT AS e
LEFT JOIN dbo.ROOM AS r
        ON r.Room_ID = e.Room_ID
LEFT JOIN dbo.MAINTENANCE_LOG AS ml
        ON ml.Equipment_ID = e.Equipment_ID
GROUP BY
        e.Equipment_ID,
        e.Equipment_Name,
        r.Room_Name,
        e.Operational_Status;
```

Example: reading vw_Class_Schedule on the sample data (times without seconds):

| Class_Name | Start_Time | Room_Name | Trainer_Name | Max_Capacity | Seats_Taken | Seats_Left |
|---|---|---|---|---|---|---|
| Yoga Flow | 2026-11-02 07:00 | Yoga Room | Khoa Nguyen Minh | 15 | 2 | 13 |
| Yoga Flow | 2026-11-09 07:00 | Yoga Room | Khoa Nguyen Minh | 15 | 3 | 12 |
| HIIT Burn | 2026-11-09 18:00 | Cardio Zone | Ha Tran Thu | 3 | 3 | 0 |
| Yoga Flow | 2026-11-10 07:00 | Yoga Room | Khoa Nguyen Minh | 15 | 1 | 14 |

| | | | | | | |
|---|---|---|---|---|---|---|
| Spinning | 2026-11-11 18:00 | Cardio Zone | Duc Hoang Anh | 25 | 2 | 23 |
| Pilates Intro | 2026-11-13 10:00 | NULL | Duc Hoang Anh | 15 | 0 | 15 |

### 4.1.4 Indexes

Script 03 creates five indexes in addition to the indexes that the PRIMARY KEY and UNIQUE constraints already create. They speed up the overlap checks of the triggers and the lookups by member or by session (NFR-PER). Their effect is measured in Section 4.2.2.

| Index | Table | Columns | Purpose |
|---|---|---|---|
| IX_SESSION_Room_Time | SESSION | (Room_ID, Start_Time, End_Time) | Overlap check of BR-SCH-03 (room) |
| IX_SESSION_Trainer_Time | SESSION | (Trainer_ID, Start_Time, End_Time) | Overlap check of BR-SCH-04 (trainer) |
| IX_CLASS_REG_Member | CLASS_REGISTRATION | (Member_ID) | Classes of a member |
| IX_PT_BOOKING_Member | PT_BOOKING | (Member_ID) | PT bookings of a member |
| IX_SEU_Session | SESSION_EQUIPMENT_USAGE | (Session_ID) | Equipment reservations of a session |

```
CREATE NONCLUSTERED INDEX IX_SESSION_Room_Time
        ON dbo.[SESSION] (Room_ID, Start_Time, End_Time);

CREATE NONCLUSTERED INDEX IX_SESSION_Trainer_Time
        ON dbo.[SESSION] (Trainer_ID, Start_Time, End_Time);

CREATE NONCLUSTERED INDEX IX_CLASS_REG_Member
        ON dbo.CLASS_REGISTRATION (Member_ID);

CREATE NONCLUSTERED INDEX IX_PT_BOOKING_Member
        ON dbo.PT_BOOKING (Member_ID);

CREATE NONCLUSTERED INDEX IX_SEU_Session
        ON dbo.SESSION_EQUIPMENT_USAGE (Session_ID);
```

### 4.1.5 Triggers

Script 02 creates 24 AFTER triggers for the rules that a CHECK constraint cannot test, because they involve other rows or other tables (Section 1.2). SQL Server has no BEFORE trigger, so a trigger lets the change happen, tests the result, and if a rule is broken it runs ROLLBACK TRANSACTION and raises an error with THROW. The error message starts with the rule code, for example BR-SCH-02.

| Trigger | Table | Event | Rule | Error no. |
|---|---|---|---|---|
| trg_MEMBERSHIP_NoActiveOverlap | MEMBERSHIP | INSERT, UPDATE | BR-MEM-06 | 50001 |
| trg_TICKET_NoActiveMembership | DAILY_PASS_TICKET | INSERT, UPDATE | BR-MEM-08 | 50002 |
| trg_MEMBERSHIP_NoOverlappingTicket | MEMBERSHIP | INSERT, UPDATE | BR-MEM-08 | 50003 |

| | | | | |
|---|---|---|---|---|
| trg_MEMBER_Phone_NoAccessOverlap | MEMBER | UPDATE | BR-MEM-08 | 50004 |
| trg_PT_BOOKING_ActiveMembership | PT_BOOKING | INSERT | BR-SCH-05 | 50005 |
| trg_TRAINER_Disjoint | TRAINER | INSERT, UPDATE | BR-SCH-01 | 50007 |
| trg_RECEPTIONIST_Disjoint | RECEPTIONIST | INSERT, UPDATE | BR-SCH-01 | 50008 |
| trg_TRAINER_RoleMatch | TRAINER | INSERT, UPDATE | BR-SCH-01 | 50026 |
| trg_RECEPTIONIST_RoleMatch | RECEPTIONIST | INSERT, UPDATE | BR-SCH-01 | 50027 |
| trg_STAFF_RoleMatch | STAFF | UPDATE | BR-SCH-01 | 50028 |
| trg_CLASS_RoomCapacity | CLASS | INSERT, UPDATE | BR-SCH-02 | 50009 |
| trg_SESSION_RoomCapacity | SESSION | UPDATE | BR-SCH-02 | 50010 |
| trg_ROOM_RoomCapacity | ROOM | UPDATE | BR-SCH-02 | 50011 |
| trg_CLASS_REG_SeatLimit | CLASS_REGISTRATION | INSERT, UPDATE | BR-SCH-02 | 50012 |
| trg_SESSION_NoOverlap | SESSION | INSERT, UPDATE | BR-SCH-03, BR-SCH-04 | 50013, 50014 |
| trg_PT_BOOKING_RecheckOverlap | PT_BOOKING | UPDATE | BR-SCH-03, BR-SCH-04, BR-FAC-10 | 50015, 50016, 50029 |
| trg_CLASS_REG_ActiveMembership | CLASS_REGISTRATION | INSERT | BR-SCH-05 | 50017 |
| trg_CLASS_SessionType | CLASS | INSERT | BR-SCH-06 | 50018 |
| trg_PT_BOOKING_SessionType | PT_BOOKING | INSERT | BR-SCH-06 | 50019 |
| trg_SESSION_TypeLocked | SESSION | UPDATE | BR-SCH-06 | 50020 |
| trg_SEU_EquipmentActive | SESSION_EQUIPMENT_USAGE | INSERT | BR-FAC-06 | 50021 |
| trg_SEU_TimeCheck | SESSION_EQUIPMENT_USAGE | INSERT, UPDATE | BR-FAC-09, BR-FAC-10 | 50023, 50024 |
| trg_SESSION_ReservationsInside | SESSION | UPDATE | BR-FAC-09 | 50025 |
| trg_MLOG_SignerRole | MAINTENANCE_LOG | INSERT, UPDATE | BR-FAC-05 | 50022 |

Two examples (the full code is in script 02):

```

```

**Explanation of the schedule triggers**

The triggers below protect the schedule of rooms, trainers, classes and PT sessions. For each trigger, we explain what it blocks, how it checks the rule, and why the rule is needed.

**trg_STAFF_RoleMatch (STAFF, BR-SCH-01, error 50028)**

Stops a staff member's Role from being changed to something that no longer fits their TRAINER or RECEPTIONIST row. After every UPDATE of Role, it checks whether the person still has a row in the subclass table of the old role. It is needed because Role and the subclass tables are stored separately; without it, a Trainer could be changed to Manager and still keep a TRAINER row.

**trg_TRAINER_RoleMatch (TRAINER, BR-SCH-01, error 50026)**

Stops a TRAINER row from being created for a staff member whose Role is not 'Trainer'. After every INSERT or UPDATE, it joins the new row to STAFF and checks the Role. It is needed so that only real trainers can lead classes and PT sessions.

**trg_RECEPTIONIST_RoleMatch (RECEPTIONIST, BR-SCH-01, error 50027)**

Stops a RECEPTIONIST row from being created for a staff member whose Role is not 'Receptionist'. After every INSERT or UPDATE, it joins the new row to STAFF and checks the Role. It is needed so that only receptionists are recorded as front-desk staff.

**trg_TRAINER_Disjoint (TRAINER, BR-SCH-01, error 50007)**

Stops a receptionist from also becoming a trainer. After every INSERT or UPDATE, it looks for the same Staff_ID in RECEPTIONIST. It is needed because the two subclasses are disjoint (each person has exactly one job type), and a foreign key alone cannot enforce this.

**trg_RECEPTIONIST_Disjoint (RECEPTIONIST, BR-SCH-01, error 50008)**

Stops a trainer from also becoming a receptionist. After every INSERT or UPDATE, it looks for the same Staff_ID in TRAINER. It is the mirror of trg_TRAINER_Disjoint and keeps the two subclasses disjoint.

**trg_CLASS_RoomCapacity (CLASS, BR-SCH-02, error 50009)**

Stops a class from having a Max_Capacity larger than the Capacity of its room. After every INSERT or UPDATE, it joins CLASS to SESSION and ROOM and compares the two numbers; a class without a room (Room_ID is NULL) is skipped. It is needed so that a room is never promised more seats than it has.

**trg_SESSION_RoomCapacity (SESSION, BR-SCH-02, error 50010)**

Stops a class session from being moved to a room that is too small for the class. After an UPDATE of Room_ID, it checks that the class Max_Capacity does not exceed the Capacity of the new room. It is needed because trg_CLASS_RoomCapacity only fires when the CLASS row changes, so it would miss a change made to the SESSION row alone.

**trg_ROOM_RoomCapacity (ROOM, BR-SCH-02, error 50011)**

Stops the Capacity of a room from being lowered below the Max_Capacity of a class held in that room. After an UPDATE of Capacity, it looks for such classes through SESSION and CLASS. It is needed so that editing a room cannot silently break classes that are already planned.

**trg_CLASS_REG_SeatLimit (CLASS_REGISTRATION, BR-SCH-02, error 50012)**

Stops a registration when the class is already full. After every INSERT or UPDATE, it counts the registrations of the class that are not 'Cancelled' and compares the count with Max_Capacity. It is needed because the number of taken seats depends on other rows, so a CHECK constraint cannot test it.

**trg_CLASS_REG_ActiveMembership (CLASS_REGISTRATION, BR-SCH-05, error 50017)**

Stops a member without an Active membership from registering for a class. After every INSERT, it looks for a MEMBERSHIP row of the member with Status 'Active' whose period includes today. It is needed so that only members with a valid membership can take a seat in a class.

**trg_PT_BOOKING_ActiveMembership (PT_BOOKING, BR-SCH-05, error 50005)**

Stops a member without an Active membership from booking a PT session. It applies the same test as trg_CLASS_REG_ActiveMembership to every new PT_BOOKING row. It is needed so that personal training is only given to members whose membership is valid.

**trg_SESSION_NoOverlap (SESSION, BR-SCH-03 and BR-SCH-04, errors 50013 and 50014)**

Stops two sessions from using the same room (error 50013) or the same trainer (error 50014) at overlapping times. After every INSERT or UPDATE of the room, trainer or time, it looks for another session where S1 < E2 and S2 < E1, so sessions that only touch (one ends when the next starts) are allowed. Cancelled classes and cancelled PT bookings are ignored, and a session without a room is skipped. It is needed because a room or a trainer cannot be in two places at once.

**trg_PT_BOOKING_RecheckOverlap (PT_BOOKING, BR-SCH-03, BR-SCH-04 and BR-FAC-10, errors 50015, 50016 and 50029)**

Stops a cancelled PT booking from being restored when its room, trainer or equipment has been taken by another session in the meantime. After an UPDATE of Booking_Status from 'Cancelled' to another status, it repeats the overlap checks. It is needed because a cancelled booking is ignored by trg_SESSION_NoOverlap, so its time slot can be reused by someone else.

**trg_CLASS_SessionType (CLASS, BR-SCH-06, error 50018)**

Stops a CLASS row from being attached to a session that is not a 'Class' session, or that is already a PT booking. After every INSERT, it reads the Session_Type and looks in PT_BOOKING. It is needed so that a session is either a class or a PT session, never both.

**trg_PT_BOOKING_SessionType (PT_BOOKING, BR-SCH-06, error 50019)**

Stops a PT_BOOKING row from being attached to a session that is not a 'PT' session, or that is already a class. It works like trg_CLASS_SessionType, after every INSERT. It is needed for the same reason: a session is either a class or a PT session, never both.

**trg_SESSION_TypeLocked (SESSION, BR-SCH-06, error 50020)**

Stops Session_Type from being changed once a CLASS or PT_BOOKING row exists for the session. After an UPDATE of Session_Type, it compares the old and new value and looks for those child rows. It is needed because a changed type would leave a class attached to a PT session, or the other way round.

**trg_SESSION_ReservationsInside (SESSION, BR-FAC-09, error 50025)**

Stops a session from being moved to a time that no longer contains its equipment reservations. After an UPDATE of Start_Time or End_Time, it checks every SESSION_EQUIPMENT_USAGE row of the session. It is needed because every reservation must lie inside the session time, and the trigger on the usage table only fires when a reservation changes, not when the session moves.

**trg_SEU_EquipmentActive (SESSION_EQUIPMENT_USAGE, BR-FAC-06, error 50021)**

Stops a new reservation for equipment that is not 'Active'. After every INSERT, it joins the new usage row to EQUIPMENT and rejects the row if Operational_Status is 'Broken' or 'Under Maintenance'. It is needed so that no session is planned on a machine that cannot be used. Reservations made earlier are not changed when a machine breaks later.

**trg_SEU_TimeCheck (SESSION_EQUIPMENT_USAGE, BR-FAC-09 and BR-FAC-10, errors 50023 and 50024)**

Stops a reservation that lies outside the time of its session (error 50023) or that overlaps another reservation of the same machine (error 50024). After every INSERT or UPDATE, it compares Reserved_From and Reserved_To with the session time, then looks for other usage rows with the same Equipment_ID and overlapping times. Reservations of a cancelled class or a cancelled PT booking are ignored. It is needed so that one machine is never promised to two sessions at the same time.

**trg_MLOG_SignerRole (MAINTENANCE_LOG, BR-FAC-05, error 50022)**

Stops a maintenance log from being signed by a staff member who is not a Receptionist or a Manager. After every INSERT or UPDATE, it joins Logged_By to STAFF and checks the Role. It is needed so that every repair record is approved by a person who is allowed to do so; a Trainer cannot sign.

### 4.1.6 Script List and Run Order

Run order: 01 -> 04 -> 02 -> 03 -> 08 -> 05 -> 09 -> 10 -> 11. Scripts 06, 06b and 07 are optional (large data set). Run script 09 on the sample data only (before script 06, or after script 06b), because the results in Section 4.2.1 belong to the sample data.

| Script | Content |
|---|---|
| 01_create_tables.sql | Creates the database and the 16 tables with their primary key, foreign key, UNIQUE, CHECK and DEFAULT constraints. |
| 04_sample_data.sql | Inserts sample rows into every table, consistent with the business rules. |
| 02_triggers.sql | Creates the 24 triggers for the rules in Section 1.2. |
| 03_indexes.sql | Creates five indexes for the overlap and lookup queries. |
| 08_views.sql | Creates the 5 views of Section 4.1.3. |
| 05_trigger_tests.sql | Runs 37 tests of the business rules (triggers, CHECK constraint and unique index). Each test runs inside a transaction that is rolled back and prints PASS or FAIL: 23 invalid changes must be blocked and 14 valid changes must be accepted. |
| 09_advanced_queries.sql | Runs the 13 advanced queries of Section 4.2.1 on the sample data. |
| 10_security_rbac.sql | Creates 4 demo logins, users and roles, and the GRANT, DENY and REVOKE statements of Section 5.2. |
| 11_security_tests.sql | Runs 19 access tests as the demo users (Section 5.2.4). Each test is rolled back. |
| 06_bulk_data.sql | Adds 36,998 generated rows (IDs from 1001) to test the rules and indexes with a larger data set (Section 4.2.2). |
| 07_scale_checks.sql | Repeats six rule tests on the large data set, compares queries with and without indexes, and times a 200-row insert (Section 4.2.2). |
| 06b_remove_bulk_data.sql | Deletes the rows added by script 06 (IDs from 1001); the sample data of script 04 is kept. |

## 4.2 Advanced Queries and Performance Test Cases

### 4.2.1 Advanced Queries

Script 09 contains 13 queries that use the main techniques of the course: joins of several tables, GROUP BY with HAVING, subqueries, common table expressions (CTE), CASE, window functions and a self-join. They run on the sample data of script 04. The dates of that data are computed from the day it is loaded, so the queries use the variable @today (set to today); the results below were taken on 05/11/2026 and shift with the date on another day. The results are shown as SSMS returns them, except that times are written without seconds.

All queries run in one batch after this line:

```
SET DATEFORMAT ymd;
GO

DECLARE @today DATE = CAST(GETDATE() AS DATE);
```

#### Q1. Upcoming classes with seats taken

| | Session_ID | Class_Name | Start_Time | Room_Name | Trainer_Name | Max_Capacity | Seats_Taken |
|---|---|---|---|---|---|---|---|
| 1 | 1 | Yoga Flow | 2026-11-09 07:00:00.000 | Yoga Room | Khoa Nguyen Minh | 15 | 3 |
| 2 | 2 | HIIT Burn | 2026-11-09 18:00:00.000 | Cardio Zone | Ha Tran Thu | 3 | 3 |
| 3 | 3 | Yoga Flow | 2026-11-10 07:00:00.000 | Yoga Room | Khoa Nguyen Minh | 15 | 1 |
| 4 | 6 | Spinning | 2026-11-11 18:00:00.000 | Cardio Zone | Duc Hoang Anh | 25 | 2 |
| 5 | 9 | Pilates Intro | 2026-11-13 10:00:00.000 | NULL | Duc Hoang Anh | 15 | 0 |

#### Q2. Members with an active membership today

| | Member_ID | Member_Name | Plan_Name | End_Date | Days_Left |
|---|---|---|---|---|---|
| 1 | 4 | Dung Pham | Monthly | 2026-11-14 | 9 |
| 2 | 3 | Cuong Le | Monthly | 2026-11-24 | 19 |
| 3 | 2 | Binh Tran | Quarterly | 2027-01-03 | 59 |
| 4 | 1 | An Nguyen | Annual | 2027-09-05 | 304 |

#### Q3. Plans that earned more than 70.00 from paid membership bills

| | Plan_Name | Paid_Bills | Revenue |
|---|---|---|---|
| 1 | Annual | 1 | 280.00 |
| 2 | Quarterly | 1 | 80.00 |

#### Q4. Members who never booked a PT session

| | Member_ID | Member_Name | Registration_Date |
|---|---|---|---|
| 1 | 4 | Dung Pham | 2026-06-07 |
| 2 | 5 | Em Vo | 2026-07-07 |
| 3 | 6 | Giang Dang | 2026-11-01 |

### Q5. Fill rate of each class

| | Session_ID | Class_Name | Seats_Taken | Max_Capacity | Percent_Full | Class_Status |
|---|---|---|---|---|---|---|
| 1 | 2 | HIIT Burn | 3 | 3 | 100.0 | Full |
| 2 | 1 | Yoga Flow | 3 | 15 | 20.0 | Seats available |
| 3 | 7 | Yoga Flow | 2 | 15 | 13.3 | Seats available |
| 4 | 6 | Spinning | 2 | 25 | 8.0 | Seats available |
| 5 | 3 | Yoga Flow | 1 | 15 | 6.7 | Seats available |
| 6 | 9 | Pilates Intro | 0 | 15 | 0.0 | Seats available |

### Q6. Workload of each trainer

| | Staff_ID | Full_Name | Sessions | Class_Sessions | PT_Sessions | Teaching_Hours |
|---|---|---|---|---|---|---|
| 1 | 1 | Khoa Nguyen Minh | 4 | 3 | 1 | 4.0 |
| 2 | 2 | Ha Tran Thu | 3 | 1 | 2 | 3.0 |
| 3 | 6 | Duc Hoang Anh | 3 | 3 | 0 | 3.0 |

### Q7. Overdue bills

| | Bill_ID | Member_Name | Billing_Status | Due_Date | Days_Overdue | Suggested_Action |
|---|---|---|---|---|---|---|
| 1 | 5 | Em Vo | Failed | 2026-08-13 | 84 | Contact the member |
| 2 | 3 | Cuong Le | Pending | 2026-11-01 | 4 | Send a reminder |

### Q8. Members who joined a class in every room

| | Member_ID | Member_Name |
|---|---|---|
| 1 | 1 | An Nguyen |
| 2 | 2 | Binh Tran |
| 3 | 4 | Dung Pham |

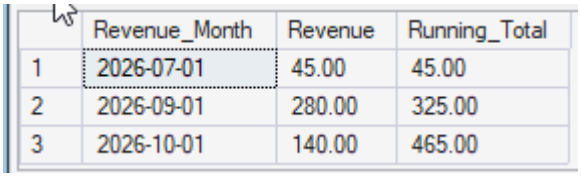
### Q9. Equipment that is not Active

| | Equipment_ID | Equipment_Name | Operational_Status | Room_Name | Last_Maintenance | Total_Cost |
|---|---|---|---|---|---|---|
| 1 | 4 | Cable Machine C-01 | Under Maintenance | PT Studio | 2026-11-02 | 120.00 |
| 2 | 5 | Rowing Machine R-01 | Broken | Cardio Zone | 2026-10-25 | 0.00 |

### Q10. Ranking of members by activity

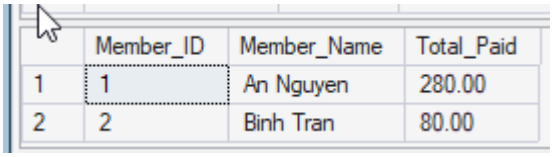
| | Member_ID | Member_Name | Activities | Activity_Rank |
|---|---|---|---|---|
| 1 | 1 | An Nguyen | 5 | 1 |
| 2 | 2 | Binh Tran | 5 | 1 |
| 3 | 3 | Cuong Le | 2 | 2 |
| 4 | 4 | Dung Pham | 2 | 2 |
| 5 | 5 | Em Vo | 0 | 3 |
| 6 | 6 | Giang Dang | 0 | 3 |

**Q11. Monthly revenue with a running total**



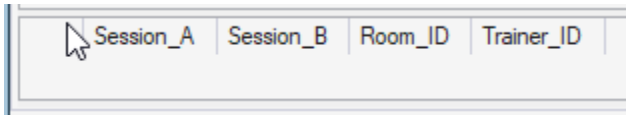
| | Revenue_Month | Revenue | Running_Total |
|---|---|---|---|
| 1 | 2026-07-01 | 45.00 | 45.00 |
| 2 | 2026-09-01 | 280.00 | 325.00 |
| 3 | 2026-10-01 | 140.00 | 465.00 |

**Q12. Members who paid more than the average**



| | Member_ID | Member_Name | Total_Paid |
|---|---|---|---|
| 1 | 1 | An Nguyen | 280.00 |
| 2 | 2 | Binh Tran | 80.00 |

**Q13. Audit: overlapping sessions**



| Session_A | Session_B | Room_ID | Trainer_ID |
|---|---|---|---|

### 4.2.2 Performance Test Cases: Larger Data Set and Index Effect

Script 07 was run on 05/11/2026 on the larger data set: script 03 (indexes) and script 06 (36,998 generated rows, IDs from 1001) were loaded first, and all 24 triggers stayed on.

Part A. Six rule tests on the large data set. Each test must end with the expected error and is rolled back afterwards.

- A1 (BR-SCH-03, room already used): error 50013, rejected as expected.
- A2 (BR-MEM-06, second overlapping Active membership): error 50001, rejected as expected.
- A3 (BR-SCH-02, class is full): error 50012, rejected as expected.
- A4 (BR-SCH-05, member has no active membership): error 50017, rejected as expected.
- A5 (BR-MEM-08, daily pass on a day with an Active membership): error 50002, rejected as expected.
- A6 (BR-FAC-06, equipment not Active): error 50021, rejected as expected.

Part B. Logical reads of the same query with the index and without it (SET STATISTICS IO; a smaller number is better).

- B1 timetable of one room for one week: 27 reads with IX_SESSION_Room_Time, 23 without.
- B2 weekly schedule of one trainer: 8 reads with IX_SESSION_Trainer_Time, 23 without (about 3 times fewer).
- B3 all classes of one member: 54 reads with IX_CLASS_REG_Member, 191 without (about 3.5 times fewer).
- B4 seats taken per class (join of 3 tables): 20 reads on CLASS and 191 on CLASS_REGISTRATION, 5 ms elapsed.

B2 and B3 show a clear gain from the index. In B1 the forced index did not help: SESSION is small (a full scan reads only 23 pages) and the index does not contain Session_Type and Trainer_ID, so each row needs an extra lookup. The numbers are reported as measured.

Part C. Inserting 200 valid sessions with all session triggers on took 79 ms (the insert is rolled back afterwards), so the rule checks stay fast on the larger data set.

## 4.3 Java Application

The application is a desktop program written in Java with Swing and JDBC (Maven build, Microsoft JDBC driver for SQL Server). It connects to the database GymPlanner_dev with the login set in the file db.properties (copied from db.properties.example; db.properties is not committed to the repository). Its purpose is to show that the rules of Section 1.2 also work when the data is changed from a program, not only from SSMS.

**Structure:** the code has four layers. The ui classes (Swing windows and dialogs) show the tables and read the input; the service classes check the input and group the SQL steps of one action into one transaction; the dao classes hold the SQL statements, written as PreparedStatement; DbConnection opens the connection. The ui classes contain no SQL.

**Screens (one tab each):**

**Equipment:** lists the machines with their status and a count per status; one dialog changes the status of a machine and records a maintenance log in the same transaction (the trigger of BR-FAC-05 checks the signer); another dialog reserves a machine for a class or PT session (the triggers of BR-FAC-06, BR-FAC-09 and BR-FAC-10 check the machine and the time).

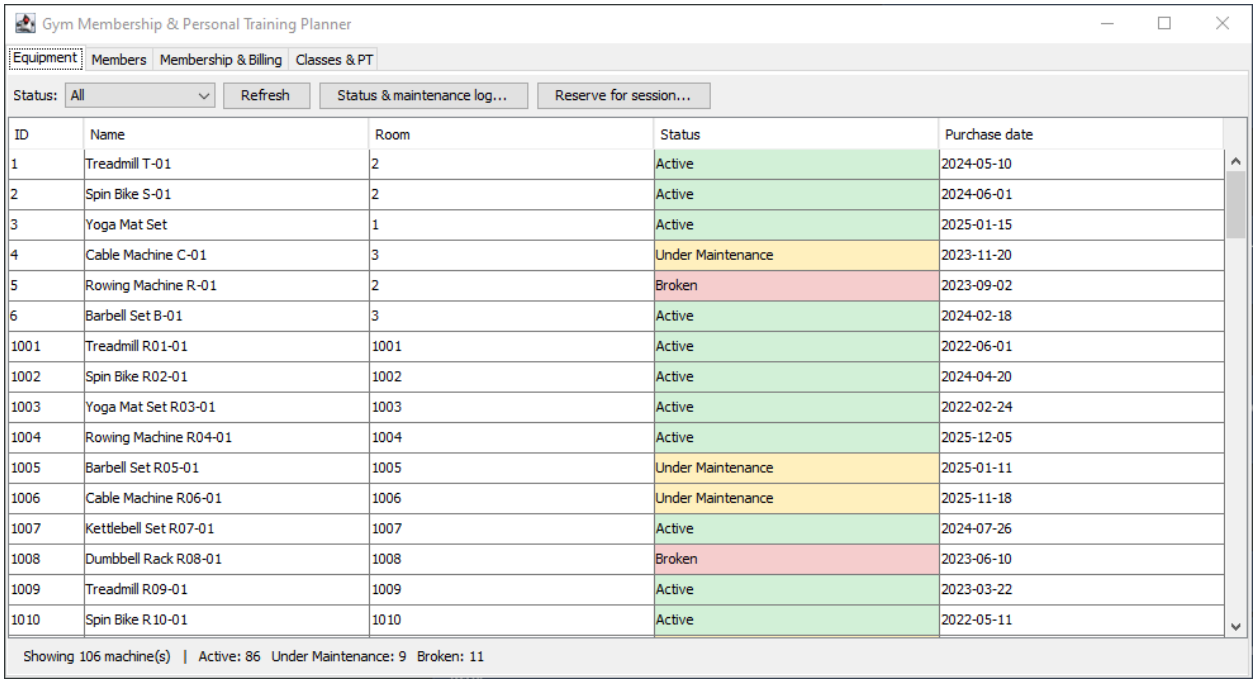
**Members:** searches members by name or phone and adds a new member.

**Membership & Billing:** three sub-tabs. Membership / Renewal buys or renews a plan (the end date is calculated from the plan duration); Daily Pass buys a daily pass; Invoices lists the bills. Each purchase creates the membership or the daily pass and its Pending bill in one transaction (BR-MEM-10).

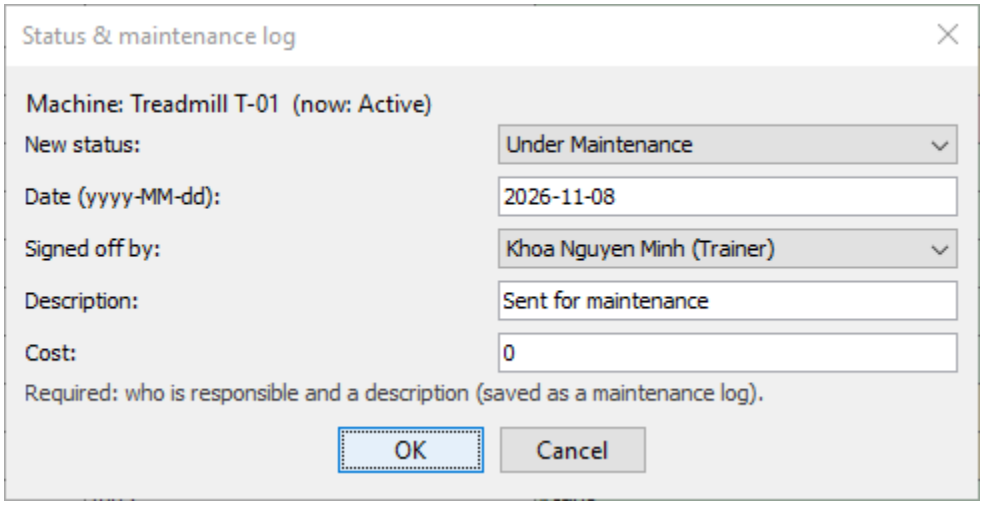
**Classes & PT:** lists the sessions; registers a member for a class; books a member into an existing PT session; creates a new PT session and its booking together in one transaction.

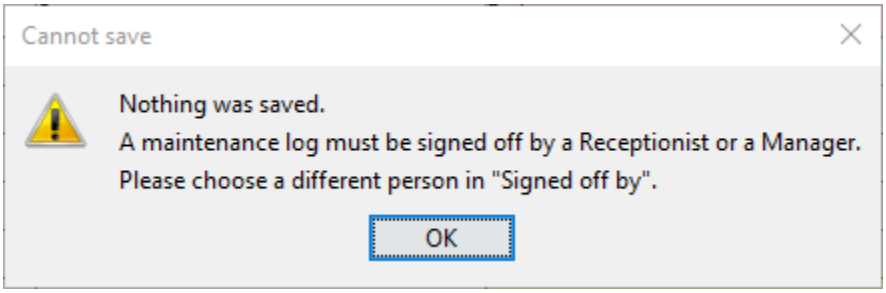
**Error messages:** when a trigger or a constraint rejects a change, SQL Server returns an error number (for example 50005, 50012, 50013, 50021 or 547). The class DbErrorMessages turns it into a short message shown in a dialog, and the transaction is rolled back, so no half-finished data remains. This is the application side of the tests of Section 5.1.

**Application Figure 1: Equipment tab with the status filter and the count per status.**

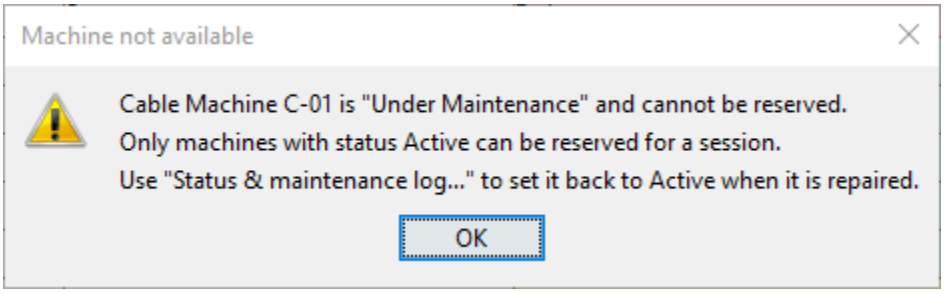



**Application Figure 2: A maintenance log signed by a Trainer is rejected (BR-FAC-05).**

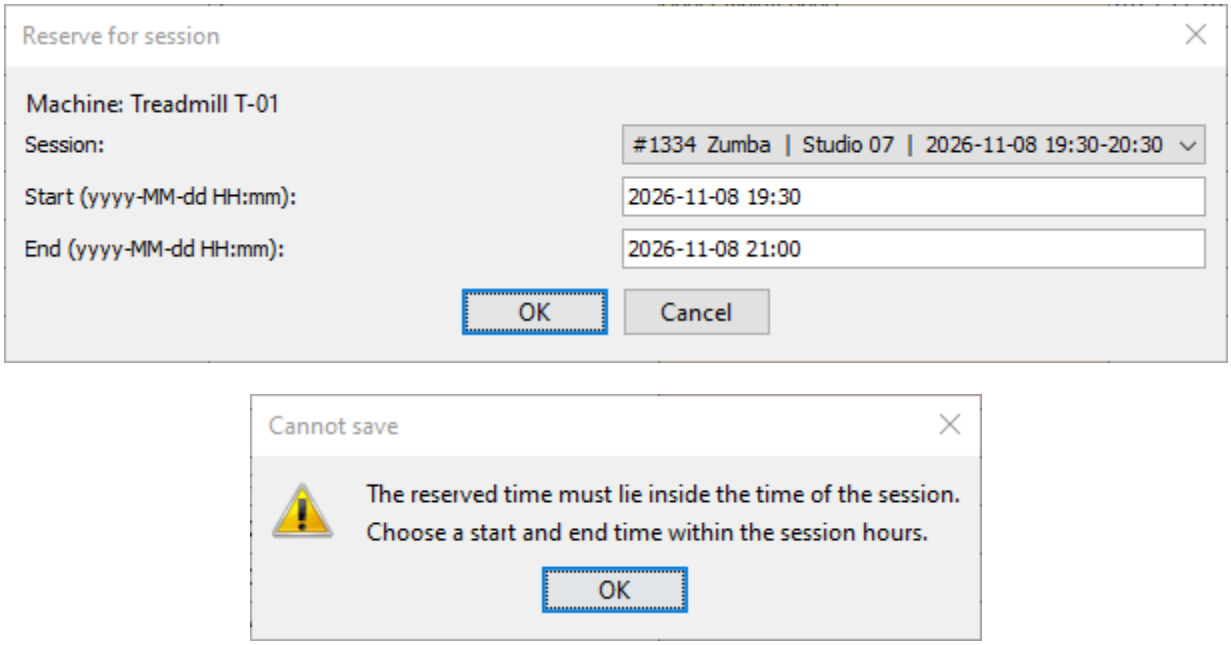


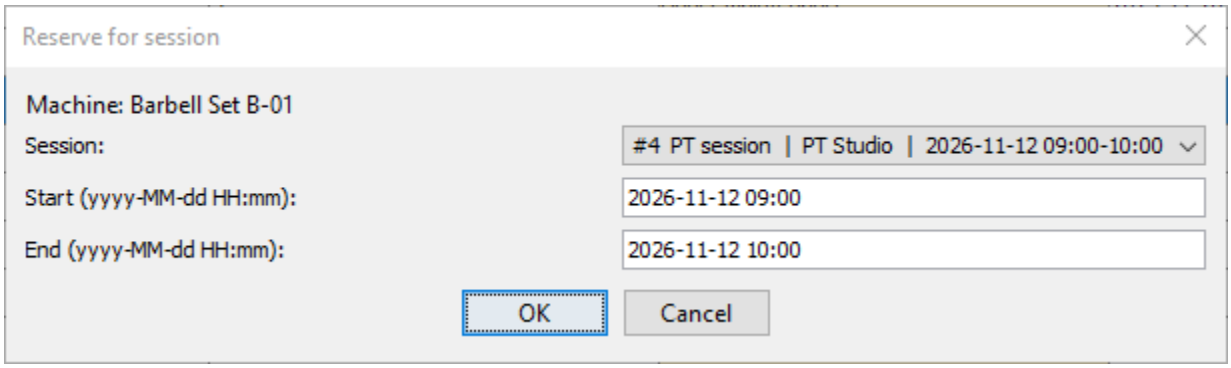
**Application Figure 3: A machine that is not Active cannot be reserved (BR-FAC-06).**

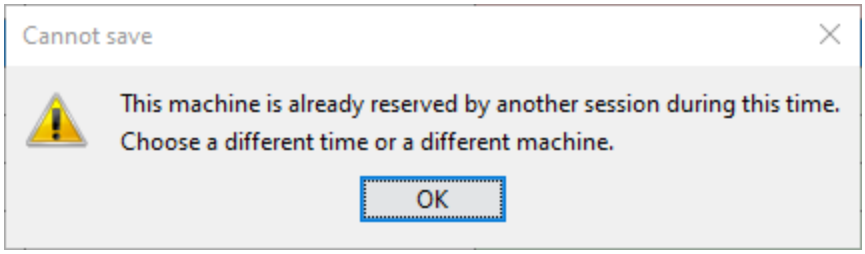



**Application Figure 4: A reservation outside the session time is rejected (BR-FAC-09).**

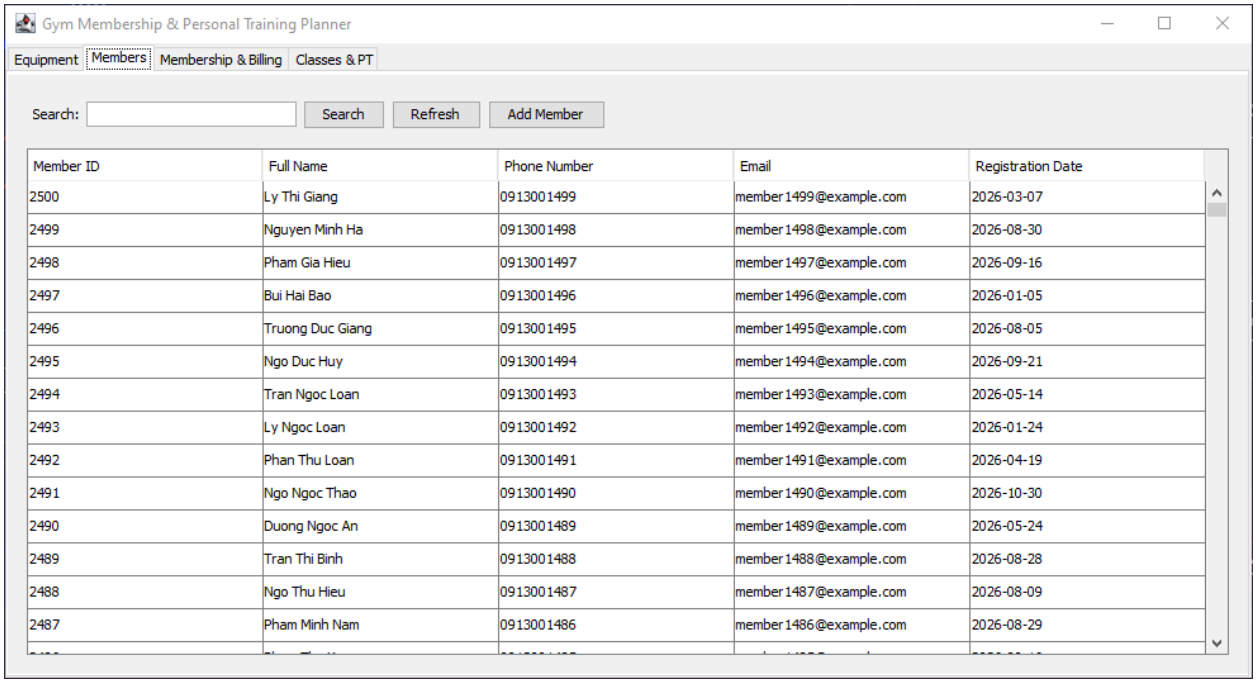



**Application Figure 5: A machine already reserved by another session at the same time is rejected (BR-FAC-10).**

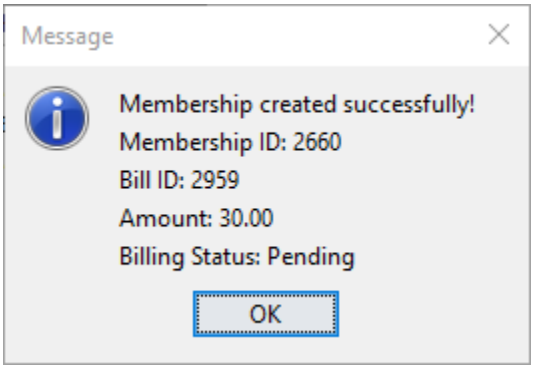




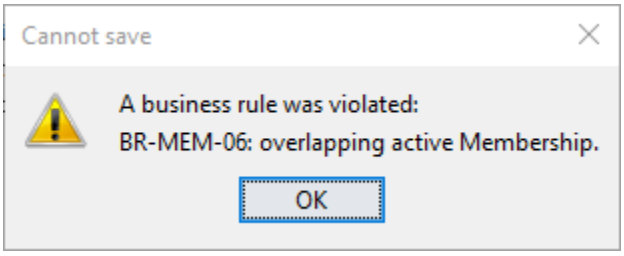

**Application Figure 6: Members tab (search and add a member).**



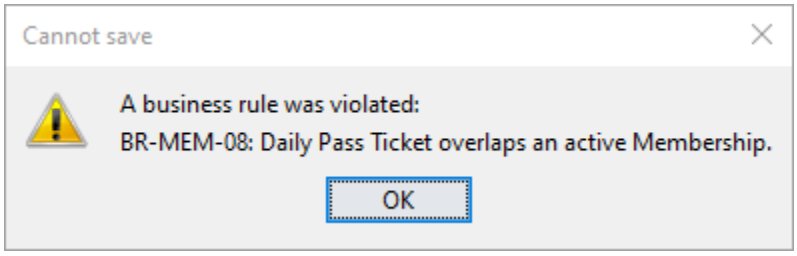

**Application Figure 7: Purchasing a membership creates the membership and a bill with status Pending (BR-MEM-10, BR-MEM-11).**



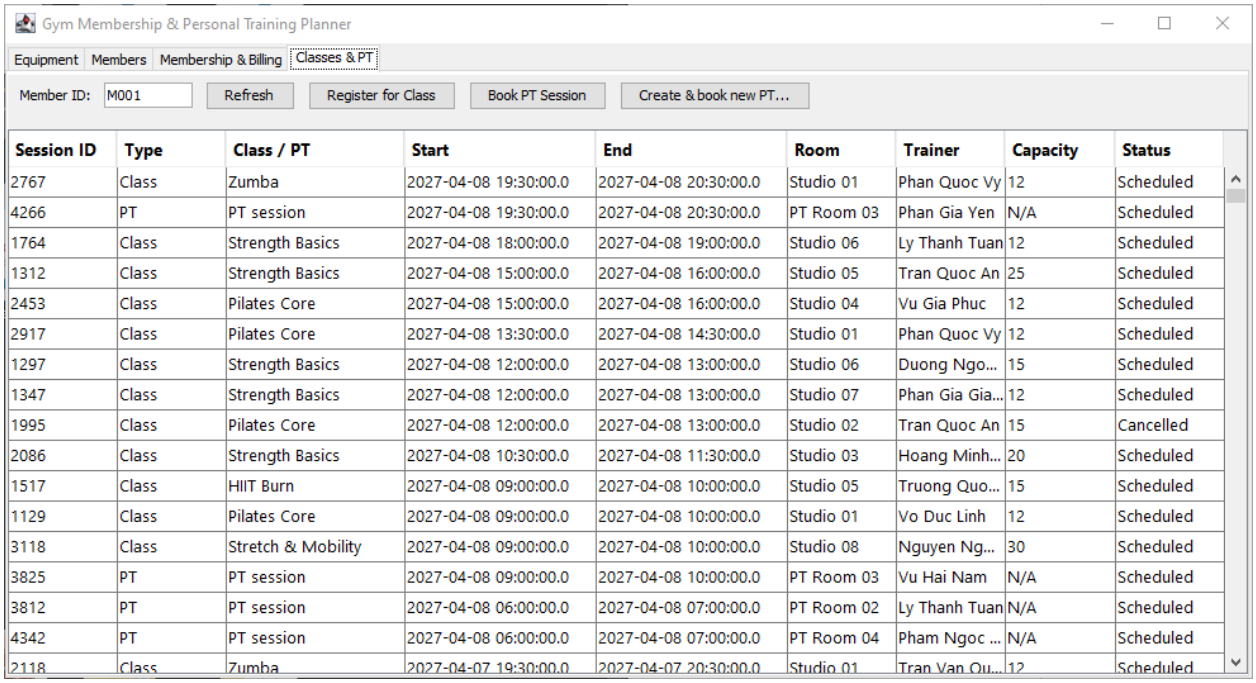
**Application Figure 8: A membership that overlaps an active membership is rejected (BR-MEM-06).**




**Application Figure 9: A daily pass that overlaps an active membership is rejected (BR-MEM-08).**

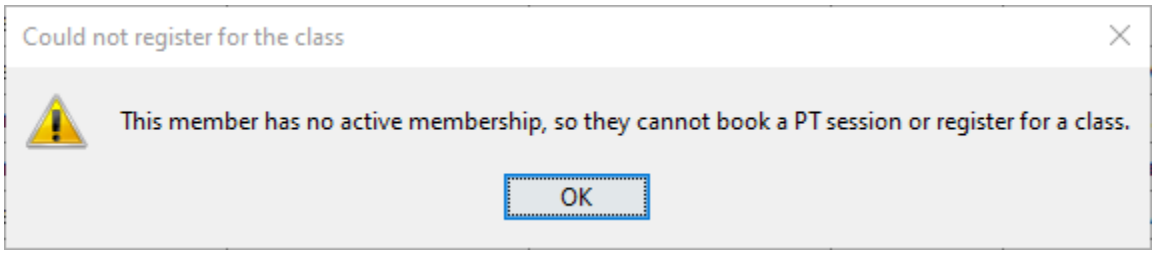



**Application Figure 10: Classes & PT tab with the list of sessions.**

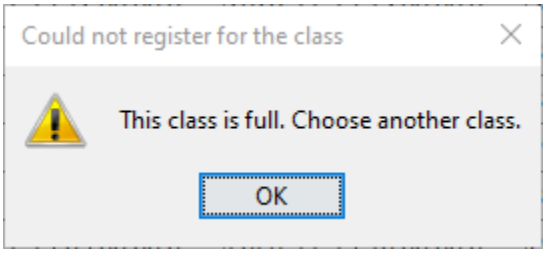



| Session ID | Type | Class / PT | Start | End | Room | Trainer | Capacity | Status |
|---|---|---|---|---|---|---|---|---|
| 2767 | Class | Zumba | 2027-04-08 19:30:00.0 | 2027-04-08 20:30:00.0 | Studio 01 | Phan Quoc Vy | 12 | Scheduled |
| 4266 | PT | PT session | 2027-04-08 19:30:00.0 | 2027-04-08 20:30:00.0 | PT Room 03 | Phan Gia Yen | N/A | Scheduled |
| 1764 | Class | Strength Basics | 2027-04-08 18:00:00.0 | 2027-04-08 19:00:00.0 | Studio 06 | Ly Thanh Tuan | 12 | Scheduled |
| 1312 | Class | Strength Basics | 2027-04-08 15:00:00.0 | 2027-04-08 16:00:00.0 | Studio 05 | Tran Quoc An | 25 | Scheduled |
| 2453 | Class | Pilates Core | 2027-04-08 15:00:00.0 | 2027-04-08 16:00:00.0 | Studio 04 | Vu Gia Phuc | 12 | Scheduled |
| 2917 | Class | Pilates Core | 2027-04-08 13:30:00.0 | 2027-04-08 14:30:00.0 | Studio 01 | Phan Quoc Vy | 12 | Scheduled |
| 1297 | Class | Strength Basics | 2027-04-08 12:00:00.0 | 2027-04-08 13:00:00.0 | Studio 06 | Duong Ngo... | 15 | Scheduled |
| 1347 | Class | Strength Basics | 2027-04-08 12:00:00.0 | 2027-04-08 13:00:00.0 | Studio 07 | Phan Gia Gia... | 12 | Scheduled |
| 1995 | Class | Pilates Core | 2027-04-08 12:00:00.0 | 2027-04-08 13:00:00.0 | Studio 02 | Tran Quoc An | 15 | Cancelled |
| 2086 | Class | Strength Basics | 2027-04-08 10:30:00.0 | 2027-04-08 11:30:00.0 | Studio 03 | Hoang Minh... | 20 | Scheduled |
| 1517 | Class | HIIT Burn | 2027-04-08 09:00:00.0 | 2027-04-08 10:00:00.0 | Studio 05 | Truong Quo... | 15 | Scheduled |
| 1129 | Class | Pilates Core | 2027-04-08 09:00:00.0 | 2027-04-08 10:00:00.0 | Studio 01 | Vo Duc Linh | 12 | Scheduled |
| 3118 | Class | Stretch & Mobility | 2027-04-08 09:00:00.0 | 2027-04-08 10:00:00.0 | Studio 08 | Nguyen Ng... | 30 | Scheduled |
| 3825 | PT | PT session | 2027-04-08 09:00:00.0 | 2027-04-08 10:00:00.0 | PT Room 03 | Vu Hai Nam | N/A | Scheduled |
| 3812 | PT | PT session | 2027-04-08 06:00:00.0 | 2027-04-08 07:00:00.0 | PT Room 02 | Ly Thanh Tuan | N/A | Scheduled |
| 4342 | PT | PT session | 2027-04-08 06:00:00.0 | 2027-04-08 07:00:00.0 | PT Room 04 | Pham Ngoc ... | N/A | Scheduled |
| 2118 | Class | Zumba | 2027-04-07 19:30:00.0 | 2027-04-07 20:30:00.0 | Studio 01 | Tran Van Qu... | 12 | Scheduled |

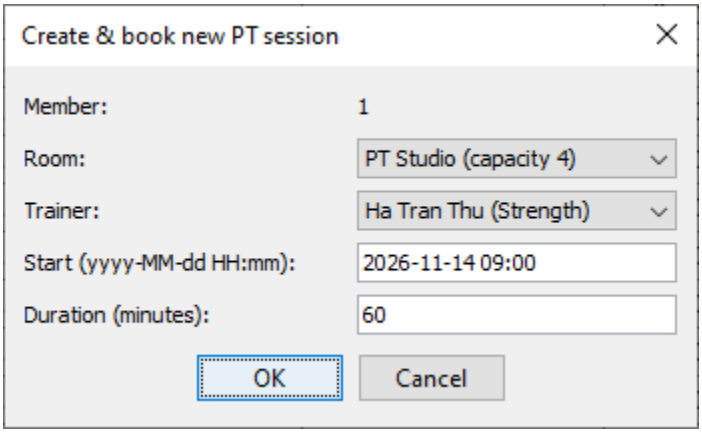
**Application Figure 11: A member without an active membership cannot register for a class (BR-SCH-05).**

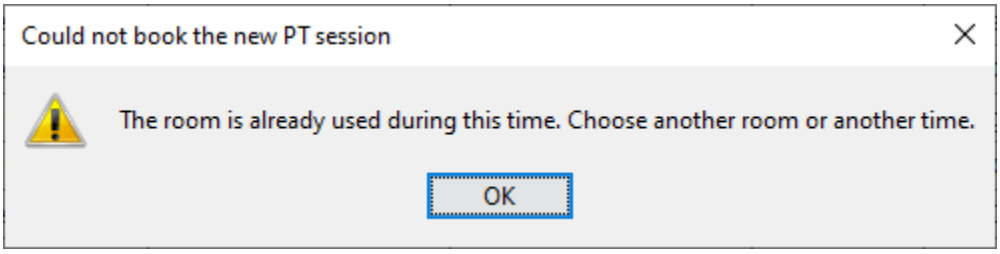



**Application Figure 12: Registration for a full class is rejected (BR-SCH-02).**




**Application Figure 13: A new PT session in a room that is already in use is rejected (BR-SCH-03).**




## 5. Verification and Security

This section shows that the database rejects bad data (Section 5.1) and that access to the data is limited by role (Section 5.2).

## 5.1 Test Cases: Constraints Prevent Bad Data

Script 05 runs 37 tests on the sample data. Tests T01 to T23 must be blocked by a trigger, a CHECK constraint or a unique index. Tests A01 to A14 are valid changes that must be accepted. On 07/11/2026 all 37 tests gave the expected result in SQL Server.

| No. | Rule | Change tried | Expected result |
|---|---|---|---|
| T01 | BR-MEM-06 | Second Active membership with overlapping dates for member 1 | Blocked: trg_MEMBERSHIP_NoActiveOverlap (50001) |
| T02 | BR-MEM-08 | Daily pass on a day when the same phone (member 1) has an Active membership | Blocked: trg_TICKET_NoActiveMembership (50002) |
| T03 | BR-MEM-08 | Active membership for member 6 that overlaps a daily pass with the same phone | Blocked: trg_MEMBERSHIP_NoOverlappingTicket (50003) |
| T04 | BR-MEM-08 | Change the phone of member 3 to the phone of a daily pass inside the membership dates | Blocked: trg_MEMBER_Phone_NoAccessOverlap (50004) |
| T05 | BR-SCH-05 | PT booking by member 5 (membership Cancelled) | Blocked: trg_PT_BOOKING_ActiveMembership (50005) |
| T06 | BR-SCH-01 | Add a TRAINER row for the manager (staff 5) | Blocked: trg_TRAINER_RoleMatch (50026) |
| T07 | BR-SCH-01 | Add a RECEPTIONIST row for trainer (staff 1) | Blocked: trg_RECEPTIONIST_Disjoint (50008) |
| T08 | BR-SCH-02 | Class with Max_Capacity 10 in PT Studio (Capacity 4) | Blocked: trg_CLASS_RoomCapacity (50009) |
| T09 | BR-SCH-02 | Lower the Capacity of room 2 to 2 while class 2 has Max_Capacity 3 | Blocked: trg_ROOM_RoomCapacity (50011) |
| T10 | BR-SCH-02 | Member 3 takes a seat in class 2, which already has 3 confirmed seats | Blocked: trg_CLASS_REG_SeatLimit (50012) |
| T11 | BR-SCH-05 | Class registration by member 5 (membership Cancelled) | Blocked: trg_CLASS_REG_ActiveMembership (50017) |
| T12 | BR-SCH-03 | New session in room 1 that overlaps session 1 | Blocked: trg_SESSION_NoOverlap (50013) |
| T13 | BR-SCH-04 | New session for trainer 1 that overlaps session 1 | Blocked: trg_SESSION_NoOverlap (50014) |
| T14 | BR-SCH-06 | CLASS row for a PT session (session 4) | Blocked: trg_CLASS_SessionType (50018) |
| T15 | BR-SCH-06 | PT booking for a Class session (session 1) | Blocked: trg_PT_BOOKING_SessionType (50019) |
| T16 | BR-SCH-06 | Change session 1 (Class) to type PT | Blocked: trg_SESSION_TypeLocked (50020) |
| T17 | BR-FAC-06 | Reserve equipment 4 (Under Maintenance) | Blocked: trg_SEU_EquipmentActive (50021) |
| T18 | BR-FAC-09 | Equipment reservation longer than its session | Blocked: trg_SEU_TimeCheck (50023) |
| T19 | BR-FAC-09 | Move session 1 so that its equipment reservation falls outside it | Blocked: trg_SESSION_ReservationsInside (50025) |
| T20 | BR-FAC-05 | Maintenance log signed by a trainer (staff 1) | Blocked: trg_MLOG_SignerRole (50022) |
| T21 | BR-MEM-09 | Bill with no source (Membership_ID and Ticket_ID both NULL) | Blocked: CK_BILL_Exclusive_Source (547) |
| T22 | BR-MEM-10 | Second bill for membership 1 | Blocked: unique index UX_BILL_Membership_NotNull (2601) |

| | | | |
|---|---|---|---|
| T23 | BR-FAC-10 | Barbell set (equipment 6) reserved twice at the same time | Blocked: trg_SEU_TimeCheck (50024) |
| A01 | BR-MEM-06 | Member 6 buys a first Active membership | Accepted |
| A02 | BR-MEM-06 | Member 4 renews on the End_Date of the Active membership (adjacent dates) | Accepted |
| A03 | BR-MEM-08 | Daily pass for a phone that has no membership | Accepted |
| A04 | BR-SCH-03 | Class session with no room yet (Room_ID NULL) | Accepted |
| A05 | BR-SCH-03 | New session in room 2 at the time of the cancelled class 10 | Accepted |
| A06 | BR-SCH-05 | PT booking by member 1 (Active membership) | Accepted |
| A07 | BR-SCH-01 | New staff member with Role Trainer, then its TRAINER row | Accepted |
| A08 | BR-SCH-02 | Class with Max_Capacity 20 in room 1 (Capacity 20) | Accepted |
| A09 | BR-SCH-04 | Two back-to-back sessions of trainer 1 (09:00-10:00 and 10:00-11:00) | Accepted |
| A10 | BR-SCH-06 | CLASS row for a session of type Class | Accepted |
| A11 | BR-FAC-06 | Active treadmill (equipment 1) reserved inside PT session 4 | Accepted |
| A12 | BR-FAC-05 | Maintenance log signed by a receptionist (staff 3) | Accepted |
| A13 | BR-FAC-05 | Maintenance log signed by a manager (staff 5) | Accepted |
| A14 | BR-FAC-10 | Equipment for a new session at the time of the cancelled class 10 | Accepted |

**How to run:** run the scripts in the order of Section 4.1.6, then script 05 in SSMS, and read the Messages tab. A line that starts with FAIL means that a rule does not work, and a line that starts with CHECK means that the change was blocked by another error number than expected. A rejected INSERT still uses up an IDENTITY number, so later IDs may skip numbers; this is normal.

# 5.2 Role-Based Access Control (RBAC)

## 5.2.1 Roles and Principles

A login is the account that a person uses to connect to SQL Server, a user is that login inside the database GymPlanner_dev, and a role is a named set of permissions. Permissions are given to roles only, so a new employee just joins a role. GRANT gives a permission, DENY forbids it (DENY always wins over GRANT), and REVOKE removes an earlier GRANT or DENY. Each role gets only what its work needs (least privilege).

| Role | Who | Permissions in short |
|---|---|---|
| role_Manager | Manager | Reads and changes all data. Cannot delete bills (DENY DELETE on BILLING_LOG) and cannot change the design (no CREATE or ALTER). |
| role_Receptionist | Front desk | Manages daily-pass guests, members, memberships, bills, class registrations and PT bookings; reads the schedule and the equipment. |

| | | |
|---|---|---|
| | | Cannot read STAFF (salaries). On bills only the payment columns can change; no deletes. |
| role_Trainer | Trainers | Reads the schedule, rooms and equipment, and the names (not the phones) of members. Marks attendance and the result of PT bookings. Cannot read bills, memberships or salaries. |
| role_Member | Members (future portal) | Reads only public information: class schedule, trainer list and membership plans. |

**Link to the application:** the three staff roles have the same names as the values of STAFF.Role (Manager, Receptionist, Trainer). role_Member is for a member portal that does not exist yet.

**Demo accounts:** script 10 creates one demo login for each role (gym_manager, gym_reception, gym_trainer, gym_member). Their passwords are for this course project only.

### 5.2.2 Permission Matrix

The matrix shows what each role may do on each table and view. S = SELECT, I = INSERT, U = UPDATE, D = DELETE; an asterisk means that only some columns are allowed (see the notes below the table); "-" means no permission.

| Object | Manager | Receptionist | Trainer | Member |
|---|---|---|---|---|
| MEMBER | SIUD | SIU | S* | - |
| DAILY_PASS_TICKET | SIUD | SIU | - | - |
| MEMBERSHIP_PLAN | SIUD | S | - | S |
| MEMBERSHIP | SIUD | SIU | - | - |
| BILLING_LOG | SIU | S I U* | - | - |
| STAFF | SIUD | - | - | - |
| TRAINER | SIUD | - | - | - |
| RECEPTIONIST | SIUD | - | - | - |
| ROOM | SIUD | S | S | - |
| SESSION | SIUD | S | S | - |
| CLASS | SIUD | S | S | - |
| CLASS_REGISTRATION | SIUD | SIU | S U* | - |
| PT_BOOKING | SIUD | SIU | S U* | - |
| EQUIPMENT | SIUD | S U* | S | - |
| MAINTENANCE_LOG | SIUD | SI | - | - |
| SESSION_EQUIPMENT_USAGE | SIUD | S | S | - |
| vw_Active_Membership | S | S | - | - |
| vw_Class_Schedule | S | S | S | S |
| vw_Member_Billing_Summary | S | S | - | - |
| vw_Trainer_Public | S | S | S | S |
| vw_Equipment_Status | S | S | - | - |

- Manager: BILLING_LOG has no D, because of DENY DELETE (a bill is never deleted).
- Trainer (restricted member data): only Member_ID and Full_Name, not Phone_Number.
- Receptionist, BILLING_LOG (U*): only Payment_Date, Amount_Paid and Billing_Status.
- Receptionist, EQUIPMENT (U*): only Operational_Status (to report a broken machine).
- Trainer, CLASS_REGISTRATION (U*): only Attendance_Status. Trainer, PT_BOOKING (U*): only Booking_Status.
- Views: a role needs only SELECT on a view; it needs no permission on the tables behind the view.

### 5.2.3 GRANT, DENY and REVOKE Statements

The statements below are taken from script 10 (the CREATE LOGIN statements, which contain the demo passwords, are left out).

```
IF DATABASE_PRINCIPAL_ID(N'gym_manager') IS NULL
        CREATE USER gym_manager   FOR LOGIN gym_manager;
IF DATABASE_PRINCIPAL_ID(N'gym_reception') IS NULL
        CREATE USER gym_reception FOR LOGIN gym_reception;
IF DATABASE_PRINCIPAL_ID(N'gym_trainer') IS NULL
        CREATE USER gym_trainer   FOR LOGIN gym_trainer;
IF DATABASE_PRINCIPAL_ID(N'gym_member') IS NULL
        CREATE USER gym_member        FOR LOGIN gym_member;
GO

IF DATABASE_PRINCIPAL_ID(N'role_Manager') IS NULL
        CREATE ROLE role_Manager;
IF DATABASE_PRINCIPAL_ID(N'role_Receptionist') IS NULL
        CREATE ROLE role_Receptionist;
IF DATABASE_PRINCIPAL_ID(N'role_Trainer') IS NULL
        CREATE ROLE role_Trainer;
IF DATABASE_PRINCIPAL_ID(N'role_Member') IS NULL
        CREATE ROLE role_Member;
GO

IF IS_ROLEMEMBER(N'role_Manager', N'gym_manager') = 0
        ALTER ROLE role_Manager ADD MEMBER gym_manager;
IF IS_ROLEMEMBER(N'role_Receptionist', N'gym_reception') = 0
        ALTER ROLE role_Receptionist ADD MEMBER gym_reception;
IF IS_ROLEMEMBER(N'role_Trainer', N'gym_trainer') = 0
        ALTER ROLE role_Trainer ADD MEMBER gym_trainer;
IF IS_ROLEMEMBER(N'role_Member', N'gym_member') = 0
        ALTER ROLE role_Member ADD MEMBER gym_member;
GO

-- role_Manager: all data, no design changes
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo TO role_Manager;
-- Bills are never deleted (the foreign keys of BILLING_LOG use ON DELETE
-- RESTRICT for the same reason). DENY wins over the GRANT above.
DENY DELETE ON dbo.BILLING_LOG TO role_Manager;
GO

-- role_Receptionist: front-desk work
GRANT SELECT, INSERT, UPDATE ON dbo.MEMBER          TO role_Receptionist;
GRANT SELECT, INSERT, UPDATE ON dbo.DAILY_PASS_TICKET   TO role_Receptionist;
GRANT SELECT, INSERT, UPDATE ON dbo.MEMBERSHIP              TO role_Receptionist;
GRANT SELECT, INSERT, UPDATE ON dbo.BILLING_LOG             TO role_Receptionist;
```

```
GRANT SELECT, INSERT, UPDATE ON dbo.CLASS_REGISTRATION  TO role_Receptionist;
GRANT SELECT, INSERT, UPDATE ON dbo.PT_BOOKING          TO role_Receptionist;
GRANT SELECT, INSERT         ON dbo.MAINTENANCE_LOG     TO role_Receptionist;
GRANT SELECT                 ON dbo.MEMBERSHIP_PLAN     TO role_Receptionist;
GRANT SELECT                 ON dbo.ROOM                TO role_Receptionist;
GRANT SELECT                 ON dbo.[SESSION]           TO role_Receptionist;
GRANT SELECT                 ON dbo.CLASS               TO role_Receptionist;
GRANT SELECT                 ON dbo.EQUIPMENT           TO role_Receptionist;
GRANT SELECT                 ON dbo.SESSION_EQUIPMENT_USAGE TO role_Receptionist;
GRANT SELECT ON dbo.vw_Active_Membership     TO role_Receptionist;
GRANT SELECT ON dbo.vw_Class_Schedule        TO role_Receptionist;
GRANT SELECT ON dbo.vw_Member_Billing_Summary TO role_Receptionist;
GRANT SELECT ON dbo.vw_Trainer_Public        TO role_Receptionist;
GRANT SELECT ON dbo.vw_Equipment_Status      TO role_Receptionist;
-- A receptionist reports a broken machine: only the status column may change.
GRANT UPDATE (Operational_Status) ON dbo.EQUIPMENT TO role_Receptionist;
GO

-- Tighten the permission on bills with REVOKE: the table-wide UPDATE is
-- removed, then only the columns that record a payment are allowed. A
-- receptionist can no longer change the invoice date, due date or owner.
REVOKE UPDATE ON dbo.BILLING_LOG FROM role_Receptionist;
GRANT UPDATE (Payment_Date, Amount_Paid, Billing_Status) ON dbo.BILLING_LOG TO role_Receptionist;
GO

-- role_Trainer: schedule and attendance
GRANT SELECT ON dbo.[SESSION]            TO role_Trainer;
GRANT SELECT ON dbo.CLASS                TO role_Trainer;
GRANT SELECT ON dbo.PT_BOOKING           TO role_Trainer;
GRANT SELECT ON dbo.CLASS_REGISTRATION           TO role_Trainer;
GRANT SELECT ON dbo.ROOM                 TO role_Trainer;
GRANT SELECT ON dbo.EQUIPMENT            TO role_Trainer;
GRANT SELECT ON dbo.SESSION_EQUIPMENT_USAGE TO role_Trainer;
GRANT SELECT ON dbo.vw_Class_Schedule            TO role_Trainer;
GRANT SELECT ON dbo.vw_Trainer_Public            TO role_Trainer;
-- A trainer sees the names of the people in a session, but not their phone.
GRANT SELECT (Member_ID, Full_Name) ON dbo.MEMBER TO role_Trainer;
-- A trainer marks attendance and the result of a PT booking, nothing else.
GRANT UPDATE (Attendance_Status) ON dbo.CLASS_REGISTRATION TO role_Trainer;
GRANT UPDATE (Booking_Status)            ON dbo.PT_BOOKING               TO role_Trainer;
GO

-- role_Member: public information only
GRANT SELECT ON dbo.vw_Class_Schedule TO role_Member;
GRANT SELECT ON dbo.vw_Trainer_Public TO role_Member;
GRANT SELECT ON dbo.MEMBERSHIP_PLAN   TO role_Member;
GO
```

### 5.2.4 Access Tests

Script 11 tests the roles. Each test pretends to be one demo user with EXECUTE AS USER, tries one action inside a transaction, and then rolls everything back. "allowed" means that the action works; "denied" means that SQL Server refuses it with error 229, 230 or 262 (permission denied). The last result grid lists every test with PASS or FAIL, and on 05/11/2026 all 19 tests passed (19 PASS, 0 FAIL).

| No. | User | Action tried | Expected | Why |
|---|---|---|---|---|
| 1 | gym_member | Read the view vw_Class_Schedule | allowed | SELECT on the view |
| 2 | gym_member | Read the view vw_Trainer_Public | allowed | SELECT on the view |
| 3 | gym_member | Read the table MEMBERSHIP_PLAN | allowed | SELECT on the table |
| 4 | gym_member | Read the table STAFF (salaries) | denied | no permission on STAFF |
| 5 | gym_member | Read the table BILLING_LOG | denied | no permission |
| 6 | gym_member | Insert a row into MEMBER | denied | no INSERT permission |
| 7 | gym_reception | Read the table MEMBER | allowed | SELECT on the table |
| 8 | gym_reception | Read the table STAFF (salaries) | denied | no permission on STAFF |
| 9 | gym_reception | Delete from MEMBERSHIP | denied | no DELETE permission |
| 10 | gym_reception | Record a payment: Billing_Status and Payment_Date | allowed | UPDATE on these columns |
| 11 | gym_reception | Change the Due_Date of a bill | denied | UPDATE only on the payment columns (REVOKE, then column GRANT) |
| 12 | gym_trainer | Read the table BILLING_LOG | denied | no permission |
| 13 | gym_trainer | Read the column MEMBER.Full_Name | allowed | column in the SELECT list |
| 14 | gym_trainer | Read the column MEMBER.Phone_Number | denied | column not in the SELECT list |
| 15 | gym_trainer | Mark attendance: update Attendance_Status | allowed | UPDATE on this column |
| 16 | gym_trainer | Change the member of a PT booking | denied | UPDATE only on Booking_Status |
| 17 | gym_manager | Update a salary in STAFF | allowed | GRANT on the schema dbo |
| 18 | gym_manager | Delete from BILLING_LOG | denied | DENY DELETE wins over the GRANT |
| 19 | gym_manager | Create a table (design change) | denied | no CREATE TABLE permission |

### 5.2.5 Views and the Java Application

**Views and permissions:** A view and its tables have the same owner (dbo), so SQL Server checks only the permission on the view (ownership chaining). This is how role_Member can read vw_Trainer_Public without any permission on STAFF.

**Java application:** The application connects with one SQL login (the login set in db.properties; the example file uses gym_dev). It shows the role of each staff member in the signer list, but it does not restrict actions by role; the rule that only a Receptionist or Manager can sign a maintenance log (BR-FAC-05) is checked by the database trigger. The four roles of this section protect direct access to the database (for example from SSMS or a report tool). To enforce role-based database permissions in the application, the system would need to authenticate users and establish database connections using credentials mapped to the appropriate database roles (see Section 6.3).

# 6. Conclusion

# 6.1 Summary of the Design

The design has 16 relations. MEMBER and DAILY_PASS_TICKET are independent strong entities; BILLING_LOG references exactly one of MEMBERSHIP and DAILY_PASS_TICKET. The finished database has 5 views, 5 extra indexes, 24 triggers for the business rules and 13 advanced queries. Script 05 ran 37 constraint tests with the expected result (23 blocked, 14 accepted), script 07 repeated rule checks and index comparisons on 36,998 generated rows, and script 11 passed all 19 access tests of the four roles (script 05 on 07/11/2026, scripts 07 and 11 on 05/11/2026).

## 6.2 Room for Extension

The table shows how some likely future changes fit into the schema. Most of them only add rows or tables; only one (several specialties or certifications per trainer) moves columns out of an existing table.

| Future change | What is needed | Existing tables altered? |
|---|---|---|
| New membership plan (e.g., 12 months) | INSERT one row into MEMBERSHIP_PLAN. | No |
| New room, machine or class time | INSERT rows into ROOM, EQUIPMENT, SESSION and CLASS. | No |
| New staff type with its own attributes | New subclass table with Staff_ID as primary key and foreign key (same pattern as TRAINER), plus Disjoint and RoleMatch triggers like those of TRAINER; trg_STAFF_RoleMatch must also be extended to check the new table. | Only CK_STAFF_Role (add the new value) |
| Several specialties or certifications per trainer | Two new relations, TRAINER_SPECIALTY and TRAINER_CERTIFICATION (Normalization Figure 8); copy the existing values into them. | TRAINER: Specialty and Certification move out |
| New session type (e.g., small-group PT) | New subclass table of SESSION and an update of the BR-SCH-06 triggers. | Only CK_SESSION_Type (add the new value) |
| New status value | Change the CHECK constraint and add the meaning to Section 3.2. | Only the CHECK constraint |

The weak point of this approach is that the fixed value lists are kept in CHECK constraints, so a new value needs an ALTER TABLE. A lookup table for each list would avoid that; the team kept the CHECK lists because each list is short and rarely changes.

## 6.3 Known Limitations

**Two bookings at the same moment:** the seat-limit and overlap triggers read other rows inside the same transaction as the insert. Under SQL Server's default READ COMMITTED isolation with locking, a second booking for the last seat waits until the first one finishes, or SQL Server ends one of them as a deadlock victim, so the limit still holds. If the database is switched to READ_COMMITTED_SNAPSHOT, both bookings could pass the check. A production system would add locking hints (UPDLOCK, HOLDLOCK) inside the triggers or run bookings at the SERIALIZABLE isolation level; this case was not tested in the project.

**Total specialization of SESSION: the database does not force a SESSION row to have a CLASS or a PT_BOOKING row, because the subtype row needs the session to exist first. Whoever inserts a session must also insert its subtype row. A session cannot be both (BR-SCH-06 triggers). STAFF -> TRAINER / RECEPTIONIST is disjoint and partial.**

**Equipment quantity:** each EQUIPMENT row is one physical machine, and the table has no quantity column. BR-FAC-10 stops the same machine (same Equipment_ID) from being reserved for overlapping times, but the database does not limit how many machines of one kind a session reserves, as long as each one is a different free machine. Items kept in sets (e.g., yoga mats) are recorded as one row per set.

**Fixed value lists:** see Section 6.2.

**Roles and the application:** the four database roles are defined and tested (Section 5.2), but the Java application connects with one SQL login and does not restrict actions by role; the maintenance-log rule (BR-FAC-05) is checked by a trigger. A later version could connect with one login per role, so that the database also enforces the permissions of the application.

**Outside the scope of this project:** the backup schedule and audit columns such as created and updated time.

# 7. References


Date, C. J., & Fagin, R. (1992). Simple conditions for guaranteeing higher normal forms in relational databases. ACM Transactions on Database Systems, 17(3), 465-476.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Holywell, S. SQL Style Guide. https://www.sqlstyle.guide/

ISO/IEC 11179-1:2015. Information technology - Metadata registries (MDR) - Part 1: Framework.

ISO/IEC/IEEE 29148:2018. Systems and software engineering - Life cycle processes - Requirements engineering.

Microsoft. SQL Server technical documentation (CREATE TRIGGER, filtered indexes, SET STATISTICS IO, CREATE VIEW, GRANT/DENY/REVOKE, ownership chains). Microsoft Learn.